

DR1_AT — Algoritmos e Estruturas de Dados

Henrique Goldstein Maluhy Mendes Amaral · Infnet · Avaliação individual

Os doze exercícios no Google Colab

Cada exercício é um notebook independente: basta abrir o link e usar **Ambiente de execução** → **Executar tudo**. Nenhum arquivo externo é necessário.

Exercício	Abrir no Colab
1 — Busca em estruturas lineares e impacto da organização	abrir no Colab
2 — Ordenações quadráticas com instrumentação e diagnóstico	abrir no Colab
3 — Otimização orientada a Big O com requisitos de desempenho	abrir no Colab
4 — Hashtable com colisões e encadeamento por lista	abrir no Colab
5 — Pilha e fila com invariantes e árvore de expressão	abrir no Colab
6 — Simulador de eventos com múltiplas filas e índice em árvore	abrir no Colab
7 — Recursão para navegação em árvore de diretórios	abrir no Colab
8 — Programação dinâmica e memoization com hashtable e árvore de subproblemas	abrir no Colab
9 — QuickSort e QuickSelect com contagem de operações e variante em lista encadeada	abrir no Colab
10 — Lista encadeada com operações completas e análise de eficiência	abrir no Colab
11 — Lista duplamente encadeada e Deque	abrir no Colab
12 — Motor de indexação e consulta com estruturas combinadas	abrir no Colab

Repositório público: <https://github.com/thxggz/dr1-at-algoritmos>

As páginas seguintes trazem os doze notebooks impressos, já executados, com as mesmas saídas que aparecem no Colab (todos os experimentos usam semente fixa).

Exercício 1 — Busca em estruturas lineares e impacto da organização

Henrique Goldstein Maluhy Mendes Amaral · DR1_AT — Algoritmos e Estruturas de Dados · Infnet

Para rodar: **Ambiente de execução** → **Executar tudo**. O notebook é independente, não precisa de nenhum arquivo externo.

Roteiro: (1) `linear_search` e `binary_search` com contagem de comparações · (2) gerador de vetores e medição em 5 escalas · (3) pré-condição da busca binária e estratégia de falha rápida · (4) `SinglyLinkedList` mínima e comparação com o array · (5) casos de borda

In [1]:

```
import random
import sys
import time

import matplotlib.pyplot as plt
import pandas as pd

# mesma semente = mesmos números em toda execução
random.seed(42)
```

1. Busca linear e busca binária

As duas funções devolvem uma tupla `(posição, comparações)`. A posição é `-1` quando o valor não está no vetor.

Na busca binária eu conto **uma comparação por iteração**: comparar o alvo com o elemento do meio decide entre os três casos (achou, vai para a direita, vai para a esquerda).

In [2]:

```
class VetorNaoOrdenadoError(Exception):
    """A busca binária recebeu um vetor que não está ordenado."""

def linear_search(arr, target):
    comparacoes = 0
    for i in range(len(arr)):
        comparacoes += 1
        if arr[i] == target:
            return i, comparacoes
    return -1, comparacoes

def binary_search(sorted_arr, target, verificacao="amostra"):
    # pré-condição explícita: sem vetor ordenado a resposta da binária não vale nada
    verificar_ordenacao(sorted_arr, verificacao)

    comparacoes = 0
    inicio, fim = 0, len(sorted_arr) - 1
    while inicio <= fim:
        meio = (inicio + fim) // 2
        comparacoes += 1
        if sorted_arr[meio] == target:
            return meio, comparacoes
        elif sorted_arr[meio] < target:
            inicio = meio + 1 # o alvo só pode estar na metade da direita
        else:
```

```
fim = meio - 1 # o alvo só pode estar na metade da esquerda
return -1, comparacoes
```

2. Pré-condição da busca binária e falha rápida

A binária só funciona em vetor ordenado. Se eu rodar num vetor bagunçado ela não dá erro — ela só **devolve uma resposta errada**, o que é pior. Por isso existe a verificação.

Três modos:

modo	custo	o que garante
"completa"	$O(n)$ — confere todos os pares vizinhos	certeza
"amostra"	$O(k)$ — confere só k pares vizinhos sorteados	falha rápida : se achar um par fora de ordem, o vetor com certeza não está ordenado
"nenhuma"	$O(1)$	nada — para quando quem chama já garantiu a ordem

Ordenar o vetor antes custaria $O(n \log n)$. A verificação completa já custa $O(n)$, o que **anula a vantagem** da busca $O(\log n)$. A amostra custa $O(k)$ com k fixo, então a busca continua $O(\log n)$.

Comportamento na falha: levanta `VetorNaoOrdenadoError`. Não devolve `-1`, porque `-1` quer dizer "não achei" e o problema aqui é outro: a entrada é inválida.

In [3]:

```
AMOSTRA_PADRAO = 20 # quantos pares vizinhos a verificação por amostra confere

def verificar_ordenacao(arr, modo="amostra", k=AMOSTRA_PADRAO):
    """Levanta VetorNaoOrdenadoError se achar um par vizinho fora de ordem.
    Devolve quantas comparações gastou."""
    n = len(arr)
    if modo == "nenhuma" or n < 2:
        return 0

    if modo == "completa":
        posicoes = range(n - 1)
    elif modo == "amostra":
        # sorteio k posições; o par conferido é (i, i+1)
        posicoes = [random.randrange(n - 1) for _ in range(min(k, n - 1))]
    else:
        raise ValueError(f"modo de verificação desconhecido: {modo}")

    comparacoes = 0
    for i in posicoes:
        comparacoes += 1
        if arr[i] > arr[i + 1]:
            raise VetorNaoOrdenadoError(
                f"posição {i} ({arr[i]}) é maior que a posição {i + 1} ({arr[i + 1]})"
            )
    return comparacoes
```

3. Gerador de vetores

Valores distintos de `0` a `n-1`, então sempre sei em que posição cada valor está.

In [4]:

```
def gerar_vetor(n, cenario):
    if cenario == "ordenado":
        return list(range(n))
    if cenario == "reverso":
        return list(range(n - 1, -1, -1))
    if cenario == "aleatorio":
        vetor = list(range(n))
        random.shuffle(vetor)
        return vetor
    raise ValueError(f"cenário desconhecido: {cenario}")

# conferindo o gerador e as duas buscas antes de medir qualquer coisa
v = gerar_vetor(10, "ordenado")
assert v == [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
assert gerar_vetor(5, "reverso") == [4, 3, 2, 1, 0]
assert sorted(gerar_vetor(100, "aleatorio")) == list(range(100))

assert linear_search(v, 7) == (7, 8) # achou na posição 7, depois de 8 comparações
assert linear_search(v, 99) == (-1, 10) # não achou: comparou com todos
assert binary_search(v, 7)[0] == 7
assert binary_search(v, 99)[0] == -1
print("buscas e gerador conferidos")
```

buscas e gerador conferidos

4. Medição em 5 escalas (10^2 a 10^6)

Para cada tamanho e cenário eu meço:

- **linear, melhor caso:** o alvo é o primeiro elemento
- **linear, caso médio:** média de 10 alvos presentes sorteados
- **linear, pior caso:** alvo ausente — compara com todos
- **binária, pior caso:** alvo ausente, no vetor ordenado (nos outros cenários a pré-condição falha, e isso também vai para a tabela)

In [5]:

```
TAMANHOS = [10**2, 10**3, 10**4, 10**5, 10**6]
CENARIOS = ["ordenado", "reverso", "aleatorio"]

linhas = []
for n in TAMANHOS:
```

```
for cenario in CENARIOS:
    vetor = gerar_vetor(n, cenario)

    _, melhor = linear_search(vetor, vetor[0])
    # alvo ausente: n nunca está no vetor (os valores vão de 0 a n-1).
    # Uso um valor MAIOR que todos porque é o que leva a binária ao pior caso:
    # o meio arredonda para baixo, então a metade da direita é a maior.
    _, pior = linear_search(vetor, n)
    amostra = [random.randrange(n) for _ in range(10)]
    medio = sum(linear_search(vetor, alvo)[1] for alvo in amostra) / len(amostra)

    try:
        _, binaria = binary_search(vetor, n)
    except VetorNaoOrdenadoError:
        binaria = "pré-condição falhou"

    linhas.append({
        "n": n,
        "cenario": cenario,
        "linear_melhor": melhor,
        "linear_medio": round(medio),
        "linear_pior": pior,
        "linear_pior / n": pior / n,
        "binaria_pior": binaria,
    })

tabela = pd.DataFrame(linhas)
tabela
```

Out[5]:

	n	cenario	linear_melhor	linear_medio	linear_pior	linear_pior / n	binaria_pior
0	100	ordenado	1	47	100	1.0	7
1	100	reverso	1	56	100	1.0	pré-condição falhou
2	100	aleatorio	1	54	100	1.0	pré-condição falhou
3	1000	ordenado	1	463	1000	1.0	10
4	1000	reverso	1	528	1000	1.0	pré-condição falhou
5	1000	aleatorio	1	549	1000	1.0	pré-condição falhou
6	10000	ordenado	1	5397	10000	1.0	14
7	10000	reverso	1	6152	10000	1.0	pré-condição falhou
8	10000	aleatorio	1	4738	10000	1.0	pré-condição falhou
9	100000	ordenado	1	57230	100000	1.0	17
10	100000	reverso	1	59810	100000	1.0	pré-condição falhou
11	100000	aleatorio	1	63407	100000	1.0	pré-condição falhou
12	1000000	ordenado	1	647006	1000000	1.0	20
13	1000000	reverso	1	523128	1000000	1.0	pré-condição falhou
14	1000000	aleatorio	1	612982	1000000	1.0	pré-condição falhou

O que a tabela mostra:

- Linear, pior caso: `linear_pior / n` é exatamente **1,0** nas cinco escalas — o custo cresce na mesma proporção que `n`. É a assinatura de **O(n)**.
- Linear, melhor caso: **1 comparação** em qualquer tamanho — O(1).
- Linear, caso médio: fica perto de `n/2` (depende de onde os 10 alvos sorteados caíram), então também é O(n).
- A busca linear **não liga para a ordem**: os três cenários dão o mesmo pior caso.
- A binária só roda no cenário ordenado. Nos outros dois a verificação detecta o problema antes, em vez de devolver uma resposta errada.

In [6]:

```
ordenado = tabela[tabela["cenario"] == "ordenado"].copy()
ordenado["log2(n)"] = [int(n).bit_length() for n in ordenado["n"]] # bit_length(n) = /log2 n/ + 1

print("busca binária no vetor ordenado, pior caso (alvo ausente):\n")
for _, linha in ordenado.iterrows():
    assert linha["binaria_pior"] == linha["log2(n)"] # bate exatamente com a teoria
    print(f"  n = {linha['n']:>9},  comparações = {linha['binaria_pior']:>2}"
          f"      log2 n+1 = {linha['log2(n)']:>2}  linear faria {linha['linear_pior']:>9},")
```

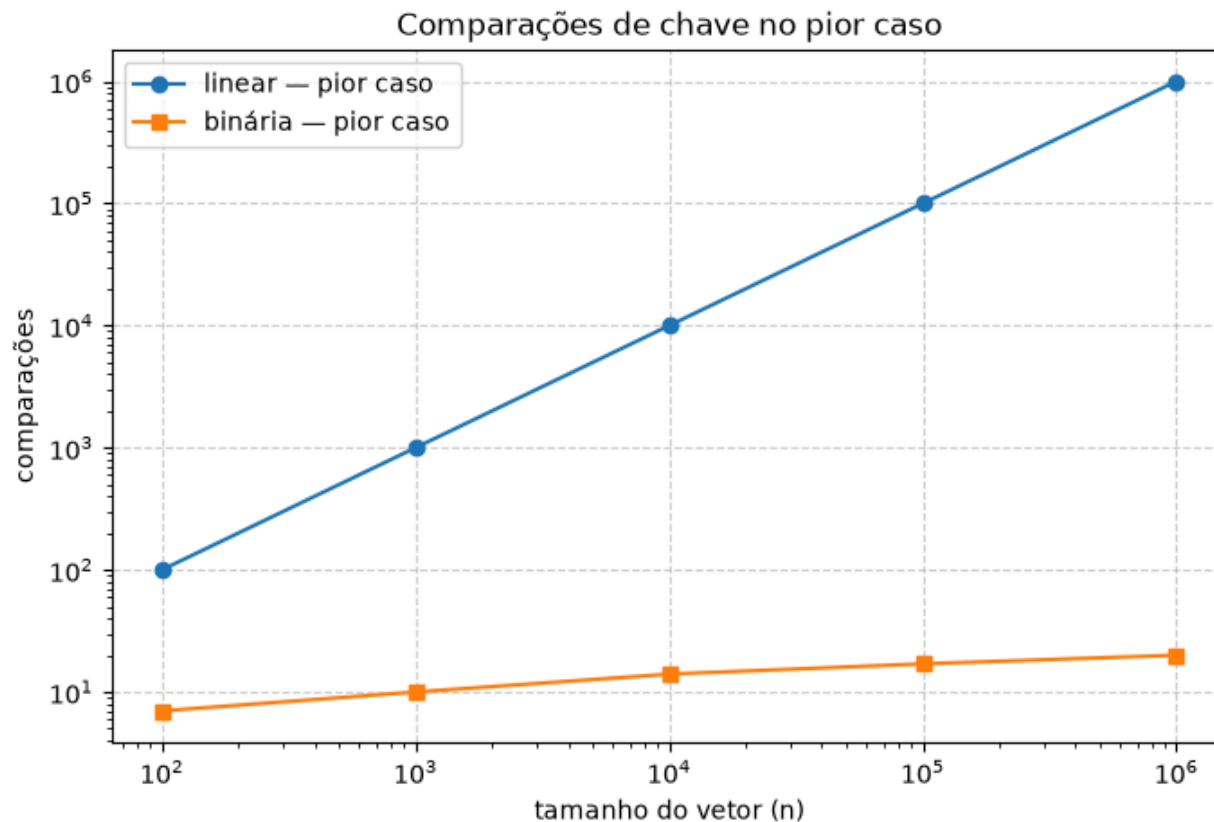
busca binária no vetor ordenado, pior caso (alvo ausente):

n =	100	comparações =	7	log2 n +1 =	7	linear faria	100
n =	1,000	comparações =	10	log2 n +1 =	10	linear faria	1,000
n =	10,000	comparações =	14	log2 n +1 =	14	linear faria	10,000
n =	100,000	comparações =	17	log2 n +1 =	17	linear faria	100,000
n =	1,000,000	comparações =	20	log2 n +1 =	20	linear faria	1,000,000

A binária faz no máximo **$\lfloor \log_2 n \rfloor + 1$** comparações: cada iteração descarta metade do que sobrou. Quando n cresce 10.000 vezes (de 10^2 para 10^6), a binária passa de 7 para 20 comparações, enquanto a linear passa de 100 para 1.000.000.

In [7]:

```
plt.figure(figsize=(8, 5))
plt.plot(ordinado["n"], ordenado["linear_pior"], marker="o", label="linear - pior caso")
plt.plot(ordinado["n"], ordenado["binaria_pior"], marker="s", label="binária - pior caso")
plt.xscale("log")
plt.yscale("log")
plt.title("Comparações de chave no pior caso")
plt.xlabel("tamanho do vetor (n)")
plt.ylabel("comparações")
plt.grid(True, linestyle="--", alpha=0.6)
plt.legend()
plt.show()
```



No gráfico em escala log-log a linear é uma **reta de inclinação 1** ($O(n)$); a binária quase não sai do lugar ($O(\log n)$).

5. A falha rápida funciona? E o risco de falso positivo

A verificação por amostra tem um ponto fraco que precisa ficar claro: ela **nunca acusa um vetor ordenado de estar bagunçado**, mas pode deixar passar um vetor bagunçado. Isso acontece quando nenhum dos k pares sorteados está fora de ordem — o **falso positivo** (dizer "está ordenado" sem estar).

O risco depende de quantos pares vizinhos estão fora de ordem:

- **aleatório**: cerca de metade dos pares está fora de ordem, então passar batido tem chance de uns $(1/2)^k$ — com $k = 20$, praticamente zero;
- **reverso**: todos os pares estão fora de ordem, pega na primeira comparação;
- **quase ordenado com uma única troca**: só 1 par em $n-1$ está errado, chance de pegar $\approx k/(n-1)$. **Esse é o caso perigoso.**

```
def quase_ordenado(n):
    vetor = list(range(n))
    i = random.randrange(n - 1)
    vetor[i], vetor[i + 1] = vetor[i + 1], vetor[i] # uma única troca de vizinhos
    return vetor

n = 10_000
tentativas = 500
casos = {
    "aleatorio": lambda: gerar_vetor(n, "aleatorio"),
    "reverso": lambda: gerar_vetor(n, "reverso"),
    "quase ordenado (1 troca)": lambda: quase_ordenado(n),
}

resultado = []
for nome, fabrica in casos.items():
    detectou = 0
    for _ in range(tentativas):
        try:
            verificar_ordenacao(fabrica(), "amostra")
        except VetorNaoOrdenadoError:
            detectou += 1
    resultado.append({
        "cenario": nome,
        "detectou": f"{detectou}/{tentativas}",
        "taxa_de_deteccao": detectou / tentativas,
        "falso_positivo": 1 - detectou / tentativas,
    })

print(f"n = {n:}, k = {AMOSTRA_PADRAO} pares sorteados, {tentativas} vetores por cenário")
print(f"previsão para 1 troca: k/(n-1) = {AMOSTRA_PADRAO / (n - 1):.4f}\n")
pd.DataFrame(resultado)
```

n = 10,000, k = 20 pares sorteados, 500 vetores por cenário
 previsão para 1 troca: k/(n-1) = 0.0020

Out[8]:

	cenario	detectou	taxa_de_deteccao	falso_positivo
0	aleatorio	500/500	1.000	0.000
1	reverso	500/500	1.000	0.000
2	quase ordenado (1 troca)	1/500	0.002	0.998

Conclusão sobre a falha rápida: ela cumpre o papel — é $O(k)$ em vez de $O(n \log n)$, e pega na hora os casos grosseiros (aleatório e reverso). Mas no vetor com **uma única troca** ela quase sempre deixa passar, e a taxa de detecção fica perto da previsão $k/(n-1)$.

Por isso a amostra serve como **proteção barata contra erro grosseiro**, não como garantia. Quando a ordenação precisa ser certa (por exemplo, o vetor veio de fora do sistema), o certo é usar `verificacao="completa"` e aceitar o $O(n)$ — ou manter a garantia na origem, ordenando uma vez só na hora de construir o vetor.

6. A mesma busca linear numa `SinglyLinkedList`

Lista mínima, como pede o enunciado: o nó tem só `value` e `next`.

In [9]:

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None  # guardo o último nó para inserir no fim sem percorrer tudo

    def insert_last(self, value):
        novo = Node(value)
        if self.head is None:
            self.head = novo
        else:
            self.tail.next = novo
            self.tail = novo

def linear_search_linked(lista, target):
    comparacoes = 0
    posicao = 0
    atual = lista.head
    while atual is not None:
        comparacoes += 1
        if atual.value == target:
            return posicao, comparacoes
        atual = atual.next  # sem índice: só dá para andar de nó em nó
        posicao += 1
    return -1, comparacoes

def lista_de(vetor):
    lista = SinglyLinkedList()
    for valor in vetor:
        lista.insert_last(valor)
    return lista

# mesma resposta que a versão em array
lista = lista_de([5, 3, 8])
assert linear_search_linked(lista, 8) == (2, 3)
assert linear_search_linked(lista, 9) == (-1, 3)
assert linear_search_linked(SinglyLinkedList(), 1) == (-1, 0)
print("busca linear na lista conferida")
```

busca linear na lista conferida

Array × lista encadeada, com os mesmos dados

Mesmo vetor, mesmos alvos. Comparo as **comparações** (que devem ser iguais) e o **tempo** e a **memória** (que não são).

In [10]:

```
def tamanho_em_bytes(lista):
    # soma o tamanho de cada nó; o valor em si é o mesmo objeto nas duas estruturas
    total = sys.getsizeof(lista)
    atual = lista.head
    while atual is not None:
        total += sys.getsizeof(atual) + sys.getsizeof(atual.__dict__)
        atual = atual.next
    return total

comparacao = []
for n in [10**3, 10**4, 10**5]:
    vetor = gerar_vetor(n, "aleatorio")
    lista = lista_de(vetor)
    alvos = [random.randrange(n) for _ in range(20)] + [-1] # 20 presentes + 1 ausente

    comp_array = comp_lista = 0
    inicio = time.perf_counter()
    for alvo in alvos:
        comp_array += linear_search(vetor, alvo)[1]
    tempo_array = time.perf_counter() - inicio

    inicio = time.perf_counter()
    for alvo in alvos:
        comp_lista += linear_search_linked(lista, alvo)[1]
    tempo_lista = time.perf_counter() - inicio

    comparacao.append({
        "n": n,
        "comparacoes_array": comp_array,
        "comparacoes_lista": comp_lista,
        "iguais?": comp_array == comp_lista,
        "ns_por_comparacao_array": tempo_array / comp_array * 1e9,
        "ns_por_comparacao_lista": tempo_lista / comp_lista * 1e9,
        "memoria_array_KB": sys.getsizeof(vetor) / 1024,
        "memoria_lista_KB": tamanho_em_bytes(lista) / 1024,
    })

df = pd.DataFrame(comparacao)
df["lista/array (memória)"] = df["memoria_lista_KB"] / df["memoria_array_KB"]
df
```

Out[10]:

	n	comparacoes_array	comparacoes_lista	iguais?	ns_por_comparacao_array	ns_por_comparacao_lista	memoria_array_KB	memoria_lista_KB	lista/array (memória)
0	1000	9271	9271	True	41.322403	45.464351	7.867188	132.859375	16.887786
1	10000	108754	108754	True	35.513177	46.365191	78.179688	1328.171875	16.988708
2	100000	923397	923397	True	52.838378	54.418522	781.304688	13281.296875	16.998870

A diferença prática:

- **Comparações: exatamente iguais** (coluna `iguais?`). O algoritmo é o mesmo — olhar um elemento de cada vez até achar. Em Big O as duas são **O(n)**.

- **Memória: a lista ocupa cerca de 17 vezes mais**, e essa razão fica estável nas três escalas. No array, cada posição guarda só uma referência de 8 bytes, lado a lado. Na lista, cada valor vira um **objeto** `Node` **inteiro**, com cabeçalho, o `value` e o ponteiro `next`. Essa é a diferença prática mais sólida da tabela.
- **Tempo por comparação: não dá para cravar um vencedor em Python**. Eu esperava a lista mais lenta — em C, andar por um bloco contínuo aproveita o cache do processador, e seguir ponteiro não. Mas a medição não mostra isso de forma consistente: aqui o custo de cada passo é dominado pelo próprio interpretador (no array, `arr[i]` ainda passa por `range` e por indexação), e isso esconde o efeito de cache. Por isso eu não baseio nenhuma conclusão no tempo: a evidência que não depende da máquina é a coluna de comparações.

Conclusão: a organização dos dados não muda a **classe** da busca linear — as duas são $O(n)$ com o mesmo número de comparações. O que muda é o **custo de memória**. O que mudaria a classe é a **ordenação**: com vetor ordenado e acesso por índice, a busca cai de $O(n)$ para $O(\log n)$ — coisa que a lista encadeada não consegue fazer, porque não tem acesso direto ao meio.

7. Casos de borda

In [11]:

```
# vetor vazio
assert linear_search([], 5) == (-1, 0)
assert binary_search([], 5) == (-1, 0)

# um elemento só
assert linear_search([7], 7) == (0, 1)
assert binary_search([7], 7) == (0, 1)
assert binary_search([7], 3) == (-1, 1)

# valor repetido: a linear devolve a PRIMEIRA ocorrência
assert linear_search([4, 9, 4, 9], 9) == (1, 2)

# alvo nas pontas do vetor ordenado
v = list(range(1000))
assert binary_search(v, 0)[0] == 0
assert binary_search(v, 999)[0] == 999

# vetor bagunçado: a verificação completa sempre pega
try:
    binary_search([1, 2, 5, 4, 6], 4, verificacao="completa")
    print("ERRO: deveria ter falhado")
except VetorNaoOrdenadoError as erro:
    print(f"vetor não ordenado detectado → {erro}")

# modo de verificação inválido também é tratado
try:
    binary_search([1, 2, 3], 2, verificacao="rapida")
except ValueError as erro:
    print(f"modo inválido → {erro}")

print("\ntodos os casos de borda passaram")
```

vetor não ordenado detectado → posição 2 (5) é maior que a posição 3 (4)
modo inválido → modo de verificação desconhecido: rapida

todos os casos de borda passaram

Resumo

operação	melhor caso	caso médio	pior caso
<code>linear_search</code> (array)	$O(1)$	$O(n)$	$O(n)$
<code>linear_search_linked</code> (lista)	$O(1)$	$O(n)$	$O(n)$
<code>binary_search</code> (array ordenado)	$O(1)$	$O(\log n)$	$O(\log n)$
verificação completa	—	$O(n)$	$O(n)$
verificação por amostra	—	$O(k)$	$O(k)$

Todas as afirmações acima estão sustentadas pelas tabelas deste notebook: a razão `linear_pior / n` fixa em 1,0 nas cinco escalas, a binária batendo com $\lfloor \log_2 n \rfloor + 1$, e as comparações idênticas entre array e lista.

Exercício 2 — Ordenações quadráticas com instrumentação e diagnóstico

Henrique Goldstein Maluhy Mendes Amaral · DR1_AT — Algoritmos e Estruturas de Dados · Infnet

Para rodar: **Ambiente de execução** → **Executar tudo**. O notebook é independente, não precisa de nenhum arquivo externo.

Roteiro: (1) bubble, selection e insertion contando comparações e cópias · (2) experimento com 4 padrões × 4 tamanhos · (3) quem ganha no "quase ordenado" e por quê · (4) BST para validar a ordenação e comparação de custo com o insertion · (5) remoção na BST · (6) casos de borda

In [1]:

```
import random
import matplotlib.pyplot as plt
import pandas as pd
random.seed(42)
```

1. Os três algoritmos

Os três trabalham numa **cópia** da lista (a entrada não é alterada) e devolvem `(lista_ordenada, comparacoes, copias)`.

Regras de contagem, iguais para os três:

- **comparação** = cada vez que comparo duas chaves da lista;
- **cópia** = cada vez que um valor é escrito numa posição. Uma **troca vale 3 cópias** (`tmp = a; a = b; b = tmp`), como pede o enunciado.

In [2]:

```
def bubble_sort(lista):
    lista = lista.copy()
    n = len(lista)
    comparacoes = copias = 0
    for i in range(n):
        trocou = False
        # a cada passada o maior elemento que sobrou "borbulha" para o fim
        for j in range(0, n - i - 1):
            comparacoes += 1
            if lista[j] > lista[j + 1]:
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
                copias += 3
                trocou = True
        # se uma passada inteira não trocou nada, a lista já está ordenada
        if not trocou:
            break
    return lista, comparacoes, copias

def selection_sort(lista):
    lista = lista.copy()
    n = len(lista)
    comparacoes = copias = 0
    for i in range(n):
        # procura o menor elemento na parte que ainda não foi ordenada
        min_index = i
        for j in range(i + 1, n):
            comparacoes += 1
            if lista[j] < lista[min_index]:
                min_index = j
        # só troco se o menor não já estiver no lugar
        if min_index != i:
            lista[i], lista[min_index] = lista[min_index], lista[i]
            copias += 3
    return lista, comparacoes, copias

def insertion_sort(lista):
    lista = lista.copy()
    comparacoes = copias = 0
    for i in range(1, len(lista)):
        chave = lista[i]
        copias += 1
        j = i - 1
        # empurro para a direita os elementos maiores que a chave
        while j >= 0:
            comparacoes += 1
            if lista[j] <= chave:
                break
            lista[j + 1] = lista[j] # desloca: 1 cópia, não uma troca
            copias += 1
            j -= 1
        lista[j + 1] = chave
        copias += 1
    return lista, comparacoes, copias

ALGORITMOS = {"bubble": bubble_sort, "selection": selection_sort, "insertion": insertion_sort}

# conferindo antes de medir
```

```

exemplo = [64, 34, 25, 12, 22, 11, 90, 18, 8, 9]
for nome, ordenar in ALGORITMOS.items():
    saida, _, _ = ordenar(exemplo)
    assert saida == sorted(exemplo), nome
assert exemplo == [64, 34, 25, 12, 22, 11, 90, 18, 8, 9] # a entrada não foi mexida
print("os três ordenam corretamente e não alteram a entrada")

```

os três ordenam corretamente e não alteram a entrada

2. Os quatro padrões de entrada

- **ordenado** e **reverso**: os extremos;
- **aleatório**: embaralhado com semente fixa, para o experimento ser reproduzível;
- **quase ordenado**: começa ordenado e 5% das posições trocam com um vizinho até 10 casas de distância.

In [3]:

```

def gerar_padrao(n, padrao):
    if padrao == "ordenado":
        return list(range(n))
    if padrao == "reverso":
        return list(range(n - 1, -1, -1))
    if padrao == "aleatorio":
        lista = list(range(n))
        random.shuffle(lista)
        return lista
    if padrao == "quase ordenado":
        lista = list(range(n))
        for _ in range(n // 20): # 5% das posições
            i = random.randrange(n)
            j = min(n - 1, i + random.randint(1, 10)) # vizinho próximo
            lista[i], lista[j] = lista[j], lista[i]
        return lista
    raise ValueError(f"padrão desconhecido: {padrao}")

PADROES = ["ordenado", "reverso", "quase ordenado", "aleatorio"]
TAMANHOS = [500, 1000, 2000, 4000]

```

3. O experimento: 3 algoritmos × 4 padrões × 4 tamanhos

A mesma lista de entrada é usada pelos três algoritmos em cada rodada, para a comparação ser justa.

In [4]:

```

linhas = []
for n in TAMANHOS:
    for padrao in PADROES:
        entrada = gerar_padrao(n, padrao)
        for nome, ordenar in ALGORITMOS.items():
            saida, comparacoes, copias = ordenar(entrada)

```

```

assert saida == sorted(entrada)
linhas.append({
    "algoritmo": nome, "padrao": padrao, "n": n,
    "comparacoes": comparacoes, "copias": copias,
    "comparacoes / n²": comparacoes / n**2,
})

resultados = pd.DataFrame(linhas)
resultados[resultados["n"] == 4000]

```

Out[4]:

	algoritmo	padrao	n	comparacoes	copias	comparacoes / n²
36	bubble	ordenado	4000	3999	0	0.000250
37	selection	ordenado	4000	7998000	0	0.499875
38	insertion	ordenado	4000	3999	7998	0.000250
39	bubble	reverso	4000	7998000	23994000	0.499875
40	selection	reverso	4000	7998000	6000	0.499875
41	insertion	reverso	4000	7998000	8005998	0.499875
42	bubble	quase ordenado	4000	43934	5472	0.002746
43	selection	quase ordenado	4000	7998000	588	0.499875
44	insertion	quase ordenado	4000	5823	9822	0.000364
45	bubble	aleatorio	4000	7996047	11786097	0.499753
46	selection	aleatorio	4000	7998000	11967	0.499875
47	insertion	aleatorio	4000	3932694	3936697	0.245793

Lendo a tabela (n = 4.000):

- **Reverso — o pior caso dos três:** os três fazem **exatamente as mesmas comparações**, $n(n-1)/2 = 7.998.000$. Se eu tivesse contado só comparações, eles pareceriam iguais.
- **É a coluna de cópias que separa os três:** o bubble **troca** (3 cópias por inversão), o insertion **desloca** (1 cópia por inversão) e o selection faz no máximo uma troca por posição. Por isso o enunciado pede as duas contagens.
- **Selection faz sempre o mesmo número de comparações**, qualquer que seja a entrada: ele varre toda a parte não ordenada para achar o mínimo, sem nunca parar antes.
- **Ordenado:** bubble e insertion fazem só $n-1$ comparações e nenhuma troca — o bubble para depois de uma passada sem trocar, e o insertion nunca entra no laço de deslocamento.

A coluna `comparacoes / n2` fica estável quando `n` cresce — perto de 0,5 no reverso, no selection e no bubble aleatório, e perto de 0,25 no insertion aleatório (em média ele só precisa percorrer metade da parte já ordenada). Razão estável contra n^2 é a evidência de que o custo é **$O(n^2)$** nesses casos.

In [5]:

```
reverso = resultados[resultados["padrao"] == "reverso"]
print("reverso: comparações iguais para os três em todos os tamanhos")
for n in TAMANHOS:
    valores = set(reverso[reverso["n"] == n]["comparacoes"])
    print(f"  n = {n:>5}: {valores} → teoria n(n-1)/2 = {n * (n - 1) // 2:,}")
    assert valores == {n * (n - 1) // 2}

selection = resultados[resultados["algoritmo"] == "selection"]
for n in TAMANHOS:
    assert set(selection[selection["n"] == n]["comparacoes"]) == {n * (n - 1) // 2}
print("\nselection: sempre n(n-1)/2 comparações, nos quatro padrões")
```

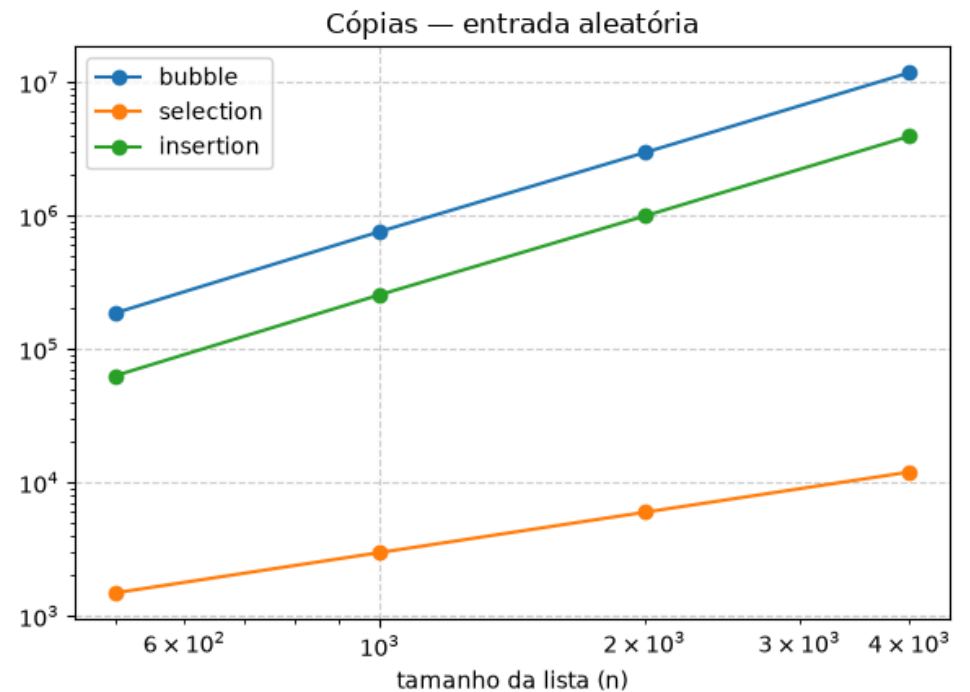
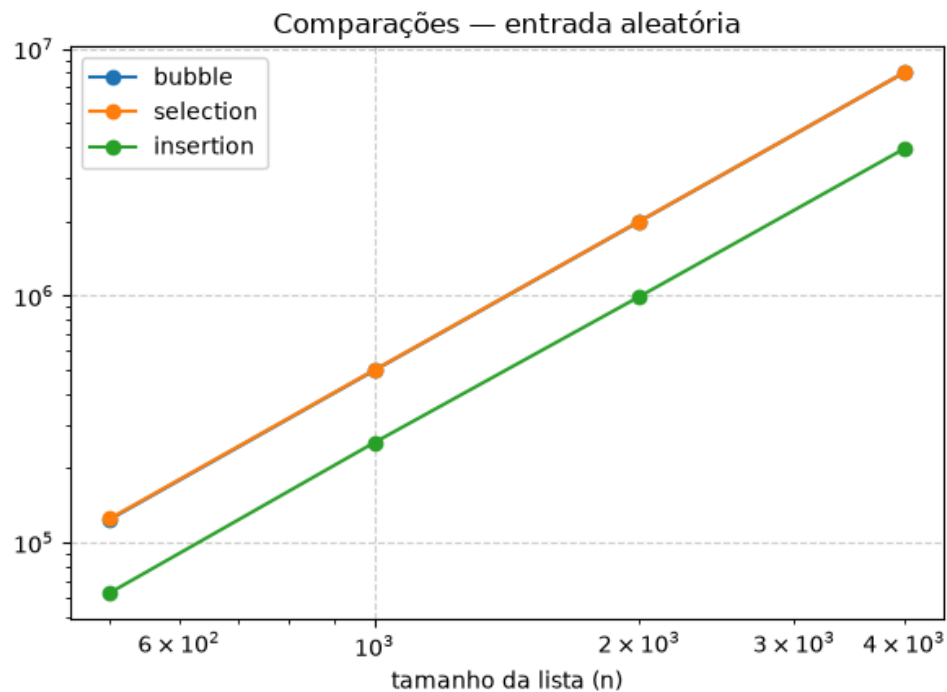
reverso: comparações iguais para os três em todos os tamanhos?

```
n = 500: {124750} → teoria n(n-1)/2 = 124,750
n = 1000: {499500} → teoria n(n-1)/2 = 499,500
n = 2000: {1999000} → teoria n(n-1)/2 = 1,999,000
n = 4000: {7998000} → teoria n(n-1)/2 = 7,998,000
```

selection: sempre $n(n-1)/2$ comparações, nos quatro padrões

In [6]:

```
fig, eixos = plt.subplots(1, 2, figsize=(12, 4.5))
for nome in ALGORITMOS:
    parte = resultados[(resultados["algoritmo"] == nome) & (resultados["padrao"] == "aleatorio")]
    eixos[0].plot(parte["n"], parte["comparacoes"], marker="o", label=nome)
    eixos[1].plot(parte["n"], parte["copias"], marker="o", label=nome)
eixos[0].set_title("Comparações – entrada aleatória")
eixos[1].set_title("Cópias – entrada aleatória")
for eixo in eixos:
    eixo.set_xlabel("tamanho da lista (n)")
    eixo.set_xscale("log")
    eixo.set_yscale("log")
    eixo.grid(True, linestyle="--", alpha=0.6)
    eixo.legend()
plt.tight_layout()
plt.show()
```



Em escala log-log as três curvas têm **inclinação 2**: dobrar n multiplica o custo por 4. É o comportamento quadrático aparecendo no gráfico.

4. Quem mais se beneficia do "quase ordenado"?

In [7]:

```
quase = resultados[resultados["padrao"] == "quase ordenado"].pivot(
    index="n", columns="algoritmo", values="comparacoes")
aleat = resultados[resultados["padrao"] == "aleatorio"].pivot(
    index="n", columns="algoritmo", values="comparacoes")

print("comparações no QUASE ORDENADO como fração do ALEATÓRIO (quanto menor, mais o algoritmo aproveitou):\n")
print((quase / aleat).round(4))
```

comparações no QUASE ORDENADO como fração do ALEATÓRIO (quanto menor, mais o algoritmo aproveitou):

algoritmo	bubble	insertion	selection
n			
500	0.0439	0.0114	1.0
1000	0.0319	0.0059	1.0
2000	0.0130	0.0030	1.0
4000	0.0055	0.0015	1.0

O insertion sort é o que mais ganha. No quase ordenado ele faz uma fração pequena das comparações do caso aleatório, enquanto o selection não ganha nada (faz sempre o mesmo) e o bubble fica no meio.

Por quê: cada comparação do insertion que empurra um elemento para a direita desfaz **exatamente uma inversão** (um par fora de ordem). Então o custo dele é proporcional a $n + d$, onde d é o número de inversões. Numa lista quase ordenada d é pequeno, e o insertion vira praticamente linear: **$O(n + d)$** .

O bubble também para cedo, mas só depois de todas as passadas necessárias, e cada passada custa até n comparações. Um elemento pequeno que ficou lá no fim anda só **uma posição por passada** para a esquerda — isso atrasa o bubble.

Para provar a relação com d , conto as inversões de verdade e comparo com as comparações do insertion. A contagem de inversões usa **divisão e conquista** (é um merge sort que conta os pares fora de ordem enquanto intercala), então sai em $O(n \log n)$:

In [8]:

```
def contar_inversoes(lista):
    # devolve (lista ordenada, número de pares fora de ordem)
    if len(lista) <= 1:
        return lista, 0
    meio = len(lista) // 2
    esquerda, inv_esquerda = contar_inversoes(lista[:meio])
    direita, inv_direita = contar_inversoes(lista[meio:])

    junta = []
    inversoes = inv_esquerda + inv_direita
    i = j = 0
    while i < len(esquerda) and j < len(direita):
        if esquerda[i] <= direita[j]:
            junta.append(esquerda[i])
            i += 1
        else:
            junta.append(direita[j])
            j += 1
            # direita[j] é menor que todos os que ainda sobraram na esquerda
            inversoes += len(esquerda) - i
    junta += esquerda[i:] + direita[j:]
    return junta, inversoes

assert contar_inversoes([3, 1, 2])[1] == 2      # (3,1) e (3,2)
assert contar_inversoes([1, 2, 3])[1] == 0

n = 4000
sensibilidade = []
for trocas in [0, 50, 200, 800, 3200, 12800]:
    lista = list(range(n))
    for _ in range(trocas):
        i = random.randrange(n)
        j = min(n - 1, i + random.randint(1, 10))
        lista[i], lista[j] = lista[j], lista[i]
    d = contar_inversoes(lista)[1]
    _, comparacoes, _ = insertion_sort(lista)
    sensibilidade.append({
        "trocas_aplicadas": trocas, "inversoes (d)": d,
        "comparacoes_insertion": comparacoes,
        "comparacoes / (n + d)": comparacoes / (n + d),
    })

pd.DataFrame(sensibilidade)
```

Out[8]:

	trocas_aplicadas	inversoes (d)	comparacoes_insertion	comparacoes / (n + d)
0	0	0	3999	0.999750
1	50	462	4461	0.999776
2	200	1796	5795	0.999827
3	800	6454	10453	0.999904
4	3200	16501	20498	0.999854
5	12800	35165	39162	0.999923

A razão $\text{comparacoes} / (n + d)$ fica praticamente em **1,0** em todos os níveis de desordem, enquanto d cresce centenas de vezes. Isso confirma que o custo do insertion é **$\Theta(n + d)$** : com poucas inversões ele é quase linear, e só vira $O(n^2)$ quando d chega na casa de $n^2/4$ (lista aleatória).

5. Validando a ordenação com uma BST

Insiro todos os elementos numa árvore binária de busca e faço a travessia **in-order** (esquerda → raiz → direita). Pela propriedade da BST, a in-order sai em ordem crescente.

Duas decisões:

- **Repetidos vão para a direita**, assim a in-order devolve todos eles, na ordem certa.
- **Inserção e in-order são iterativas**. Numa entrada já ordenada a BST vira uma "lista torta" com altura n . Com $n = 4.000$, uma versão recursiva estouraria o limite de recursão do Python (1.000 chamadas).

In [9]:

```

class NoArvore:
    def __init__(self, chave):
        self.chave = chave
        self.esquerda = None
        self.direita = None

class BinarySearchTree:
    def __init__(self):
        self.raiz = None
        self.tamanho = 0

    def insert(self, chave):
        """Insere e devolve quantas comparações de chave gastou."""
        novo = NoArvore(chave)
        self.tamanho += 1
        if self.raiz is None:

```

```

        self.raiz = novo
        return 0
    comparacoes = 0
    atual = self.raiz
    while True:
        comparacoes += 1
        if chave < atual.chave:
            if atual.esquerda is None:
                atual.esquerda = novo
                return comparacoes
            atual = atual.esquerda
        else: # repetidos vão para a direita
            if atual.direita is None:
                atual.direita = novo
                return comparacoes
            atual = atual.direita

    def search(self, chave):
        atual = self.raiz
        while atual is not None:
            if chave == atual.chave:
                return atual
            atual = atual.esquerda if chave < atual.chave else atual.direita
        return None

    def in_order(self):
        # in-order com pilha explícita: desce tudo à esquerda, visita, vai para a direita
        resultado = []
        pilha = []
        atual = self.raiz
        while pilha or atual is not None:
            while atual is not None:
                pilha.append(atual)
                atual = atual.esquerda
            atual = pilha.pop()
            resultado.append(atual.chave)
            atual = atual.direita
        return resultado

    def altura(self):
        # altura contada em níveis, percorrendo por largura
        if self.raiz is None:
            return 0
        nivel, altura = [self.raiz], 0
        while nivel:
            altura += 1
            nivel = [filho for no in nivel for filho in (no.esquerda, no.direita) if filho]
        return altura

def bst_sort(lista):
    """Ordena inserindo tudo na BST e lendo in-order. Devolve (ordenada, comparacoes, arvore)."""
    arvore = BinarySearchTree()
    comparacoes = 0
    for valor in lista:
        comparacoes += arvore.insert(valor)
    return arvore.in_order(), comparacoes, arvore

# a in-order confere com sorted(), inclusive com valores repetidos
original = gerar_padrao(1000, "aleatorio")
ordenada, _, arvore = bst_sort(original)
assert ordenada == sorted(original)
assert bst_sort([3, 1, 4, 1, 5, 9, 2, 6, 5, 3])[0] == [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]
print(f"BST com {arvore.tamanho} nós e altura {arvore.altura()}: in-order conferida com sorted()")

```

BST com 1000 nós e altura 20: in-order conferida com sorted()

Custo: ordenar pela BST × insertion sort

A in-order não compara chaves — só percorre os n nós, $O(n)$. **Tudo o custo da BST está nas inserções:** cada inserção desce um caminho da raiz até uma folha, custando $O(\text{altura})$.

In [10]:

```
comparacao = []
for n in TAMANHOS:
    for padrao in ["ordenado", "aleatorio"]:
        entrada = gerar_padrao(n, padrao)
        _, comp_bst, arvore = bst_sort(entrada)
        _, comp_ins, _ = insertion_sort(entrada)
        comparacao.append({
            "padrao": padrao, "n": n,
            "bst_comparacoes": comp_bst, "bst_altura": arvore.altura(),
            "insertion_comparacoes": comp_ins,
            "bst / insertion": comp_bst / comp_ins,
        })

pd.DataFrame(comparacao)
```

Out[10]:

	padrao	n	bst_comparacoes	bst_altura	insertion_comparacoes	bst / insertion
0	ordenado	500	124750	500	499	250.000000
1	aleatorio	500	4520	19	58731	0.076961
2	ordenado	1000	499500	1000	999	500.000000
3	aleatorio	1000	11702	24	249857	0.046835
4	ordenado	2000	1999000	2000	1999	1000.000000
5	aleatorio	2000	23826	26	1013247	0.023515
6	ordenado	4000	7998000	4000	3999	2000.000000
7	aleatorio	4000	55390	25	4024304	0.013764

O melhor caso de um é o pior caso do outro:

	entrada	insertion sort	ordenar pela BST
ordenada	$O(n)$ — nunca desloca	$O(n^2)$ — a árvore vira uma lista torta (altura = n) e cada inserção desce até o fim	
aleatória	$O(n^2)$ — em média $n^2/4$ comparações	$O(n \log n)$ — altura perto de $2 \cdot \log_2 n$	

Na tabela: com entrada ordenada a BST tem **altura igual a n** e faz milhares de vezes mais comparações que o insertion; com entrada aleatória a altura fica baixa e a BST ganha com folga.

Além disso a BST custa **O(n) de memória extra** (um nó por elemento), enquanto o insertion ordena no próprio lugar. Então não existe "a BST é melhor": depende de como os dados costumam chegar. Dados que já chegam quase ordenados (logs, ids sequenciais) são o pior caso da BST e o melhor do insertion.

6. Remoção na BST — os três casos

O enunciado deste exercício pede inserção e in-order, mas a remoção completa faz parte do que se espera de uma BST, então implemento e testo aqui os três casos:

caso	o que acontece
folha	o pai passa a apontar para <code>None</code>
um filho	o pai passa a apontar direto para o neto
dois filhos	o nó recebe a chave do sucessor in-order (a menor chave da subárvore direita) e o sucessor é removido no lugar dele

O sucessor in-order cabe na vaga porque é a menor chave maior que a removida — não viola a ordenação. E ele tem no máximo um filho (à direita), então removê-lo cai num dos casos simples.

In [11]:

```
def delete(arvore, chave):
    """Remove uma ocorrência de `chave`. Devolve True se removeu, False se não achou."""
    pai, atual = None, arvore.raiz
    while atual is not None and atual.chave != chave:
        pai = atual
        atual = atual.esquerda if chave < atual.chave else atual.direita
    if atual is None:
        return False

    # dois filhos: copio o sucessor in-order para cá e passo a remover o sucessor
    if atual.esquerda is not None and atual.direita is not None:
        pai_sucessor, sucessor = atual, atual.direita
        while sucessor.esquerda is not None:
            pai_sucessor, sucessor = sucessor, sucessor.esquerda
        atual.chave = sucessor.chave
        pai, atual = pai_sucessor, sucessor

    # agora `atual` tem no máximo um filho: o pai aponta direto para ele
    filho = atual.esquerda if atual.esquerda is not None else atual.direita
    if pai is None:
        arvore.raiz = filho
    elif pai.esquerda is atual:
        pai.esquerda = filho
    else:
        pai.direita = filho
    arvore.tamanho -= 1
    return True

def validar(arvore):
```

```

"""Confere a propriedade da BST com limites herdados de cima (não só pai x filho)."""
contados = 0
pilha = [(arvore.raiz, None, None)]
while pilha:
    no, minimo, maximo = pilha.pop()
    if no is None:
        continue
    contados += 1
    assert minimo is None or no.chave >= minimo, f"{no.chave} deveria ser >= {minimo}"
    assert maximo is None or no.chave < maximo, f"{no.chave} deveria ser < {maximo}"
    pilha.append((no.esquerda, minimo, no.chave))
    pilha.append((no.direita, no.chave, maximo))
assert contados == arvore.tamanho, "o tamanho não bate com os nós alcançáveis"

def arvore_exemplo():
    #
    #      50
    #     / \
    #    30  70
    #   / \ / \
    #  20 40 60 80
    #           \
    #            90
    arvore = BinarySearchTree()
    for chave in [50, 30, 70, 20, 40, 60, 80, 90]:
        arvore.insert(chave)
    return arvore

# caso 1 - folha: o 20 não tem filhos
a = arvore_exemplo()
assert delete(a, 20) and a.raiz.esquerda.esquerda is None
validar(a)
print("folha removida →", a.in_order())

# caso 2 - um filho: o 80 só tem o 90
a = arvore_exemplo()
assert delete(a, 80) and a.raiz.direita.direita.chave == 90 # o 70 apontou direto para o neto
validar(a)
print("um filho removido →", a.in_order())

# caso 3 - dois filhos: o 30 tem o 20 e o 40; o sucessor in-order é o 40
a = arvore_exemplo()
assert delete(a, 30) and a.raiz.esquerda.chave == 40 and a.raiz.esquerda.direita is None
validar(a)
print("dois filhos →", a.in_order(), " (o 40 assumiu o lugar do 30)")

# a raiz também pode ser removida, e chave ausente não quebra nada
a = arvore_exemplo()
assert delete(a, 50) and not delete(a, 999)
validar(a)

# sequência longa: insere 300 valores, remove todos em ordem aleatória, validando a cada passo
a = BinarySearchTree()
valores = gerar_padrao(300, "aleatorio")
for v in valores:
    a.insert(v)
random.shuffle(valores)
for v in valores:
    assert delete(a, v)
    validar(a)
    assert a.in_order() == sorted(a.in_order())
assert a.raiz is None and a.tamanho == 0
print("\n300 remoções em ordem aleatória: a árvore continuou válida a cada passo e terminou vazia")

```

folha removida → [30, 40, 50, 60, 70, 80, 90]
um filho removido → [20, 30, 40, 50, 60, 70, 90]
dois filhos → [20, 40, 50, 60, 70, 80, 90] (o 40 assumiu o lugar do 30)

300 remoções em ordem aleatória: a árvore continuou válida a cada passo e terminou vazia

Os `assert` olham a **estrutura** (qual ponteiro mudou), não só a in-order. E o `validar` usa limites herdados: um nó da subárvore esquerda da raiz precisa ser menor que a raiz, não só menor que o próprio pai — conferir só pai × filho deixaria passar árvores erradas.

7. Casos de borda

In [12]:

```
for nome, ordenar in ALGORITMOS.items():
    assert ordenar([]) == ([], 0, 0), nome           # lista vazia: nada a fazer
    assert ordenar([7])[0] == [7], nome             # um elemento
    assert ordenar([2, 2, 2])[0] == [2, 2, 2], nome  # todos iguais
    assert ordenar([5, -1, 0, -1])[0] == [-1, -1, 0, 5], nome # negativos e repetidos

# Lista toda igual: bubble e insertion param logo, sem mover nada
assert bubble_sort([2] * 100)[2] == 0
assert insertion_sort([2] * 100)[1] == 99

# BST vazia e BST com um nó
vazia = BinarySearchTree()
assert vazia.in_order() == [] and vazia.altura() == 0 and not delete(vazia, 1)
um = BinarySearchTree()
um.insert(5)
assert um.in_order() == [5] and delete(um, 5) and um.raiz is None

print("todos os casos de borda passaram")
```

todos os casos de borda passaram

Resumo

algoritmo	melhor caso	caso médio	pior caso	trocas ou deslocamentos
bubble (com parada antecipada)	$O(n)$ — já ordenado	$O(n^2)$	$O(n^2)$	até 3 cópias por inversão
selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	no máximo n trocas
insertion	$O(n)$ — já ordenado	$O(n^2)$	$O(n^2)$	1 cópia por inversão
insertion em lista quase ordenada		$O(n + d)$		
ordenar pela BST	$O(n \log n)$	$O(n \log n)$	$O(n^2)$ — entrada ordenada	$O(n)$ de memória

Cada linha está sustentada por uma tabela deste notebook: as comparações idênticas no reverso, o selection constante nos quatro padrões, a razão `comparacoes / (n + d)` fixa em 1,0 e a altura da BST igual a `n` na entrada ordenada.

Exercício 3 — Otimização orientada a Big O com requisitos de desempenho

Henrique Goldstein Maluhy Mendes Amaral · DR1_AT — Algoritmos e Estruturas de Dados · Infnet

Para rodar: **Ambiente de execução** → **Executar tudo**. O notebook é independente, não precisa de nenhum arquivo externo.

Roteiro: (1) deduplicação lenta × rápida, com testes de equivalência · (2) `k_smallest` ordenando tudo × QuickSelect · (3) variante em BST com travessia controlada e o pior caso · (4) casos de borda

In [1]:

```
import math
import random

import matplotlib.pyplot as plt
import pandas as pd

random.seed(42)

class Contador:
    """Guarda as contagens; passo o mesmo objeto para as funções auxiliares."""
    def __init__(self):
        self.comparacoes = 0
```

1. Deduplicação: de $O(n^2)$ para $O(n)$

As duas versões mantêm a **primeira ocorrência** de cada valor, na ordem em que apareceu.

- **Lenta:** para cada elemento, procuro nos resultados que já guardei se ele já apareceu — uma busca linear por elemento.
- **Rápida:** guardo o que já vi num **conjunto** (tabela hash). Perguntar "já vi?" custa $O(1)$ em média, em vez de percorrer uma lista.

Na versão rápida eu conto **uma comparação por consulta ao conjunto**: a tabela hash vai direto ao bucket e, em média, compara com uma quantidade constante de chaves.

In [2]:

```
def deduplicate_slow(arr):
    cont = Contador()
    resultado = []
    for valor in arr:
        repetido = False
        for visto in resultado:           # busca linear no que já guardei
            cont.comparacoes += 1
            if visto == valor:
                repetido = True
```

```

        break
    if not repetido:
        resultado.append(valor)
    return resultado, cont.comparacoes

def deduplicate_fast(arr):
    cont = Contador()
    resultado = []
    vistos = set()
    for valor in arr:
        cont.comparacoes += 1          # uma consulta à tabela hash
        if valor not in vistos:
            vistos.add(valor)
            resultado.append(valor)    # a lista garante a ordem de chegada
    return resultado, cont.comparacoes

assert deduplicate_slow([3, 1, 3, 2, 1])[0] == [3, 1, 2]
assert deduplicate_fast([3, 1, 3, 2, 1])[0] == [3, 1, 2]

```

Teste de equivalência

Comparo as duas em 300 listas sorteadas, com tamanhos e quantidade de repetição variados. Para ter uma referência independente, confiro também com `dict.fromkeys`, que é a forma pronta do Python de deduplicar mantendo a ordem (uso **só no teste**, nunca dentro dos algoritmos).

In [3]:

```

for _ in range(300):
    n = random.randint(0, 200)
    distintos = random.randint(1, 50)
    arr = [random.randint(1, distintos) for _ in range(n)]
    lento, _ = deduplicate_slow(arr)
    rapido, _ = deduplicate_fast(arr)
    assert lento == rapido == list(dict.fromkeys(arr))
print("300 listas sorteadas: as duas versões devolvem exatamente o mesmo resultado")

```

300 listas sorteadas: as duas versões devolvem exatamente o mesmo resultado

Medindo o ganho

Uma coisa que a medição mostrou e que eu não esperava: a versão lenta **não é sempre $O(n^2)$** . Ela percorre só a lista de resultados, que tem `u` elementos (a quantidade de valores **distintos**). Então o custo real é **$O(n \cdot u)$** . Ela só chega em $O(n^2)$ quando quase tudo é distinto. Por isso meço nos dois cenários.

In [4]:

```

linhas = []
for n in [500, 1000, 2000, 4000]:
    for cenario, distintos in [("todos distintos", n), ("%5% distintos", n // 20)]:
        arr = random.sample(range(10 * n), n) if distintos == n else \
            [random.randint(1, distintos) for _ in range(n)]
        _, lento = deduplicate_slow(arr)

```

```
_, rapido = deduplicate_fast(arr)
linhas.append({
    "cenario": cenario, "n": n,
    "lenta": lento, "rapida": rapido,
    "lenta / n²": lento / n**2,
    "rapida / n": rapido / n,
    "ganho (lenta / rapida)": lento / rapido,
})

dedup = pd.DataFrame(linhas)
dedup
```

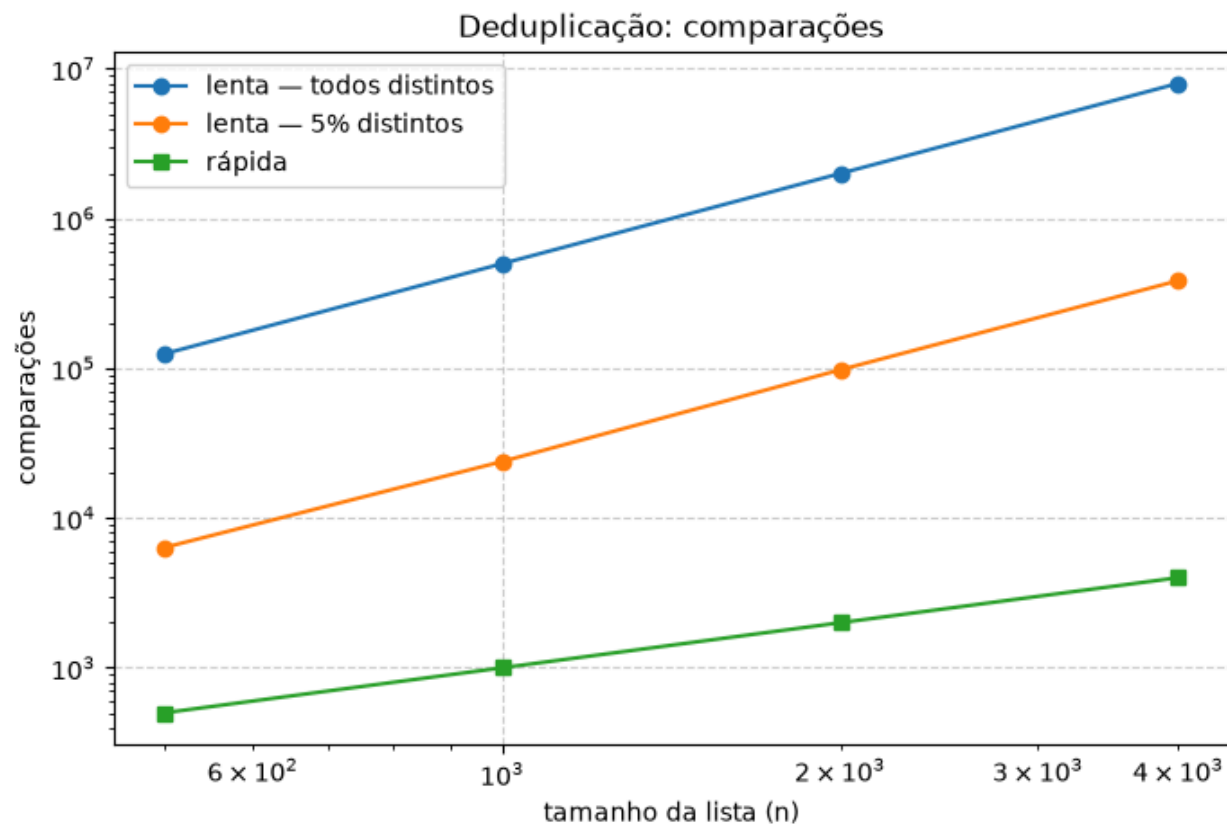
Out[4]:

	cenario	n	lenta	rapida	lenta / n²	rapida / n	ganho (lenta / rapida)
0	todos distintos	500	124750	500	0.499000	1.0	249.50000
1	5% distintos	500	6345	500	0.025380	1.0	12.69000
2	todos distintos	1000	499500	1000	0.499500	1.0	499.50000
3	5% distintos	1000	23844	1000	0.023844	1.0	23.84400
4	todos distintos	2000	1999000	2000	0.499750	1.0	999.50000
5	5% distintos	2000	97671	2000	0.024418	1.0	48.83550
6	todos distintos	4000	7998000	4000	0.499875	1.0	1999.50000
7	5% distintos	4000	385137	4000	0.024071	1.0	96.28425

- **Rápida:** `rapida / n` é exatamente **1,0** — uma consulta por elemento, $O(n)$.
- **Lenta com tudo distinto:** `lenta / n²` fica estável perto de **0,5** — cada elemento compara com todos os anteriores, $n(n-1)/2$, $O(n^2)$. O ganho cresce junto com `n`.
- **Lenta com 5% distintos:** bem mais barata, porque a lista de resultados é curta. Ainda assim o custo cresce com `n·u`, e `u` cresce junto com `n` aqui.

In [5]:

```
plt.figure(figsize=(8, 5))
for cenario in ["todos distintos", "5% distintos"]:
    parte = dedup[dedup["cenario"] == cenario]
    plt.plot(parte["n"], parte["lenta"], marker="o", label=f"lenta - {cenario}")
parte = dedup[dedup["cenario"] == "todos distintos"]
plt.plot(parte["n"], parte["rapida"], marker="s", label="rápida")
plt.xscale("log")
plt.yscale("log")
plt.title("Deduplicação: comparações")
plt.xlabel("tamanho da lista (n)")
plt.ylabel("comparações")
plt.grid(True, linestyle="--", alpha=0.6)
plt.legend()
plt.show()
```



2. Os k menores: ordenar tudo × QuickSelect

- **Versão A:** ordena o vetor inteiro com **merge sort** ($O(n \log n)$ garantido) e pega os k primeiros.
- **Versão B:** usa **QuickSelect** para colocar o k -ésimo menor no lugar certo. Como consequência da partição, tudo à esquerda dele é menor — então os k primeiros já são os k menores, só fora de ordem. Ordeno apenas esses k para devolver na mesma forma da versão A.

No QuickSelect o pivô é **sorteado**: com pivô fixo, uma entrada já ordenada levaria ao pior caso $O(n^2)$.

In [6]:

```
def merge_sort(arr, cont):
    if len(arr) <= 1:
        return arr
    meio = len(arr) // 2
    esquerda = merge_sort(arr[:meio], cont)
    direita = merge_sort(arr[meio:], cont)
    junta = []
    i = j = 0
```

```

while i < len(esquerda) and j < len(direita):
    cont.comparacoes += 1
    if esquerda[i] <= direita[j]:
        junta.append(esquerda[i])
        i += 1
    else:
        junta.append(direita[j])
        j += 1
return junta + esquerda[i:] + direita[j:]

def particionar(arr, inicio, fim, cont):
    # sorteio o pivô e Levo ele para o fim
    p = random.randint(inicio, fim)
    arr[p], arr[fim] = arr[fim], arr[p]
    pivo = arr[fim]
    i = inicio
    for j in range(inicio, fim):
        cont.comparacoes += 1
        if arr[j] < pivo:
            arr[i], arr[j] = arr[j], arr[i]
            i += 1
    arr[i], arr[fim] = arr[fim], arr[i] # o pivô vai para a posição definitiva
    return i

def quickselect(arr, indice, cont):
    """Deixa em arr[indice] o elemento que estaria ali se arr estivesse ordenado."""
    inicio, fim = 0, len(arr) - 1
    while inicio < fim:
        p = particionar(arr, inicio, fim, cont)
        if p == indice:
            break
        elif indice < p:
            fim = p - 1 # a resposta está à esquerda do pivô
        else:
            inicio = p + 1 # a resposta está à direita
    return arr[indice]

def k_smallest_A(arr, k):
    cont = Contador()
    return merge_sort(arr, cont)[:k], cont.comparacoes

def k_smallest_B(arr, k):
    cont = Contador()
    k = min(k, len(arr)) # pedir mais do que existe devolve tudo
    if k <= 0:
        return [], 0
    copia = arr.copy()
    quickselect(copia, k - 1, cont)
    return merge_sort(copia[:k], cont), cont.comparacoes

def k_smallest(arr, k, versao="B"):
    """A interface pedida no enunciado: escolhe a versão A (ordena tudo) ou B (QuickSelect)."""
    if versao == "A":
        return k_smallest_A(arr, k)
    if versao == "B":
        return k_smallest_B(arr, k)
    raise ValueError(f"versão desconhecida: {versao!r} (use 'A' ou 'B')")

assert k_smallest([9, 4, 7, 1, 8], 3, "A")[0] == k_smallest([9, 4, 7, 1, 8], 3, "B")[0] == [1, 4, 7]

# as duas devolvem a mesma coisa em 200 casos sorteados, incluindo valores repetidos
for _ in range(200):

```

```
n = random.randint(1, 300)
arr = [random.randint(0, 50) for _ in range(n)]
k = random.randint(1, n)
assert k_smallest_A(arr, k)[0] == k_smallest_B(arr, k)[0] == sorted(arr)[:k]
print("200 casos sorteados: A e B devolvem exatamente os mesmos k menores")
```

200 casos sorteados: A e B devolvem exatamente os mesmos k menores

In [7]:

```
linhas = []
for n in [1_000, 10_000, 50_000]:
    arr = random.sample(range(10 * n), n)
    for k in [1, 10, 100]:
        _, comp_a = k_smallest_A(arr, k)
        _, comp_b = k_smallest_B(arr, k)
        linhas.append({
            "n": n, "k": k,
            "A_ordena_tudo": comp_a,
            "B_quickselect": comp_b,
            "A / (n·log2 n)": comp_a / (n * math.log2(n)),
            "B / n": comp_b / n,
            "ganho (A / B)": comp_a / comp_b,
        })

ksel = pd.DataFrame(linhas)
ksel
```

Out[7]:

	n	k	A_ordena_tudo	B_quickselect	A / (n·log2 n)	B / n	ganho (A / B)
0	1000	1	8698	4784	0.872786	4.78400	1.818144
1	1000	10	8698	1043	0.872786	1.04300	8.339406
2	1000	100	8698	2911	0.872786	2.91100	2.987977
3	10000	1	120403	17137	0.906123	1.71370	7.025909
4	10000	10	120403	16024	0.906123	1.60240	7.513917
5	10000	100	120403	19111	0.906123	1.91110	6.300194
6	50000	1	718427	88659	0.920491	1.77318	8.103261
7	50000	10	718427	53340	0.920491	1.06680	13.468823
8	50000	100	718427	76210	0.920491	1.52420	9.426939

- **Versão A:** $A / (n \cdot \log_2 n)$ fica praticamente constante e **não depende de** k — ela ordena tudo de qualquer jeito. É $O(n \log n)$.
- **Versão B:** B / n fica numa faixa estável (algumas unidades), então a seleção é **$O(n)$ em média**. Cada partição descarta, em média, metade do que sobrou: $n + n/2 + n/4 + \dots \approx 2n$ comparações. Somando a ordenação dos k escolhidos, o total é **$O(n + k \log k)$** .

- O ganho **tende a crescer com** n , porque a diferença entre as duas é um fator $\log n$ — ganho de classe, não de constante. Ele não cresce de forma perfeitamente regular porque o pivô é sorteado: o número de comparações do QuickSelect varia de uma execução para outra (por isso B / n oscila entre 1 e 4), enquanto o merge sort é sempre igual.

O pior caso do QuickSelect continua sendo $O(n^2)$ (se o sorteio escolhesse sempre o pior pivô), mas com pivô aleatório isso é extremamente improvável. A versão A tem a vantagem de ser $O(n \log n)$ **sempre**.

3. A variante em árvore: `k_smallest_bst`

Insiro os valores numa BST e faço uma **in-order controlada**: ela para assim que junta k valores, em vez de percorrer a árvore inteira.

O custo é **$O(h + k)$** : primeiro desço até o menor elemento (até h passos, a altura) e depois visito mais k nós.

In [8]:

```
class NoArvore:
    def __init__(self, chave):
        self.chave = chave
        self.esquerda = None
        self.direita = None

def inserir(raiz, chave, cont):
    """Insere de forma iterativa (a árvore degenerada tem altura n) e devolve a raiz."""
    novo = NoArvore(chave)
    if raiz is None:
        return novo
    atual = raiz
    while True:
        cont.comparacoes += 1
        if chave < atual.chave:
            if atual.esquerda is None:
                atual.esquerda = novo
                return raiz
            atual = atual.esquerda
        else:
            if atual.direita is None:
                atual.direita = novo
                return raiz
            atual = atual.direita

def k_smallest_bst(root, k):
    """In-order que para ao juntar k valores. Devolve (valores, nós visitados)."""
    resultado = []
    visitados = 0
    pilha = []
    atual = root
    while (pilha or atual is not None) and len(resultado) < k:
        # desço tudo à esquerda: o menor que falta está lá embaixo
        while atual is not None:
            visitados += 1
            pilha.append(atual)
            atual = atual.esquerda
        atual = pilha.pop()
        resultado.append(atual.chave)
        atual = atual.direita
```

```

    return resultado, visitados

def altura(raiz):
    nivel, h = ([raiz] if raiz else []), 0
    while nivel:
        h += 1
        nivel = [f for no in nivel for f in (no.esquerda, no.direita) if f]
    return h

def construir(valores):
    cont = Contador()
    raiz = None
    for v in valores:
        raiz = inserir(raiz, v, cont)
    return raiz, cont.comparacoes

raiz, _ = construir([50, 30, 70, 20, 40, 60, 80])
assert k_smallest_bst(raiz, 3)[0] == [20, 30, 40]
assert k_smallest_bst(raiz, 100)[0] == [20, 30, 40, 50, 60, 70, 80]  # k maior que a árvore
print("k_smallest_bst conferida")

```

k_smallest_bst conferida

O pior caso: quando a BST vira uma lista encadeada

Se os valores chegam **em ordem**, cada novo valor vira filho do anterior e a árvore degenera numa lista torta, com altura n . Meço três formatos com os mesmos 2.000 valores:

In [9]:

```

n = 2000
valores = list(range(n))
embaralhados = valores.copy()
random.shuffle(embaralhados)
formatos = {
    "aleatória": embaralhados,
    "degenerada (crescente)": valores,
    "degenerada (decrescente)": valores[::-1],
}

linhas = []
for nome, entrada in formatos.items():
    raiz, custo_construcao = construir(entrada)
    h = altura(raiz)
    for k in [1, 10, 100]:
        resposta, visitados = k_smallest_bst(raiz, k)
        assert resposta == list(range(k))
        linhas.append({
            "formato": nome, "altura": h, "construcao_comparacoes": custo_construcao,
            "k": k, "nos_visitados": visitados,
        })

pd.DataFrame(linhas)

```

Out[9]:

	formato	altura	construcao_comparacoes	k	nos_visitados
0	aleatória	24	24291	1	11
1	aleatória	24	24291	10	17
2	aleatória	24	24291	100	103
3	degenerada (crescente)	2000	1999000	1	1
4	degenerada (crescente)	2000	1999000	10	10
5	degenerada (crescente)	2000	1999000	100	100
6	degenerada (decrecente)	2000	1999000	1	2000
7	degenerada (decrecente)	2000	1999000	10	2000
8	degenerada (decrecente)	2000	1999000	100	2000

O que a medição mostra:

- **Construir** a árvore degenerada custa $n(n-1)/2$ comparações — **$O(n^2)$** , contra $O(n \log n)$ na aleatória. Isso vale nas duas degeneradas.
- **Consultar** depende de **para que lado** ela degenerou, e isso eu não esperava:
 - **crescente:** o menor valor é a raiz, e a in-order já começa por ele. `k_smallest` visita só `k` nós — parece ótima;
 - **decrecente:** o menor valor está no fundo, a `n` níveis da raiz. Só para achar o primeiro é preciso descer a árvore inteira: `n` nós visitados, mesmo com `k = 1`.

Então "a BST degenerou" sozinho não diz o custo: o custo é **$O(h + k)$** , e o que importa é **onde** está o menor elemento. Com a árvore aleatória, `h` fica perto de $2 \cdot \log_2 n$ e a consulta é barata. Para garantir isso em qualquer entrada, a solução é uma árvore **balanceada** (como a AVL), que mantém `h =  $O(\log n)$` sempre.

4. Comparação final das três formas

forma	construção	consulta dos k menores	observação
A — ordena tudo	—	$O(n \log n)$	sempre o mesmo custo, qualquer <code>k</code>
B — QuickSelect	—	$O(n + k \log k)$ em média	a melhor para uma consulta só
BST	$O(n \log n)$ média, $O(n^2)$ pior	$O(h + k)$	vale a pena quando há muitas consultas sobre os mesmos dados: constrói uma vez e cada consulta sai barata

5. Casos de borda

In [10]:

```
# Listas vazias e com um elemento
assert deduplicate_slow([]) == ([], 0) and deduplicate_fast([]) == ([], 0)
assert deduplicate_fast([5])[0] == [5]

# tudo repetido e tudo distinto
assert deduplicate_fast([7] * 50)[0] == [7]
assert deduplicate_slow(list(range(10)))[0] == list(range(10))

# ordem da primeira ocorrência é mantida, mesmo com tipos diferentes
assert deduplicate_fast(["b", "a", "b", "c", "a"])[0] == ["b", "a", "c"]

# k nos extremos
dados = [9, 4, 7, 1, 8]
assert k_smallest_B(dados, 0)[0] == []
assert k_smallest_B(dados, 1)[0] == [1]
assert k_smallest_B(dados, 5)[0] == [1, 4, 7, 8, 9]
assert k_smallest_A(dados, 5)[0] == [1, 4, 7, 8, 9]
assert k_smallest_B(dados, 10)[0] == k_smallest_A(dados, 10)[0] == [1, 4, 7, 8, 9] # k > n

# árvore vazia
assert k_smallest_bst(None, 3) == ([], 0)

print("todos os casos de borda passaram")
```

todos os casos de borda passaram

Exercício 4 — Hashtable com colisões e encadeamento por lista

Henrique Goldstein Maluhy Mendes Amaral · DR1_AT — Algoritmos e Estruturas de Dados · Infnet

Para rodar: **Ambiente de execução** → **Executar tudo**. O notebook é independente, não precisa de nenhum arquivo externo.

Roteiro: (1) `HashTableChained` com buckets em lista encadeada · (2) atualização sem duplicata · (3) redimensionamento e rehash, com teste de que nada se perde · (4) custo medido × fator de carga · (5) cenário adversarial · (6) casos de borda

In [1]:

```
import random

import matplotlib.pyplot as plt
import pandas as pd

random.seed(42)
```

1. A implementação

Decisões:

- **Cada bucket é uma lista encadeada** de nós `(key, value, next)`. Um bucket vazio é só `None`.
- **Inserção no início do bucket:** depois de conferir que a chave não existe, o nó novo entra na frente — $O(1)$, sem precisar andar até o fim.
- **Função hash própria:** para inteiros uso a própria chave (como na aula: `chave % tamanho`); para strings, um hash polinomial. Não uso o `hash()` do Python porque ele sorteia o hash de strings a cada execução, e aí a distribuição nos buckets mudaria toda vez que o notebook roda.
- **Limiar de fator de carga: 0,75.** Quando `quantidade / capacidade` passa disso, a capacidade **dobra** e todas as chaves são redistribuídas (rehash).
- **Chave ausente levanta `KeyError`**, como o dicionário do Python. Devolver `None` seria ambíguo, porque `None` pode ser um valor legítimo guardado na tabela.

A tabela guarda um contador de comparações feitas dentro dos buckets, para as medições.

In [2]:

```
class NoBucket:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.next = None

class HashTableChained:
    LIMIAR = 0.75

    def __init__(self, capacidade=8, redimensionar=True):
        self.capacidade = capacidade
        self.buckets = [None] * capacidade
        self.quantidade = 0
        self.redimensionar = redimensionar  # desligo só nos experimentos com fator de carga fixo
        self.comparacoes = 0
        self.historico = []  # (capacidade antiga, nova, fator de carga no momento)

    def _funcao_hash(self, key):
        if isinstance(key, bool) or not isinstance(key, (int, str)):
            raise TypeError(f"tipo de chave não suportado: {type(key).__name__}")
        if isinstance(key, int):
            return key
        h = 0
        for caractere in key:
            # hash polinomial: cada letra pesa pela posição
            h = (h * 31 + ord(caractere)) % (2**32)
        return h

    def _indice(self, key):
        return self._funcao_hash(key) % self.capacidade

    def fator_de_carga(self):
        return self.quantidade / self.capacidade

    def put(self, key, value):
        i = self._indice(key)
        atual = self.buckets[i]
        while atual is not None:
            self.comparacoes += 1
            if atual.key == key:
                atual.value = value  # chave já existe: só atualizo, não duplico
            else:
                # chave não existe: insiro no início
                novo = NoBucket(key, value)
                novo.next = atual
                self.buckets[i] = novo
                self.quantidade += 1
                if self.fator_de_carga() > self.LIMIAR:
                    self._rehash()

    def _rehash(self):
        # redistribui todas as chaves para a nova capacidade
        nova_capacidade = self.capacidade * 2
        novo_hash = HashTableChained(nova_capacidade, self.redimensionar)
        for i in range(self.capacidade):
            atual = self.buckets[i]
            while atual is not None:
                novo_hash.put(atual.key, atual.value)
                atual = atual.next
        self.buckets = novo_hash.buckets
        self.capacidade = nova_capacidade
        self.quantidade = novo_hash.quantidade
        self.comparacoes = novo_hash.comparacoes
        self.historico.append((self.capacidade, nova_capacidade, self.fator_de_carga()))
```

```

        return
        atual = atual.next
    novo = NoBucket(key, value)
    novo.next = self.buckets[i]      # entra no início do bucket
    self.buckets[i] = novo
    self.quantidade += 1
    if self.redimensionar and self.fator_de_carga() > self.LIMIAR:
        self._rehash()

def get(self, key):
    atual = self.buckets[self._indice(key)]
    while atual is not None:
        self.comparacoes += 1
        if atual.key == key:
            return atual.value
        atual = atual.next
    raise KeyError(f"chave não encontrada: {key!r}")

def delete(self, key):
    i = self._indice(key)
    anterior, atual = None, self.buckets[i]
    while atual is not None:
        self.comparacoes += 1
        if atual.key == key:
            if anterior is None:
                self.buckets[i] = atual.next    # era o primeiro do bucket
            else:
                anterior.next = atual.next      # pula o nó removido
            self.quantidade -= 1
            return atual.value
        anterior, atual = atual, atual.next
    raise KeyError(f"chave não encontrada: {key!r}")

def __len__(self):
    return self.quantidade

def _rehash(self):
    antigos = self.buckets
    self.historico.append((self.capacidade, self.capacidade * 2, self.fator_de_carga()))
    self.capacidade *= 2
    self.buckets = [None] * self.capacidade
    # cada nó vai para o bucket novo; reaproveito o próprio nó, sem criar outro
    for cabeca in antigos:
        atual = cabeca
        while atual is not None:
            proximo = atual.next
            i = self._indice(atual.key)
            atual.next = self.buckets[i]
            self.buckets[i] = atual
            atual = proximo

def tamanhos_dos_buckets(self):
    tamanhos = []
    for cabeca in self.buckets:
        n, atual = 0, cabeca
        while atual is not None:
            n, atual = n + 1, atual.next
        tamanhos.append(n)
    return tamanhos

```

2. Funciona? Inserção, busca, atualização e remoção

In [3]:

```
tabela = HashTableChained()
tabela.put(11, "Alice")
tabela.put(45, "Paula")
tabela.put(58, "Helena")
tabela.put("curso", "Algoritmos")

assert tabela.get(11) == "Alice"
assert tabela.get("curso") == "Algoritmos"
assert len(tabela) == 4

# atualizar uma chave que já existe não cria outra entrada
tabela.put(11, "Alice Souza")
assert tabela.get(11) == "Alice Souza"
assert len(tabela) == 4

# remover
assert tabela.delete(45) == "Paula"
assert len(tabela) == 3
try:
    tabela.get(45)
except KeyError as erro:
    print(f"depois de remover, buscar a chave 45 dá: KeyError {erro}")

print("put, get, delete, __len__ e atualização conferidos")
```

depois de remover, buscar a chave 45 dá: KeyError 'chave não encontrada: 45'
put, get, delete, __len__ e atualização conferidos

3. Redimensionamento e rehash — nenhuma chave se perde

Insiro 10.000 chaves (metade inteiros, metade strings), registro cada redimensionamento e depois confiro **todas** as chaves.

In [4]:

```
tabela = HashTableChained()
esperado = {} # dicionário do Python usado SÓ como gabarito do teste
for i in range(5000):
    tabela.put(i * 7, f"int-{i}")
    tabela.put(f"chave-{i}", i)
    esperado[i * 7] = f"int-{i}"
    esperado[f"chave-{i}"] = i

print(f"{len(tabela)} chaves, capacidade final {tabela.capacidade}, "
      f"fator de carga {tabela.fator_de_carga():.3f}\n")
print("redimensionamentos:")
for antiga, nova, fator in tabela.historico:
    print(f" {antiga:>6} → {nova:>6} buckets (fator de carga tinha chegado a {fator:.3f})")

# todas as chaves continuam acessíveis e com o valor certo
for chave, valor in esperado.items():
    assert tabela.get(chave) == valor
assert len(tabela) == len(esperado)
```

```
# reinserir tudo com valores novos não duplica nada
for chave in esperado:
    tabela.put(chave, "novo")
assert len(tabela) == len(esperado)
assert all(tabela.get(chave) == "novo" for chave in esperado)
print("\nas 10.000 chaves continuam acessíveis depois de todos os rehashes, e reinserir não duplica")
```

10000 chaves, capacidade final 16384, fator de carga 0.610

redimensionamentos:

```
8 → 16 buckets (fator de carga tinha chegado a 0.875)
16 → 32 buckets (fator de carga tinha chegado a 0.812)
32 → 64 buckets (fator de carga tinha chegado a 0.781)
64 → 128 buckets (fator de carga tinha chegado a 0.766)
128 → 256 buckets (fator de carga tinha chegado a 0.758)
256 → 512 buckets (fator de carga tinha chegado a 0.754)
512 → 1024 buckets (fator de carga tinha chegado a 0.752)
1024 → 2048 buckets (fator de carga tinha chegado a 0.751)
2048 → 4096 buckets (fator de carga tinha chegado a 0.750)
4096 → 8192 buckets (fator de carga tinha chegado a 0.750)
8192 → 16384 buckets (fator de carga tinha chegado a 0.750)
```

as 10.000 chaves continuam acessíveis depois de todos os rehashes, e reinserir não duplica

Assim que o fator de carga passa de 0,75, a capacidade dobra e ele cai para perto de 0,375. Com poucos buckets ele chega a passar um pouco do limiar antes do rehash (com 8 buckets, a 7ª chave já leva a 0,875), mas nas capacidades grandes fica colado em 0,750. Como a capacidade **dobra** (e não aumenta de um em um), o custo total dos rehashes é $O(n)$: cada rehash custa o tamanho atual, e a soma $8 + 16 + 32 + \dots + n$ fica abaixo de $2n$. Por isso o `put` é **$O(1)$ amortizado**.

4. Custo × fator de carga

Com encadeamento, o bucket de uma chave tem em média $\alpha = n / m$ elementos (n chaves, m buckets). A teoria diz:

- **busca com sucesso:** $\approx 1 + \alpha/2$ comparações (acho a chave, em média, na metade do bucket);
- **busca sem sucesso:** $\approx \alpha$ comparações (percorro o bucket inteiro).

Para controlar o α , desligo o redimensionamento e uso capacidade fixa de 1.000 buckets, variando o número de chaves:

In [5]:

```
m = 1000
linhas = []
for n in [250, 500, 750, 1000, 2000, 4000, 8000]:
    tabela = HashTableChained(capacidade=m, redimensionar=False)
    chaves = random.sample(range(10**9), n)
    for chave in chaves:
        tabela.put(chave, None)

    tabela.comparacoes = 0
    for chave in chaves:
        tabela.get(chave)
    sucesso = tabela.comparacoes / n

    tabela.comparacoes = 0
    ausentes = random.sample(range(10**9, 2 * 10**9), 2000) # nenhuma delas está na tabela
```

```

for chave in ausentes:
    try:
        tabela.get(chave)
    except KeyError:
        pass
sem_sucesso = tabela.comparacoes / len(ausentes)

alfa = n / m
linhas.append({
    "alfa": alfa,
    "sucesso_medido": sucesso, "sucesso_teorias (1 + α/2)": 1 + alfa / 2,
    "sem_sucesso_medido": sem_sucesso, "sem_sucesso_teorias (α)": alfa,
    "medido / teorias": sucesso / (1 + alfa / 2),
})

custo = pd.DataFrame(linhas)
custo

```

Out[5]:

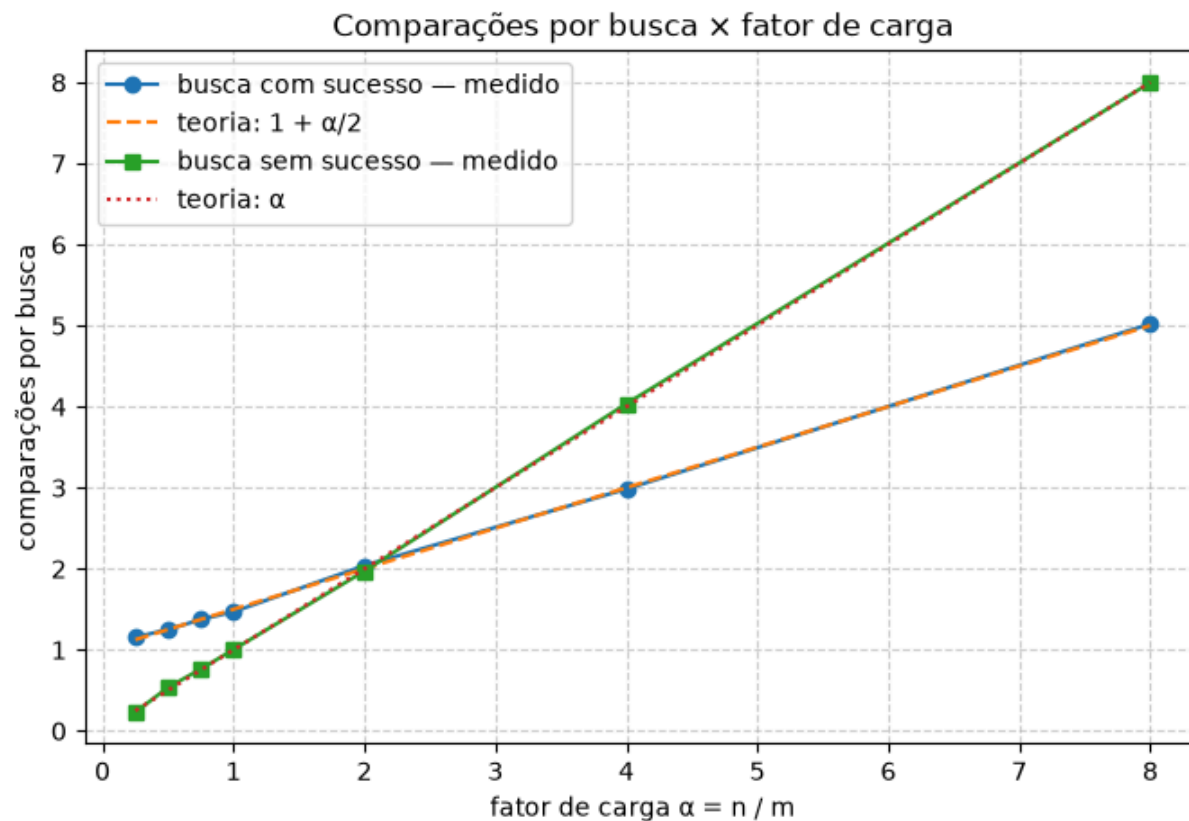
	alfa	sucesso_medido	sucesso_teorias (1 + α/2)	sem_sucesso_medido	sem_sucesso_teorias (α)	medido / teorias
0	0.25	1.156000	1.125	0.2300	0.25	1.027556
1	0.50	1.250000	1.250	0.5360	0.50	1.000000
2	0.75	1.373333	1.375	0.7625	0.75	0.998788
3	1.00	1.469000	1.500	1.0030	1.00	0.979333
4	2.00	2.039000	2.000	1.9715	2.00	1.019500
5	4.00	2.983500	3.000	4.0360	4.00	0.994500
6	8.00	5.020625	5.000	7.9975	8.00	1.004125

In [6]:

```

plt.figure(figsize=(8, 5))
plt.plot(custo["alfa"], custo["sucesso_medido"], marker="o", label="busca com sucesso - medido")
plt.plot(custo["alfa"], custo["sucesso_teorias (1 + α/2)"], linestyle="--", label="teorias: 1 + α/2")
plt.plot(custo["alfa"], custo["sem_sucesso_medido"], marker="s", label="busca sem sucesso - medido")
plt.plot(custo["alfa"], custo["sem_sucesso_teorias (α)"], linestyle=":", label="teorias: α")
plt.title("Comparações por busca x fator de carga")
plt.xlabel("fator de carga α = n / m")
plt.ylabel("comparações por busca")
plt.grid(True, linestyle="--", alpha=0.6)
plt.legend()
plt.show()

```



Os pontos medidos caem **em cima das retas teóricas** (`medido / teoria` fica perto de 1,0 enquanto α cresce 32 vezes). Ou seja: o custo da busca **não** é " $O(1)$ " incondicional — é **$O(1 + \alpha)$** . Ele só é constante porque o redimensionamento segura `$\alpha \leq 0,75$` , o que dá no máximo cerca de 1,4 comparação por busca com sucesso.

5. Cenário adversarial: todas as chaves no mesmo bucket

"Hashtable é $O(1)$ " vale **em média, se o hash espalhar as chaves**. Aqui eu quebro essa condição de propósito: escolho chaves que são **múltiplos de 1.024**. Como a capacidade é sempre uma potência de 2 (8, 16, ..., 1.024), todo múltiplo de 1.024 cai no bucket 0 — em **qualquer** dessas capacidades.

A consulta também é adversarial: busco uma chave ausente que **também** cai no bucket 0 (se eu buscasse uma chave qualquer, ela cairia num bucket vazio e o teste ia parecer ótimo).

In [7]:

```
n = 400
benignas = random.sample(range(10**9), n)
adversariais = [i * 1024 for i in range(n)]
```

```

resultados = []
for nome, chaves, ausente in [("benigno", benignas, 10**9 + 1), ("adversarial", adversariais, n * 1024)]:
    tabela = HashTableChained()
    for chave in chaves:
        tabela.put(chave, None)

    tabela.comparacoes = 0
    for chave in chaves:
        tabela.get(chave)
    por_busca = tabela.comparacoes / n

    tabela.comparacoes = 0
    try:
        tabela.get(ausente)
    except KeyError:
        pass

    tamanhos = tabela.tamanhos_dos_buckets()
    resultados.append({
        "cenario": nome, "capacidade_final": tabela.capacidade,
        "buckets_ocupados": sum(1 for t in tamanhos if t > 0),
        "maior_bucket": max(tamanhos),
        "comparacoes_por_busca_com_sucesso": por_busca,
        "busca_ausente": tabela.comparacoes,
        "busca_ausente / n": tabela.comparacoes / n,
    })

pd.DataFrame(resultados)

```

Out[7]:

	cenario	capacidade_final	buckets_ocupados	maior_bucket	comparacoes_por_busca_com_sucesso	busca_ausente	busca_ausente / n
0	benigno	1024	334	3	1.18	0	0.0
1	adversarial	1024	1	400	200.50	400	1.0

- **Benigno:** as 400 chaves se espalham por centenas de buckets e cada busca custa perto de **1** comparação.
- **Adversarial:** as mesmas 400 chaves ficam **todas num único bucket**. A tabela virou uma lista encadeada: a busca com sucesso custa em média $(n+1)/2$ e a busca de uma chave ausente custa **exatamente** n ($busca_ausente / n = 1,0$). Isso é **O(n)**.
- **Crescer não resolveu:** a tabela dobrou de 8 para 1.024 buckets durante o ataque. Não adianta, porque se dois números são congruentes módulo 1.024, também são congruentes módulo 8, 16, 32... As chaves continuam juntas em qualquer capacidade que seja potência de 2.

Como se defende disso na prática: usar uma função hash que o atacante não consiga prever (o Python faz isso com strings, sorteando o hash a cada execução — justamente o que eu desliguei aqui para ter reprodutibilidade), ou usar um número primo como capacidade, o que quebra esse padrão específico de múltiplos.

6. Casos de borda

In [8]:

```

tabela = HashTableChained()
assert len(tabela) == 0

```

```
# chave ausente em tabela vazia
for operacao in (tabela.get, tabela.delete):
    try:
        operacao("nada")
        raise AssertionError("deveria ter levantado KeyError")
    except KeyError:
        pass

# None como valor é permitido e não se confunde com "não achei"
tabela.put("vazio", None)
assert tabela.get("vazio") is None and len(tabela) == 1

# remover o primeiro, o do meio e o último de um mesmo bucket
tabela = HashTableChained(capacidade=4, redimensionar=False)
for chave in [0, 4, 8, 12]:
    # todas caem no bucket 0
    tabela.put(chave, chave)
assert tabela.tamANHos_dos_buckets()[0] == 4
tabela.delete(12)
# o primeiro do bucket (inserção é no início)
tabela.delete(4)
# um do meio
tabela.delete(0)
# o último
assert tabela.get(8) == 8 and len(tabela) == 1

# chaves negativas e strings vazias funcionam
tabela.put(-5, "negativo")
tabela.put("", "string vazia")
assert tabela.get(-5) == "negativo" and tabela.get("") == "string vazia"

# tipo de chave não suportado é recusado com mensagem clara
try:
    tabela.put([1, 2], "lista")
except TypeError as erro:
    print(f"chave inválida → {erro}")

print("todos os casos de borda passaram")
```

chave inválida → tipo de chave não suportado: list
todos os casos de borda passaram

Resumo

operação		caso médio	pior caso (todas no mesmo bucket)
put	O(1) amortizado		O(n)
get	O(1 + α)		O(n)
delete	O(1 + α)		O(n)
__len__	O(1)		O(1)
rehash	O(n), mas raro: O(1) amortizado por inserção		O(n)

O "O(1)" da hashtable é uma **promessa condicional**: depende do fator de carga ficar controlado (o rehash garante isso) e do hash espalhar as chaves (o cenário adversarial mostra o que acontece quando não espalha).

Exercício 5 — Pilha e fila com invariantes e árvore de expressão

Henrique Goldstein Maluhy Mendes Amaral · DR1_AT — Algoritmos e Estruturas de Dados · Infnet

Para rodar: **Ambiente de execução** → **Executar tudo**. O notebook é independente, não precisa de nenhum arquivo externo.

Roteiro: (1) `Stack` e `Queue` em vetor de capacidade fixa, com exceções próprias · (2) teste longo das invariantes, incluindo o wraparound da fila · (3) árvore de expressão a partir da pós-fixa, com detecção de entradas inválidas · (4) avaliação recursiva × iterativa e o risco da profundidade · (5) casos de borda

In [1]:

```
import random
import sys
import time

import pandas as pd

random.seed(42)
```

1. `Stack` e `Queue` com capacidade fixa

As duas usam um **vetor estático** (`[None] * capacidade`), criado uma vez só — como na aula do simulador de fila de impressão.

- **Pilha:** `topo` guarda o índice do último elemento; `-1` quer dizer vazia.
- **Fila circular:** `inicio` aponta para o primeiro, `fim` para a próxima posição livre, e `total` diz quantos há. Quando `fim` chega no final do vetor, ele **volta para a posição 0** (`% capacidade`) — é o wraparound. Assim a fila reaproveita o espaço liberado na frente, sem nunca deslocar elementos.

Cada situação de erro tem a **sua própria exceção**, com uma mensagem dizendo o que aconteceu.

In [2]:

```
class StackOverflowError(Exception):
    """push em pilha cheia."""

class StackUnderflowError(Exception):
    """pop ou peek em pilha vazia."""

class QueueOverflowError(Exception):
    """enqueue em fila cheia."""

class QueueUnderflowError(Exception):
    """dequeue ou peek em fila vazia."""

class Stack:
    def __init__(self, capacidade):
```

```

        if capacidade < 1:
            raise ValueError(f"a capacidade precisa ser pelo menos 1, recebi {capacidade}")
        self.capacidade = capacidade
        self.itens = [None] * capacidade
        self.topo = -1

    def is_empty(self):
        return self.topo == -1

    def is_full(self):
        return self.topo == self.capacidade - 1

    def __len__(self):
        return self.topo + 1

    def push(self, item):
        if self.is_full():
            raise StackOverflowError(
                f"push({item!r}) em pilha cheia (capacidade {self.capacidade})")
        self.topo += 1
        self.itens[self.topo] = item

    def pop(self):
        if self.is_empty():
            raise StackUnderflowError("pop em pilha vazia")
        item = self.itens[self.topo]
        self.itens[self.topo] = None    # limpo a posição para não segurar a referência
        self.topo -= 1
        return item

    def peek(self):
        if self.is_empty():
            raise StackUnderflowError("peek em pilha vazia")
        return self.itens[self.topo]

class Queue:
    def __init__(self, capacidade):
        if capacidade < 1:
            raise ValueError(f"a capacidade precisa ser pelo menos 1, recebi {capacidade}")
        self.capacidade = capacidade
        self.itens = [None] * capacidade
        self.inicio = 0
        self.fim = 0
        self.total = 0
        self.voltas = 0    # quantas vezes o `fim` deu a volta no vetor (só para medir)

    def is_empty(self):
        return self.total == 0

    def is_full(self):
        return self.total == self.capacidade

    def __len__(self):
        return self.total

    def enqueue(self, item):
        if self.is_full():
            raise QueueOverflowError(
                f"enqueue({item!r}) em fila cheia (capacidade {self.capacidade})")
        self.itens[self.fim] = item
        self.fim = (self.fim + 1) % self.capacidade    # wraparound
        if self.fim == 0:
            self.voltas += 1
        self.total += 1

    def dequeue(self):
        if self.is_empty():

```

```

        raise QueueUnderflowError("dequeue em fila vazia")
    item = self.itens[self.inicio]
    self.itens[self.inicio] = None
    self.inicio = (self.inicio + 1) % self.capacidade
    self.total -= 1
    return item

def peek(self):
    if self.is_empty():
        raise QueueUnderflowError("peek em fila vazia")
    return self.itens[self.inicio]

def em_ordem(self):
    """Os elementos do primeiro ao último, seguindo o buffer circular."""
    return [self.itens[(self.inicio + i) % self.capacidade] for i in range(self.total)]

```

Invariantes

Depois de **qualquer** operação, as duas estruturas precisam satisfazer:

- **pilha:** `-1 <= topo < capacidade` ; as posições acima do topo estão vazias (`None`);
- **fila:** `0 <= total <= capacidade` ; `fim == (inicio + total) % capacidade` ; as posições fora da janela ocupada estão vazias.

In [3]:

```

def verificar_pilha(p):
    assert -1 <= p.topo < p.capacidade
    assert all(x is None for x in p.itens[p.topo + 1:]), "sobrou lixo acima do topo"

def verificar_fila(f):
    assert 0 <= f.total <= f.capacidade
    assert f.fim == (f.inicio + f.total) % f.capacidade, "inicio, fim e total não batem"
    ocupadas = {(f.inicio + i) % f.capacidade for i in range(f.total)}
    for i in range(f.capacidade):
        if i not in ocupadas:
            assert f.itens[i] is None, f"posição {i} deveria estar vazia"

```

2. Teste longo: 10.000 operações sorteadas

Uma pilha e uma fila de **capacidade 8** recebem 10.000 operações aleatórias. Em paralelo, uma lista do Python faz o papel de **gabarito** (só no teste). A cada operação eu confiro:

- se a estrutura estava cheia/vazia, a exceção certa **tem** que acontecer;
- senão, o resultado tem que ser igual ao do gabarito;
- as invariantes continuam valendo.

In [4]:

```
pilha, fila = Stack(8), Queue(8)
gab_pilha, gab_fila = [], []
contagem = {"overflow pilha": 0, "underflow pilha": 0, "overflow fila": 0, "underflow fila": 0}

for passo in range(10_000):
    # pilha
    if random.random() < 0.5:
        try:
            pilha.push(passo)
            assert len(gab_pilha) < 8
            gab_pilha.append(passo)
        except StackOverflowError:
            assert len(gab_pilha) == 8 # só pode estourar se estiver mesmo cheia
            contagem["overflow pilha"] += 1
    else:
        try:
            assert pilha.pop() == gab_pilha.pop()
        except StackUnderflowError:
            assert not gab_pilha
            contagem["underflow pilha"] += 1
        except IndexError:
            # o gabarito estava vazio e a pilha não reclamou
            raise AssertionError("a pilha devia ter levantado StackUnderflowError")
    verificar_pilha(pilha)

    # fila
    if random.random() < 0.5:
        try:
            fila.enqueue(passo)
            assert len(gab_fila) < 8
            gab_fila.append(passo)
        except QueueOverflowError:
            assert len(gab_fila) == 8
            contagem["overflow fila"] += 1
    else:
        try:
            assert fila.dequeue() == gab_fila.pop(0) # FIFO: sai o mais antigo
        except QueueUnderflowError:
            assert not gab_fila
            contagem["underflow fila"] += 1
        except IndexError:
            raise AssertionError("a fila devia ter levantado QueueUnderflowError")
    verificar_fila(fila)
    assert fila.em_ordem() == gab_fila

print("10.000 operações: resultados iguais ao gabarito e invariantes válidas em todos os passos")
print(f"a fila deu {fila.voltas} voltas no vetor de 8 posições (wraparound) sem perder a ordem FIFO\n")
pd.DataFrame([contagem])
```

10.000 operações: resultados iguais ao gabarito e invariantes válidas em todos os passos
a fila deu 552 voltas no vetor de 8 posições (wraparound) sem perder a ordem FIFO

Out[4]:

	overflow pilha	underflow pilha	overflow fila	underflow fila
0	497	567	548	625

As quatro exceções aconteceram centenas de vezes cada, **sempre** no momento certo — nunca antes (estrutura com espaço) nem depois (estrutura já cheia ou vazia). E a fila deu centenas de voltas no vetor de 8 posições sem perder a ordem.

Custo: `push`, `pop`, `enqueue`, `dequeue` e `peek` são todos **O(1)**: mexem num índice só. A fila circular é o que garante isso — uma fila que tirasse sempre da posição 0 de uma lista (`pop(0)`) teria que deslocar todo mundo, O(n) por `dequeue`.

3. Árvore de expressão a partir da pós-fixa

Na pós-fixa o operador vem **depois** dos operandos (`3 4 +` quer dizer `3 + 4`). Montar a árvore com uma pilha:

1. **operando** → vira um nó folha e vai para a pilha;
2. **operador** → tira dois nós da pilha (o primeiro que sai é o da **direita**), cria um nó com eles como filhos e empilha o resultado;
3. no fim, **tem que sobrar exatamente um nó**: a raiz.

Duas entradas inválidas são detectadas, dizendo **em que posição** o problema apareceu:

- operador sem dois operandos na pilha (`3 +`);
- operandos sobrando no fim (`3 4 5 +`).

In [5]:

```
OPERADORES = {"+", "-", "*", "/", "^"}

class ExpressaoInvalidaError(Exception):
    """A string não é uma expressão pós-fixa válida."""

class NoExpressao:
    def __init__(self, valor, esquerda=None, direita=None):
        self.valor = valor
        self.esquerda = esquerda
        self.direita = direita

    def eh_folha(self):
        return self.esquerda is None and self.direita is None

def construir_arvore(posfixa):
    tokens = posfixa.split()
    if not tokens:
        raise ExpressaoInvalidaError("expressão vazia")

    # a pilha nunca precisa guardar mais nós do que há tokens
    pilha = Stack(len(tokens))
    for posicao, token in enumerate(tokens, start=1):
        if token in OPERADORES:
            if len(pilha) < 2:
                raise ExpressaoInvalidaError(
                    f"o operador '{token}' (token {posicao}) precisa de dois operandos, "
                    f"mas a pilha tem {len(pilha)}")
            direita = pilha.pop()
            esquerda = pilha.pop()
            pilha.push(NoExpressao(token, esquerda, direita))
        else:
            try:
                numero = float(token)
            except ValueError:
```

```

        raise ExpressaoInvalidaError(
            f'"{token}" (token {posicao}) não é número nem operador') from None
    pilha.push(NoExpressao(numero))

    if len(pilha) != 1:
        raise ExpressaoInvalidaError(
            f"sobraram {len(pilha)} operandos sem operador no fim da expressão")
    return pilha.pop()

def para_infixa(no):
    """Só para visualizar a árvore: reescreve com parênteses."""
    if no.eh_folha():
        return f"{no.valor:g}"
    return f"({para_infixa(no.esquerda)} {no.valor} {para_infixa(no.direita)})"

raiz = construir_arvore("3 4 + 2 *")
print("3 4 + 2 * →", para_infixa(raiz))
raiz = construir_arvore("5 1 2 + 4 * + 3 -")
print("5 1 2 + 4 * + 3 - →", para_infixa(raiz))

```

3 4 + 2 * → ((3 + 4) * 2)
 5 1 2 + 4 * + 3 - → ((5 + ((1 + 2) * 4)) - 3)

4. Avaliando de duas formas

- **Recursiva (pós-ordem):** avalia a subárvore esquerda, depois a direita, e só então aplica o operador do nó. Fica curtinha, mas cada nível da árvore vira uma chamada empilhada na pilha do **Python**.
- **Iterativa com `stack`:** faz a mesma pós-ordem com uma pilha **minha**. Cada nó entra primeiro como "ainda não avaliei os filhos" e depois como "filhos prontos, posso aplicar o operador".

As duas contam quantos nós visitaram, para comparar o trabalho.

In [6]:

```

def aplicar(operador, a, b):
    if operador == "+":
        return a + b
    if operador == "-":
        return a - b
    if operador == "*":
        return a * b
    if operador == "/":
        if b == 0:
            raise ZeroDivisionError("divisão por zero ao avaliar a expressão")
        return a / b
    return a ** b

def avaliar_recursivo(no, visitas):
    visitas[0] += 1
    if no.eh_folha():
        return no.valor
    a = avaliar_recursivo(no.esquerda, visitas)
    b = avaliar_recursivo(no.direita, visitas)
    return aplicar(no.valor, a, b)

```

```

def contar_nos(raiz):
    total, pilha = 0, [raiz]
    while pilha:
        no = pilha.pop()
        total += 1
        pilha.extend(f for f in (no.esquerda, no.direita) if f is not None)
    return total

def avaliar_iterativo(raiz, visitas):
    n = contar_nos(raiz)
    tarefas = Stack(2 * n)      # cada nó entra no máximo duas vezes
    valores = Stack(n)
    tarefas.push((raiz, False))
    while not tarefas.is_empty():
        no, filhos_prontos = tarefas.pop()
        if no.eh_folha():
            visitas[0] += 1
            valores.push(no.valor)
        elif filhos_prontos:
            b = valores.pop()
            a = valores.pop()
            valores.push(aplicar(no.valor, a, b))
        else:
            visitas[0] += 1
            tarefas.push((no, True))      # volto nele depois dos filhos
            tarefas.push((no.direita, False))
            tarefas.push((no.esquerda, False)) # empilhado por último = avaliado primeiro
    return valores.pop()

casos = {
    "3 4 + 2 *": 14,
    "5 1 2 + 4 * + 3 -": 14,
    "2 3 ^ 1 -": 7,
    "10 4 / 2 *": 5,
    "7": 7,
}

for expressao, esperado in casos.items():
    arvore = construir_arvore(expressao)
    v_rec, v_it = [0], [0]
    r = avaliar_recursivo(arvore, v_rec)
    i = avaliar_iterativo(arvore, v_it)
    assert r == i == esperado, expressao
    assert v_rec[0] == v_it[0] == contar_nos(arvore) # as duas visitam cada nó uma vez
    print(f"{expressao:<22} = {r:g}    ({v_rec[0]} nó(s) visitado(s) nas duas versões)")

```

```

3 4 + 2 *           = 14    (5 nó(s) visitado(s) nas duas versões)
5 1 2 + 4 * + 3 -   = 14    (9 nó(s) visitado(s) nas duas versões)
2 3 ^ 1 -           = 7     (5 nó(s) visitado(s) nas duas versões)
10 4 / 2 *          = 5     (5 nó(s) visitado(s) nas duas versões)
7                   = 7     (1 nó(s) visitado(s) nas duas versões)

```

O risco da recursão: profundidade

As duas versões fazem o **mesmo trabalho** (visitam cada nó uma vez, $O(n)$). A diferença é **onde fica a pilha**:

- na recursiva, cada nível da árvore é uma chamada guardada na pilha do Python, que tem limite;
- na iterativa, a pilha é minha e cresce na memória comum.

Uma expressão como `1 1 + 1 + 1 + ...` gera uma árvore que só cresce para a esquerda: a profundidade é igual ao número de operadores. Monto uma com profundidade **três vezes o limite de recursão** do Python:

In [7]:

```
limite = sys.getrecursionlimit()
profundidade = 3 * limite
expressao = "1 " + "1 + " * profundidade
arvore = construir_arvore(expressao)
print(f"limite de recursão do Python: {limite}")
print(f"expressão com {profundidade} operadores → árvore de {contar_nos(arvore)} nós\n")

try:
    avaliar_recursivo(arvore, [0])
    print("recursiva: terminou (o limite deste ambiente é maior que o esperado)")
except RecursionError:
    print(f"recursiva: RecursionError – estourou a pilha do Python")

visitas = [0]
inicio = time.perf_counter()
resultado = avaliar_iterativo(arvore, visitas)
duracao = time.perf_counter() - inicio
print(f"iterativa: resultado {resultado:g} (o esperado é {profundidade + 1}), "
      f"{visitas[0]} nós visitados em {duracao * 1000:.1f} ms")
assert resultado == profundidade + 1
```

limite de recursão do Python: 1000
expressão com 3000 operadores → árvore de 6001 nós

recursiva: RecursionError – estourou a pilha do Python
iterativa: resultado 3001 (o esperado é 3001), 6001 nós visitados em 8.3 ms

Complexidade e riscos:

	tempo	memória extra	risco
montar a árvore	O(n) — um push/pop por token	O(n) nós	—
avaliação recursiva	O(n)	O(h) chamadas na pilha do Python	quebra com RecursionError se h passar do limite (~1.000)
avaliação iterativa	O(n)	O(h) na minha pilha	só depende da memória disponível

h é a altura da árvore: em expressões balanceadas é perto de $\log_2 n$, mas numa expressão encadeada como a de cima chega a $n/2$. A recursiva é mais fácil de ler e serve bem para expressões de tamanho normal; a iterativa é a que **não quebra** em entradas grandes ou maliciosas.

5. Casos de borda

In [8]:

```
invalidas = {
    "": "expressão vazia",
```

```

    "3 +": "operador sem dois operandos",
    "+": "operador sozinho",
    "3 4 5 +": "operandos sobrando",
    "3 4 x": "token desconhecido",
    "1 2 + +": "segundo operador sem operandos",
}
for expressao, motivo in invalidas.items():
    try:
        construir_arvore(expressao)
        raise AssertionError(f'{expressao}' deveria ser inválida")
    except ExpressaoInvalidaError as erro:
        print(f"{motivo:<32} '{expressao}' → {erro}")

# divisão por zero é erro de avaliação, não de montagem
arvore = construir_arvore("1 0 /")
try:
    avaliar_iterativo(arvore, [0])
except ZeroDivisionError as erro:
    print(f"\n1 0 '/' monta a árvore, mas ao avaliar → {erro}")

# números negativos e com vírgula decimal
assert avaliar_recursivo(construir_arvore("-2 3.5 *"), [0]) == -7.0

# pilha e fila com capacidade 1
p = Stack(1)
p.push("a")
try:
    p.push("b")
except StackOverflowError as erro:
    print(f"\npilha de capacidade 1 → {erro}")
f = Queue(1)
f.enqueue("a")
assert f.dequeue() == "a"
try:
    f.dequeue()
except QueueUnderflowError as erro:
    print(f"fila de capacidade 1 → {erro}")

# capacidade inválida
try:
    Stack(0)
except ValueError as erro:
    print(f"Stack(0) → {erro}")

print("\ntodos os casos de borda passaram")

```

expressão vazia	' ' → expressão vazia
operador sem dois operandos	'3 +' → o operador '+' (token 2) precisa de dois operandos, mas a pilha tem 1
operador sozinho	'+' → o operador '+' (token 1) precisa de dois operandos, mas a pilha tem 0
operandos sobrando	'3 4 5 +' → sobram 2 operandos sem operador no fim da expressão
token desconhecido	'3 4 x' → 'x' (token 3) não é número nem operador
segundo operador sem operandos	'1 2 + +' → o operador '+' (token 4) precisa de dois operandos, mas a pilha tem 1

'1 0 '/' monta a árvore, mas ao avaliar → divisão por zero ao avaliar a expressão

pilha de capacidade 1 → push('b') em pilha cheia (capacidade 1)
 fila de capacidade 1 → dequeue em fila vazia
 Stack(0) → a capacidade precisa ser pelo menos 1, recebi 0

todos os casos de borda passaram

Exercício 6 — Simulador de eventos com múltiplas filas e índice em árvore

Henrique Goldstein Maluhy Mendes Amaral · DR1_AT — Algoritmos e Estruturas de Dados · Infnet

Para rodar: **Ambiente de execução** → **Executar tudo**. O notebook é independente, não precisa de nenhum arquivo externo.

Roteiro: (1) fila circular de capacidade fixa · (2) BST para indexar os atendidos · (3) o simulador com as quatro filas · (4) execução com log, snapshots, overflow e underflow · (5) busca e listagem no índice, com o custo de cada operação · (6) o que acontece com a altura do índice · (7) casos de borda

Regras dos eventos: letra **minúscula** = chegada na fila (**b** = chega um cliente na fila B); letra **maiúscula** = saída da fila (**B** = a fila B atende o primeiro); **qualquer caractere que não seja letra** = snapshot de todas as filas.

In [1]:

```
import random

import matplotlib.pyplot as plt
import pandas as pd

random.seed(42)
```

1. A fila: circular e com capacidade fixa

Mesma ideia da fila do exercício 5: vetor estático, `inicio`, `fim` e `total`, com o `fim` voltando para a posição 0 quando chega no final.

In [2]:

```
class QueueOverflowError(Exception):
    """enqueue em fila cheia."""

class QueueUnderflowError(Exception):
    """dequeue em fila vazia."""

class Queue:
    def __init__(self, capacidade):
        self.capacidade = capacidade
        self.itens = [None] * capacidade
        self.inicio = 0
        self.fim = 0
        self.total = 0

    def enqueue(self, item):
        if self.total == self.capacidade:
            raise QueueOverflowError(f"fila cheia (capacidade {self.capacidade})")
        self.itens[self.fim] = item
        self.fim = (self.fim + 1) % self.capacidade
        self.total += 1
```

```
def dequeue(self):
    if self.total == 0:
        raise QueueUnderflowError("fila vazia")
    item = self.itens[self.inicio]
    self.itens[self.inicio] = None
    self.inicio = (self.inicio + 1) % self.capacidade
    self.total -= 1
    return item

def em_ordem(self):
    return [self.itens[(self.inicio + i) % self.capacidade] for i in range(self.total)]
```

2. O índice: uma BST dos clientes atendidos

A chave é o **identificador completo** do cliente, mas guardado como `(fila, número)` em vez da string `"B17"`. O motivo:

In [3]:

```
como_texto = sorted(["B2", "B17", "B1", "A10", "A9"])
como_chave = sorted([(("B", 2), ("B", 17), ("B", 1), ("A", 10), ("A", 9))])
print("comparando a string: ", como_texto)
print("comparando (fila, número):", [f"{fila}{numero}" for fila, numero in como_chave])
```

```
comparando a string:      ['A10', 'A9', 'B1', 'B17', 'B2']
comparando (fila, número): ['A9', 'A10', 'B1', 'B2', 'B17']
```

Comparando a string, `"B17"` vem antes de `"B2"` (compara `"1"` com `"2"`), e a listagem ordenada ficaria errada. Com `(fila, número)`, a fila é comparada primeiro e o número é comparado **como número**. A busca continua aceitando o texto `"B17"`: eu só converto na entrada.

In [4]:

```
class NoIndice:
    def __init__(self, chave, dados):
        self.chave = chave
        self.dados = dados
        self.esquerda = None
        self.direita = None

def para_chave(identificador):
    """'B17' → ('B', 17). Levanta ValueError se o texto não for um identificador válido."""
    if (not isinstance(identificador, str) or len(identificador) < 2
        or not identificador[0].isalpha() or not identificador[1:].isdigit()):
        raise ValueError(f"identificador inválido: {identificador!r} (o formato é letra + número, ex.: 'B17')")
    return identificador[0].upper(), int(identificador[1:])

class BinarySearchTree:
    def __init__(self):
        self.raiz = None
        self.tamanho = 0

    def insert(self, chave, dados):
        """Inserção iterativa. Devolve quantas comparações fez."""
```

```

novo = NoIndice(chave, dados)
self.tamanho += 1
if self.raiz is None:
    self.raiz = novo
    return 0
atual, comparacoes = self.raiz, 0
while True:
    comparacoes += 1
    if chave < atual.chave:
        if atual.esquerda is None:
            atual.esquerda = novo
            return comparacoes
        atual = atual.esquerda
    else:
        if atual.direita is None:
            atual.direita = novo
            return comparacoes
        atual = atual.direita

def search(self, identificador):
    """Devolve (dados ou None, comparações)."""
    chave = para_chave(identificador)
    atual, comparacoes = self.raiz, 0
    while atual is not None:
        comparacoes += 1
        if chave == atual.chave:
            return atual.dados, comparacoes
        atual = atual.esquerda if chave < atual.chave else atual.direita
    return None, comparacoes

def in_order(self):
    resultado, pilha, atual = [], [], self.raiz
    while pilha or atual is not None:
        while atual is not None:
            pilha.append(atual)
            atual = atual.esquerda
        atual = pilha.pop()
        resultado.append(atual.dados)
        atual = atual.direita
    return resultado

def altura(self):
    nivel, h = ([self.raiz] if self.raiz else []), 0
    while nivel:
        h += 1
        nivel = [f for no in nivel for f in (no.esquerda, no.direita) if f]
    return h

```

3. O simulador

Decisões:

- **O número do cliente é dado na chegada**, por fila (A_1 , A_2 , ... e B_1 , B_2 , ...). Se a fila estiver cheia, o cliente é recusado, mas o número dele fica no log — assim dá para rastrear quem foi recusado.
- **Timestamp lógico** = a posição do evento na string (o 1º evento é o tick 1). Com a chegada e a saída guardadas, sai também o tempo de espera.
- **Overflow e underflow não param a simulação**: viram uma linha de ERRO no log, com o evento, o tick, a fila e o estado dela naquele momento.
- **Letra de uma fila que não existe** (ex.: e , z) também vira erro no log, em vez de quebrar tudo.

In [5]:

```
class Simulador:
    FILAS = "ABCD"

    def __init__(self, capacidade=5):
        self.capacidade = capacidade
        self.filas = {nome: Queue(capacidade) for nome in self.FILAS}
        self.proximo_numero = {nome: 1 for nome in self.FILAS}
        self.indice = BinarySearchTree()
        self.log = []
        self.snapshots = []
        self.atendidos = 0

    def _registrar(self, tick, evento, tipo, mensagem):
        self.log.append({"tick": tick, "evento": evento, "tipo": tipo, "mensagem": mensagem})

    def processar(self, eventos):
        for tick, evento in enumerate(eventos, start=1):
            if not evento.isalpha():
                self._snapshot(tick, evento)
            elif evento.upper() not in self.filas:
                self._registrar(tick, evento, "ERRO", f"não existe fila '{evento.upper()}'")
            elif evento.islower():
                self._chegada(tick, evento, evento.upper())
            else:
                self._saida(tick, evento, evento)
        return self

    def _chegada(self, tick, evento, fila):
        numero = self.proximo_numero[fila]
        self.proximo_numero[fila] += 1
        identificador = f"{fila}{numero}"
        try:
            self.filas[fila].enqueue((identificador, tick))
            self._registrar(tick, evento, "chegada", f"{identificador} entrou na fila {fila}")
        except QueueOverflowError:
            espera = [c for c, _ in self.filas[fila].em_ordem()]
            self._registrar(tick, evento, "ERRO",
                           f"overflow: {identificador} recusado, fila {fila} cheia "
                           f"({self.capacidade}/{self.capacidade}: {' '.join(espera)})")

    def _saida(self, tick, evento, fila):
        try:
            identificador, chegada = self.filas[fila].dequeue()
        except QueueUnderflowError:
            self._registrar(tick, evento, "ERRO", f"underflow: saída pedida na fila {fila}, que está vazia")
            return
        self.atendidos += 1
        dados = {
            "cliente": identificador, "fila": fila, "ordem_de_atendimento": self.atendidos,
            "chegada": chegada, "saída": tick, "espera": tick - chegada,
        }
        self.indice.insert(para_chave(identificador), dados)
        self._registrar(tick, evento, "saída", f"{identificador} atendido (esperou {tick - chegada} ticks)")

    def _snapshot(self, tick, evento):
        estado = {nome: [c for c, _ in fila.em_ordem()] for nome, fila in self.filas.items()}
        self.snapshots.append((tick, estado))
        resumo = " | ".join(f"{nome}: {' '.join(ids) if ids else '-'}" for nome, ids in estado.items())
        self._registrar(tick, evento, "snapshot", resumo)
```

4. Rodando uma simulação

Capacidade **3** por fila, para provocar overflow. A string mistura chegadas, saídas, snapshots (`*` e `#`), um underflow (saída da fila D vazia) e uma fila inexistente (`e`).

In [6]:

```
eventos = "aabb*cABaaaa#bD-CBAecc*AAA."  
sim = Simulador(capacidade=3).processar(eventos)  
  
pd.set_option("display.max_colwidth", 110)  
log = pd.DataFrame(sim.log)  
log
```

Out[6]:

	tick	evento	tipo	mensagem
0	1	a	chegada	A1 entrou na fila A
1	2	a	chegada	A2 entrou na fila A
2	3	b	chegada	B1 entrou na fila B
3	4	b	chegada	B2 entrou na fila B
4	5	c	chegada	C1 entrou na fila C
5	6	*	snapshot	A: A1 A2 B: B1 B2 C: C1 D: —
6	7	A	saída	A1 atendido (esperou 6 ticks)
7	8	B	saída	B1 atendido (esperou 5 ticks)
8	9	a	chegada	A3 entrou na fila A
9	10	a	chegada	A4 entrou na fila A
10	11	a	ERRO	overflow: A5 recusado, fila A cheia (3/3: A2 A3 A4)
11	12	a	ERRO	overflow: A6 recusado, fila A cheia (3/3: A2 A3 A4)
12	13	a	ERRO	overflow: A7 recusado, fila A cheia (3/3: A2 A3 A4)
13	14	#	snapshot	A: A2 A3 A4 B: B2 C: C1 D: —
14	15	b	chegada	B3 entrou na fila B
15	16	D	ERRO	underflow: saída pedida na fila D, que está vazia
16	17	-	snapshot	A: A2 A3 A4 B: B2 B3 C: C1 D: —
17	18	C	saída	C1 atendido (esperou 13 ticks)
18	19	B	saída	B2 atendido (esperou 15 ticks)
19	20	A	saída	A2 atendido (esperou 18 ticks)
20	21	e	ERRO	não existe fila 'E'
21	22	c	chegada	C2 entrou na fila C
22	23	c	chegada	C3 entrou na fila C
23	24	*	snapshot	A: A3 A4 B: B3 C: C2 C3 D: —
24	25	A	saída	A3 atendido (esperou 16 ticks)

	tick	evento	tipo	mensagem
25	26	A	saída	A4 atendido (esperou 16 ticks)
26	27	A	ERRO	underflow: saída pedida na fila A, que está vazia
27	28	.	snapshot	A: — B: B3 C: C2 C3 D: —

In [7]:

```

erros = log[log["tipo"] == "ERRO"]
print(f"{len(eventos)} eventos: {sim.atendidos} atendimentos, {len(erros)} erros, "
      f"{len(sim.snapshots)} snapshots — e a simulação foi até o fim\n")
for _, linha in erros.iterrows():
    print(f"    tick {linha['tick']:>2} ({linha['evento']}) : {linha['mensagem']}")

```

28 eventos: 7 atendimentos, 6 erros, 5 snapshots — e a simulação foi até o fim

```

tick 11 ('a'): overflow: A5 recusado, fila A cheia (3/3: A2 A3 A4)
tick 12 ('a'): overflow: A6 recusado, fila A cheia (3/3: A2 A3 A4)
tick 13 ('a'): overflow: A7 recusado, fila A cheia (3/3: A2 A3 A4)
tick 16 ('D'): underflow: saída pedida na fila D, que está vazia
tick 21 ('e'): não existe fila 'E'
tick 27 ('A'): underflow: saída pedida na fila A, que está vazia

```

Os três tipos de erro aparecem no log com o contexto (qual cliente, qual fila, como ela estava) e **a simulação continua** depois de cada um: os eventos seguintes são processados normalmente.

5. O índice: busca e listagem ordenada

In [8]:

```

dados, comparacoes = sim.indice.search("A3")
print(f"busca por 'A3' → {dados} ({comparacoes} comparações)")

dados, comparacoes = sim.indice.search("B9")
print(f"busca por 'B9' → {dados} ({comparacoes} comparações: não foi atendido)")

print("\nlistagem in-order (ordem do identificador, não do atendimento):")
pd.DataFrame(sim.indice.in_order())

```

busca por 'A3' → {'cliente': 'A3', 'fila': 'A', 'ordem_de_atendimento': 6, 'chegada': 9, 'saida': 25, 'espera': 16} (4 comparações)
 busca por 'B9' → None (4 comparações: não foi atendido)

listagem in-order (ordem do identificador, não do atendimento):

Out[8]:

	cliente	fila	ordem_de_atendimento	chegada	saida	espera
0	A1	A		1	1	7
1	A2	A		5	2	20
2	A3	A		6	9	25
3	A4	A		7	10	26
4	B1	B		2	3	8
5	B2	B		4	4	19
6	C1	C		3	5	18

A in-order sai ordenada **por fila e depois pelo número** — `A1, A2, A3, ..., B1, B2` —, enquanto a coluna `ordem_de_atendimento` mostra que eles foram atendidos misturados. É exatamente para isso que serve o índice: responder por identificador sem depender da ordem em que as coisas aconteceram.

Custo de cada operação (`n` = clientes atendidos, `h` = altura do índice):

operação	custo	por quê
chegada (<code>a</code>)	$O(1)$	<code>enqueue</code> na fila circular
saída (<code>A</code>)	$O(1) + O(h)$	<code>dequeue</code> e depois inserir no índice
snapshot	$O(c)$	copia os <code>c</code> clientes que estão esperando nas filas
busca por identificador	$O(h)$	desce um caminho da raiz
listagem in-order	$O(n)$	visita cada nó uma vez

`h` fica entre $\log_2 n$ (árvore equilibrada) e `n` (árvore degenerada). A próxima seção mostra qual dos dois acontece aqui.

6. Um problema que só apareceu medindo: o índice degenera

A fila é **FIFO**: dentro de uma mesma fila, os clientes saem **na ordem em que chegaram** — `A1`, depois `A2`, depois `A3`. Então as chaves chegam no índice **em ordem crescente**, que é exatamente o pior caso da BST. Com quatro filas intercaladas, a árvore vira algo parecido com quatro "listas tortas".

Para medir, gero strings longas de eventos aleatórios e comparo a altura do índice com a altura de uma BST com as **mesmas chaves inseridas em ordem aleatória**:

In [9]:

```
def eventos_aleatorios(tamanho):
    letras = []
    for _ in range(tamanho):
        fila = random.choice("abcd")
        letras.append(fila if random.random() < 0.55 else fila.upper()) # um pouco mais de chegadas
    return "".join(letras)

linhas = []
for tamanho in [1_000, 2_000, 4_000, 8_000]:
    sim_grande = Simulador(capacidade=10_000).processar(eventos_aleatorios(tamanho))
    n = sim_grande.atendidos
    chaves = [para_chave(d["cliente"]) for d in sim_grande.indice.in_order()]

    embaralhadas = chaves.copy()
    random.shuffle(embaralhadas)
    comparacao = BinarySearchTree()
    for chave in embaralhadas:
        comparacao.insert(chave, None)

    custo_busca = sum(sim_grande.indice.search(f"{f}{num}") [1] for f, num in chaves) / n
    linhas.append({
        "atendidos (n)": n,
        "altura_indice_FIFO": sim_grande.indice.altura(),
        "altura / n": sim_grande.indice.altura() / n,
        "altura_mesmas_chaves_embaralhadas": comparacao.altura(),
        "comparacoes_por_busca": custo_busca,
    })

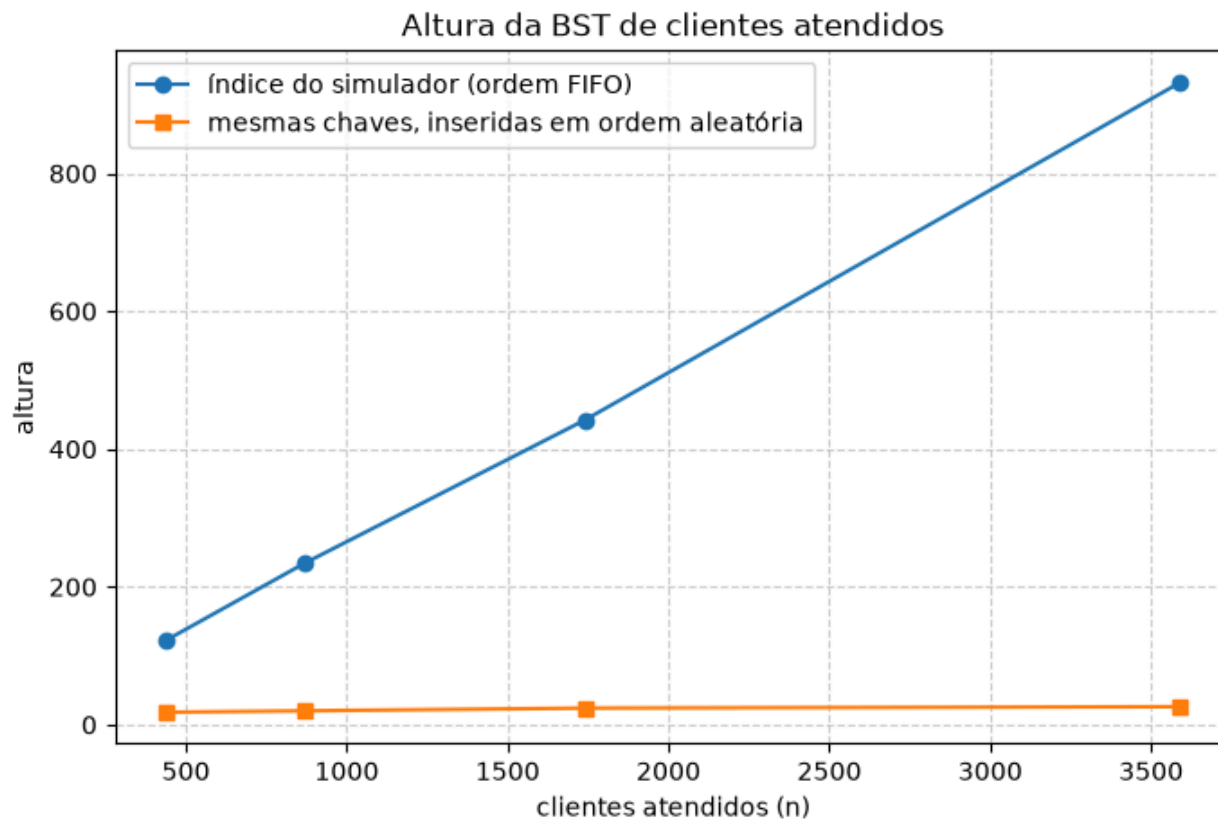
altura = pd.DataFrame(linhas)
altura
```

Out[9]:

	atendidos (n)	altura_indice_FIFO	altura / n	altura_mesmas_chaves_embaralhadas	comparacoes_por_busca
0	437	122	0.279176	18	58.624714
1	868	234	0.269585	20	111.426267
2	1740	442	0.254023	24	219.533333
3	3591	932	0.259538	26	452.619605

In [10]:

```
plt.figure(figsize=(8, 5))
plt.plot(altura["atendidos (n)"], altura["altura_indice_FIFO"], marker="o", label="índice do simulador (ordem FIFO)")
plt.plot(altura["atendidos (n)"], altura["altura_mesmas_chaves_embaralhadas"], marker="s",
        label="mesmas chaves, inseridas em ordem aleatória")
plt.title("Altura da BST de clientes atendidos")
plt.xlabel("clientes atendidos (n)")
plt.ylabel("altura")
plt.grid(True, linestyle="--", alpha=0.6)
plt.legend()
plt.show()
```



Resultado: a altura do índice cresce **junto com** n : altura / n fica estável em torno de 0,26 — perto de **1/4**, que é o esperado com quatro filas, cada uma formando a sua cadeia de chaves crescentes —, enquanto as mesmas chaves embaralhadas dão uma árvore baixa, perto de $2 \cdot \log_2 n$. Então, neste simulador, a busca por identificador é **$O(n)$** na prática, não $O(\log n)$ — e a causa é a própria regra FIFO.

Como resolver: usar uma árvore **balanceada** (como a **AVL**, que faz rotações para manter a altura em $O(\log n)$ qualquer que seja a ordem de inserção), ou, se só precisar de busca exata por identificador e não de listagem ordenada, uma **tabela hash** ($O(1)$ em média). Mantive a BST simples porque o enunciado pede BST, mas deixo registrado que é esse o gargalo.

7. Casos de borda

In [11]:

```
# string vazia: nada acontece, nada quebra
vazio = Simulador().processar("")
assert vazio.log == [] and vazio.atendidos == 0 and vazio.indice.in_order() == []

# só snapshots
```

```

so_snap = Simulador().processar("#1 ")
assert len(so_snap.snapshots) == 4 and so_snap.atendidos == 0

# só saídas: todas viram underflow, e a simulação segue
so_saida = Simulador().processar("ABCD")
assert all(e["tipo"] == "ERRO" for e in so_saida.log) and len(so_saida.log) == 4

# fila de capacidade 1: o segundo cliente é recusado, o terceiro entra depois que o primeiro sai
cap1 = Simulador(capacidade=1).processar("aaAa")
assert [e["tipo"] for e in cap1.log] == ["chegada", "ERRO", "saída", "chegada"]
assert cap1.indice.search("A1")[0]["cliente"] == "A1"
assert cap1.indice.search("A2")[0] is None # o A2 foi recusado, nunca foi atendido

# a busca aceita minúscula e recusa identificador malformado
assert cap1.indice.search("a1")[0]["cliente"] == "A1"
for invalido in ["", "B", "17", "B1x", None]:
    try:
        cap1.indice.search(invalido)
        raise AssertionError(f"deveria recusar {invalido!r}")
    except ValueError:
        pass

print("todos os casos de borda passaram")

```

todos os casos de borda passaram

Exercício 7 — Recursão para navegação em árvore de diretórios

Henrique Goldstein Maluhy Mendes Amaral · DR1_AT — Algoritmos e Estruturas de Dados · Infnet

Para rodar: **Ambiente de execução** → **Executar tudo**. O notebook é independente, não precisa de nenhum arquivo externo.

Roteiro: (1) nós de diretório e arquivo em memória · (2) `walk(root)` recursivo em pré-ordem, com o caminho atual numa lista encadeada · (3) `DirectoryTree` com `insert`, `search` e `delete`, e a política de remoção · (4) pilha de caminho: lista encadeada × lista do Python · (5) custo e profundidade · (6) casos de borda

Tudo acontece **em memória**: o notebook não lê nem escreve no disco de verdade, então roda igual em qualquer máquina.

In [1]:

```

import random
import sys
import time

import pandas as pd

random.seed(42)

```

1. A estrutura

Cada nó tem **nome**, **tipo** ("dir" ou "arquivo") e **filhos**. Só diretório tem filhos.

In [2]:

```
class NoArquivo:
    def __init__(self, nome, tipo):
        self.nome = nome
        self.tipo = tipo
        self.filhos = []

    def eh_diretorio(self):
        return self.tipo == "dir"

    def filho(self, nome):
        """Procura um filho pelo nome (busca linear entre os filhos)."""
        for f in self.filhos:
            if f.nome == nome:
                return f
        return None
```

2. A pilha do caminho atual, em lista encadeada

Durante a travessia eu preciso saber **onde estou**: ao entrar num diretório empilho o nome dele, ao sair desempilho. Implemento essa pilha como **lista encadeada**: cada nó guarda um nome e aponta para o nó de baixo.

- `push` e `pop` mexem só no topo: **$O(1)$** ;
- montar o caminho completo (`/projeto/src/main.py`) precisa percorrer a pilha inteira: **$O(d)$** , onde `d` é a profundidade.

In [3]:

```
class NoCaminho:
    def __init__(self, nome, abaixo):
        self.nome = nome
        self.abaixo = abaixo

class PathStack:
    """Pilha de nomes em lista encadeada: o topo é a pasta mais funda."""
    def __init__(self):
        self.topo = None
        self.tamanho = 0

    def push(self, nome):
        self.topo = NoCaminho(nome, self.topo)
        self.tamanho += 1

    def pop(self):
        if self.topo is None:
            raise IndexError("pop em pilha de caminho vazia")
        nome = self.topo.nome
```

```

self.topo = self.topo.abaixo
self.tamanho -= 1
return nome

def caminho(self):
    # o topo é o nome mais fundo, então junto de cima para baixo e inverte
    nomes = []
    atual = self.topo
    while atual is not None:
        nomes.append(atual.nome)
        atual = atual.abaixo
    return "/" + "/".join(reversed(nomes))

```

3. `walk(root)` recursivo

Pré-ordem: primeiro registro o nó atual, depois desço em cada filho, na ordem em que foram inseridos.

- **Caso base:** um arquivo (ou um diretório vazio) não tem filhos — o laço não roda e a recursão para ali.
- **Composição:** o resultado de um diretório é o caminho dele seguido dos resultados de cada filho.

In [4]:

```

def walk(root):
    caminhos = []
    pilha = PathStack()

    def visitar(no):
        if no is not root:
            pilha.push(no.nome)           # entrei neste nó
            caminhos.append(pilha.caminho())
            for filho in no.filhos:
                visitar(filho)           # caso base: sem filhos, nada a fazer
            if no is root:
                pilha.pop()              # sai dele: volto para o pai

    visitar(root)
    return caminhos

```

4. `DirectoryTree`

Validações:

- o caminho precisa ser absoluto (`/a/b`), sem partes vazias (`//`) e sem `.` ou `..` ;
- `insert` exige que o pai exista e seja diretório, e que o nome ainda não exista ali;
- `node_type` só pode ser `"dir"` ou `"arquivo"` ;
- a raiz `/` não pode ser inserida nem removida.

Política de remoção: remover um **diretório que tem filhos** exige `recursive=True` ; sem isso, levanta `DiretorioNaoVazioError` . É o mesmo comportamento do `rmdir` × `rm -r` :
apagar uma pasta cheia sem querer perde uma subárvore inteira, então esse caso precisa ser pedido explicitamente. Arquivo e diretório vazio são removidos direto.

In [5]:

```
class DiretorioNaoVazioError(Exception):
    """Tentativa de remover um diretório com conteúdo sem recursive=True."""

class DirectoryTree:
    def __init__(self):
        self.root = NoArquivo("", "dir")

    def _partes(self, path):
        if not isinstance(path, str) or not path.startswith("/"):
            raise ValueError(f"o caminho precisa ser absoluto (começar com '/'): {path!r}")
        if path == "/":
            return []
        partes = path.strip("/").split("/")
        for parte in partes:
            if parte in ("", ".", ".."):
                raise ValueError(f"parte inválida {parte!r} no caminho {path!r}")
        return partes

    def search(self, path):
        atual = self.root
        for nome in self._partes(path):
            if not atual.eh_diretorio():
                return None  # tentei entrar dentro de um arquivo
            atual = atual.filho(nome)
            if atual is None:
                return None
        return atual

    def insert(self, path, node_type):
        if node_type not in ("dir", "arquivo"):
            raise ValueError(f"tipo inválido {node_type!r}: use 'dir' ou 'arquivo'")
        partes = self._partes(path)
        if not partes:
            raise ValueError("a raiz já existe e não pode ser inserida")
        pai_path = "/" + "/".join(partes[:-1])
        pai = self.search(pai_path)
        if pai is None:
            raise FileNotFoundError(f"o diretório pai {pai_path!r} não existe")
        if not pai.eh_diretorio():
            raise NotADirectoryError(f"{pai_path!r} é um arquivo, não pode ter filhos")
        if pai.filho(partes[-1]) is not None:
            raise FileExistsError(f"{path!r} já existe")
        novo = NoArquivo(partes[-1], node_type)
        pai.filhos.append(novo)
        return novo

    def delete(self, path, recursive=False):
        """Remove e devolve quantos nós saíram da árvore."""
        partes = self._partes(path)
        if not partes:
            raise ValueError("a raiz não pode ser removida")
        pai = self.search("/" + "/".join(partes[:-1]))
        alvo = pai.filho(partes[-1]) if pai is not None and pai.eh_diretorio() else None
        if alvo is None:
            raise FileNotFoundError(f"{path!r} não existe")
        if alvo.filhos and not recursive:
            raise DiretorioNaoVazioError(
                f"{path!r} tem {len(alvo.filhos)} item(ns); use recursive=True para apagar tudo")
        removidos = contar(alvo)
```

```

        pai.filhos.remove(alvo)
        return removidos

def contar(no):
    """Quantos nós há na subárvore (recursivo, mesma forma do walk)."""
    return 1 + sum(contar(filho) for filho in no.filhos)

```

Montando uma árvore de exemplo

In [6]:

```

arvore = DirectoryTree()
for path, tipo in [
    ("/projeto", "dir"),
    ("/projeto/src", "dir"),
    ("/projeto/src/main.py", "arquivo"),
    ("/projeto/src/util.py", "arquivo"),
    ("/projeto/tests", "dir"),
    ("/projeto/tests/test_main.py", "arquivo"),
    ("/projeto/README.md", "arquivo"),
    ("/docs", "dir"),
    ("/docs/manual.pdf", "arquivo"),
    ("/vazia", "dir"),
]:
    arvore.insert(path, tipo)

for caminho in walk(arvore.root):
    no = arvore.search(caminho)
    print(f"{caminho:<30} {'dir' if no.eh_diretorio() else 'arquivo'}")

```

```

/                               dir
/projeto                       dir
/projeto/src                   dir
/projeto/src/main.py           arquivo
/projeto/src/util.py           arquivo
/projeto/tests                 dir
/projeto/tests/test_main.py    arquivo
/projeto/README.md             arquivo
/docs                          dir
/docs/manual.pdf               arquivo
/vazia                         dir

```

In [7]:

```

esperado = ["/", "/projeto", "/projeto/src", "/projeto/src/main.py", "/projeto/src/util.py",
            "/projeto/tests", "/projeto/tests/test_main.py", "/projeto/README.md",
            "/docs", "/docs/manual.pdf", "/vazia"]
assert walk(arvore.root) == esperado      # pré-ordem, na ordem de inserção

# busca
assert arvore.search("/projeto/src/util.py").nome == "util.py"
assert arvore.search("/projeto/nada") is None
assert arvore.search("/projeto/README.md/dentro") is None  # não existe nada "dentro" de um arquivo

# validações do insert
for path, tipo, erro_esperado in [
    ("/projeto/src", "dir", FileExistsError),
    ("/inexistente/a.txt", "arquivo", FileNotFoundError),

```

```

        ("/projeto/README.md/x", "arquivo", NotADirectoryError),
        ("projeto/b", "dir", ValueError),
        ("/projeto//b", "dir", ValueError),
        ("/projeto/./b", "dir", ValueError),
        ("/projeto/b", "pasta", ValueError),
    ]:
    try:
        arvore.insert(path, tipo)
        raise AssertionError(f"insert({path!r}) deveria falhar")
    except erro_esperado as erro:
        print(f"insert({path!r}) → {type(erro).__name__}: {erro}")

```

insert('/projeto/src') → FileExistsError: '/projeto/src' já existe
 insert('/inexistente/a.txt') → FileNotFoundError: o diretório pai '/inexistente' não existe
 insert('/projeto/README.md/x') → NotADirectoryError: '/projeto/README.md' é um arquivo, não pode ter filhos
 insert('projeto/b') → ValueError: o caminho precisa ser absoluto (começar com '/'): 'projeto/b'
 insert('/projeto//b') → ValueError: parte inválida '' no caminho '/projeto//b'
 insert('/projeto/./b') → ValueError: parte inválida '..' no caminho '/projeto/./b'
 insert('/projeto/b') → ValueError: tipo inválido 'pasta': use 'dir' ou 'arquivo'

A política de remoção em ação

In [8]:

```

# diretório com conteúdo, sem a flag: recusa e não mexe em nada
antes = walk(arvore.root)
try:
    arvore.delete("/projeto/src")
except DiretorioNaoVazioError as erro:
    print(f"delete('/projeto/src') → {erro}")
assert walk(arvore.root) == antes

# com recursive=True, a subárvore inteira sai
print(f"delete('/projeto/src', recursive=True) removeu {arvore.delete('/projeto/src', recursive=True)} nós")
assert arvore.search("/projeto/src") is None and arvore.search("/projeto/src/main.py") is None

# arquivo e diretório vazio saem direto
assert arvore.delete("/docs/manual.pdf") == 1
assert arvore.delete("/vazia") == 1

# o resto continua consistente
print("\ndepois das remoções:")
for caminho in walk(arvore.root):
    print(" ", caminho)
assert walk(arvore.root) == ["/", "/projeto", "/projeto/tests", "/projeto/tests/test_main.py",
                             "/projeto/README.md", "/docs"]

```

delete('/projeto/src') → '/projeto/src' tem 2 item(ns); use recursive=True para apagar tudo
 delete('/projeto/src', recursive=True) removeu 3 nós

depois das remoções:

```

/
/projeto
/projeto/tests
/projeto/tests/test_main.py
/projeto/README.md
/docs

```

5. Pilha de caminho: lista encadeada × lista do Python

As duas oferecem as mesmas operações. Comparando as **operações fundamentais**:

operação	lista encadeada (PathStack)	lista do Python
push	O(1) — cria um nó e aponta para o topo	O(1) amortizado — <code>append</code> , às vezes realoca o vetor
pop	O(1) — anda o topo um nó para baixo	O(1) — <code>pop()</code> do fim
ver o topo	O(1)	O(1)
montar o caminho completo	O(d) — percorre os nós e inverte	O(d) — <code>"/".join(lista)</code> , já está na ordem certa
acessar o nível <code>i</code>	O(d) — precisa descer nó a nó	O(1) — índice direto
memória	um objeto nó por nível	um bloco contínuo de referências

Para ver na prática, rodo a mesma travessia nas duas versões, contando as operações e medindo o tempo:

In [9]:

```
def gerar_arvore(n):
    """Árvore aleatória com n nós: cada novo item vai para um diretório sorteado."""
    t = DirectoryTree()
    diretorios = [""] # "" representa a raiz
    for i in range(n):
        pai = random.choice(diretorios)
        eh_dir = random.random() < 0.4
        caminho = f"{pai}/{ 'd' if eh_dir else 'f' }{i}"
        t.insert(caminho, "dir" if eh_dir else "arquivo")
        if eh_dir:
            diretorios.append(caminho)
    return t

def walk_com_lista_python(root):
    """Mesmo walk, mas com a pilha do caminho numa lista do Python."""
    caminhos, pilha = [], []

    def visitar(no):
        if no is not root:
            pilha.append(no.nome)
            caminhos.append("/" + "/".join(pilha))
            for filho in no.filhos:
                visitar(filho)
        if no is not root:
            pilha.pop()

    visitar(root)
    return caminhos

linhas = []
for n in [1_000, 5_000, 20_000]:
    t = gerar_arvore(n)
```

```

inicio = time.perf_counter()
com_encadeada = walk(t.root)
tempo_encadeada = time.perf_counter() - inicio

inicio = time.perf_counter()
com_python = walk_com_lista_python(t.root)
tempo_python = time.perf_counter() - inicio

assert com_encadeada == com_python # mesmo resultado, caminho por caminho
profundidades = [c.count("/") for c in com_encadeada[1:]]
linhas.append({
    "nos": n + 1,
    "push e pop (cada)": n,
    "profundidade_media": sum(profundidades) / n,
    "passos_para_montar_caminhos": sum(profundidades),
    "tempo_encadeada_ms": tempo_encadeada * 1000,
    "tempo_lista_python_ms": tempo_python * 1000,
    "encadeada / python": tempo_encadeada / tempo_python,
})

pd.DataFrame(linhas)

```

Out[9]:

	nos	push e pop (cada)	profundidade_media	passos_para_montar_caminhos	tempo_encadeada_ms	tempo_lista_python_ms	encadeada / python
0	1001	1000	6.95100	6951	0.8121	0.2297	3.535481
1	5001	5000	7.63440	38172	4.0420	1.1390	3.548727
2	20001	20000	8.76485	175297	18.0688	5.7518	3.141417

- **As duas fazem as mesmas operações:** um `push` e um `pop` por nó (menos a raiz), e devolvem exatamente os mesmos caminhos.
- **O que domina o custo do `walk` é montar o caminho de cada nó**, $O(d)$ por nó. No total o `walk` é $O(n \cdot d)$: a coluna `passos_para_montar_caminhos` é n vezes a profundidade média.
- **A lista encadeada perde no tempo**, e o motivo está na tabela de operações: na hora de montar o caminho ela precisa **percorrer os nós e inverter a ordem**, enquanto a lista do Python já guarda os nomes na ordem certa e faz um `join` direto num bloco contínuo. Em Big O são iguais ($O(d)$); a diferença é de constante.

Conclusão: para uma pilha de caminho, a lista do Python é a escolha prática. A lista encadeada só compensaria se eu precisasse inserir ou remover **no meio** da pilha, o que uma pilha nunca faz.

6. O limite da recursão

O `walk` recursivo usa uma chamada por nível de profundidade. Numa árvore com mais níveis do que o limite de recursão do Python, ele quebra:

In [10]:

```

limite = sys.getrecursionlimit()
funda = DirectoryTree()

```

```

caminho = ""
for i in range(limite + 500):
    caminho += f"/n{i}"
    funda.insert(caminho, "dir")
print(f"limite de recursão: {limite}; árvore com {limite + 500} níveis")
try:
    walk(funda.root)
    print("walk: terminou")
except RecursionError:
    print("walk recursivo: RecursionError – a árvore é mais funda que o limite")

```

limite de recursão: 1000; árvore com 1500 níveis
walk recursivo: RecursionError – a árvore é mais funda que o limite

Em sistemas de arquivos reais a profundidade raramente passa de algumas dezenas, então a recursão é segura na prática. Mas se a árvore puder ser muito funda (por exemplo, vindo de um usuário), a versão iterativa — trocando a pilha de chamadas por uma pilha explícita, como no exercício 5 — é a que não quebra.

Resumo dos custos (`n` nós, `d` profundidade, `k` filhos por diretório):

operação	custo
<code>walk</code>	$O(n \cdot d)$ — visita cada nó uma vez e monta um caminho de tamanho <code>d</code>
<code>search</code>	$O(d \cdot k)$ — em cada nível procura o filho entre <code>k</code> irmãos
<code>insert</code>	$O(d \cdot k)$ — acha o pai e confere se o nome já existe
<code>delete</code>	$O(d \cdot k)$ para achar + $O(\text{tamanho da subárvore})$ para contar o que sai
<code>push</code> / <code>pop</code> na pilha de caminho	$O(1)$

7. Casos de borda

In [11]:

```

t = DirectoryTree()
assert walk(t.root) == ["/"] # árvore só com a raiz
assert t.search("/") is t.root

for chamada, erro_esperado in [
    (lambda: t.delete("/"), ValueError), # a raiz não se remove
    (lambda: t.insert("/", "dir"), ValueError), # nem se insere de novo
    (lambda: t.delete("/nada"), FileNotFoundError), # remover o que não existe
    (lambda: t.search(None), ValueError), # caminho que nem é texto
]:
    try:
        chamada()
        raise AssertionError("deveria ter falhado")
    except erro_esperado:
        pass

# barra no final é aceita
t.insert("/a/", "dir")
assert t.search("/a") is not None and t.search("/a/") is not None

```

```
# nomes iguais em diretórios diferentes não conflitam
t.insert("/b", "dir")
t.insert("/a/x", "arquivo")
t.insert("/b/x", "arquivo")
assert walk(t.root) == ["/", "/a", "/a/x", "/b", "/b/x"]

# pilha de caminho vazia
try:
    PathStack().pop()
except IndexError as erro:
    print(f"pop na pilha vazia → {erro}")
assert PathStack().caminho() == "/"

print("todos os casos de borda passaram")
```

pop na pilha vazia → pop em pilha de caminho vazia
 todos os casos de borda passaram

Exercício 8 — Programação dinâmica e memoization com hashtable e árvore de subproblemas

Henrique Goldstein Maluhy Mendes Amaral · DR1_AT — Algoritmos e Estruturas de Dados · Infnet

Para rodar: **Ambiente de execução** → **Executar tudo**. O notebook é independente, não precisa de nenhum arquivo externo.

Roteiro: (1) o problema e por que o guloso falha · (2) solução recursiva pura, contando chamadas e subproblemas · (3) memoization com a minha `HashTableChained` · (4) a chave do memo · (5) árvore de subproblemas e busca por estado · (6) versão com tabela (bottom-up) · (7) casos de borda

In [1]:

```
import sys

import matplotlib.pyplot as plt
import pandas as pd

INFINITO = float("inf")
```

1. O problema: `min_coins(amount, coins)`

Qual o **menor número de moedas** que somam `amount`, podendo repetir moedas? Se não der para formar o valor, a resposta é `-1`.

Por que não resolver "pegando sempre a maior moeda que cabe" (guloso)? Porque falha. Com moedas `[1, 3, 4]` e valor `6`:

In [2]:

```
def guloso(amount, coins):
    moedas = []
    for c in sorted(coins, reverse=True):
        while amount >= c:
            amount -= c
            moedas.append(c)
    return moedas

print("guloso para 6 com [1, 3, 4]:", guloso(6, [1, 3, 4]), "→ 3 moedas")
print("mas 6 = 3 + 3          → 2 moedas")
```

```
guloso para 6 com [1, 3, 4]: [4, 1, 1] → 3 moedas
mas 6 = 3 + 3          → 2 moedas
```

Para achar o ótimo é preciso **testar todas as escolhas** de primeira moeda: $\text{min_coins}(a) = 1 + \min(\text{min_coins}(a - c))$ para cada moeda $c \leq a$. Isso gera muita repetição: `min_coins(2)`, por exemplo, é calculado de novo em vários ramos diferentes. É a **sobreposição de subproblemas** que a programação dinâmica resolve.

2. Solução recursiva pura

Conto as **chamadas recursivas** e os **subproblemas distintos** (quantos valores diferentes de `amount` apareceram). Para contar os distintos uso um `set` só como instrumento de medição — ele não participa do cálculo.

In [3]:

```
class Contador:
    def __init__(self):
        self.chamadas = 0
        self.estados = set()

def validar(amount, coins):
    if amount < 0:
        raise ValueError(f"amount não pode ser negativo: {amount}")
    if not coins or any(c <= 0 for c in coins):
        raise ValueError(f"as moedas precisam ser positivas: {coins}")

def min_coins_puro(amount, coins, cont):
    cont.chamadas += 1
    cont.estados.add(amount)
    if amount == 0:
        return 0 # caso base: zero moedas formam zero
    melhor = INFINITO
    for c in coins:
        if c <= amount:
            resto = min_coins_puro(amount - c, coins, cont)
            melhor = min(melhor, resto + 1)
    return melhor

def min_coins(amount, coins):
    validar(amount, coins)
    cont = Contador()
    r = min_coins_puro(amount, coins, cont)
    return (-1 if r == INFINITO else r), cont
```

```

assert min_coins(6, [1, 3, 4])[0] == 2
assert min_coins(0, [1, 3, 4])[0] == 0
assert min_coins(7, [2, 4])[0] == -1      # impossível: só dá para formar números pares
print("versão pura conferida")

```

versão pura conferida

3. Memoization com a `HashTableChained`

A mesma recursão, mas antes de calcular eu pergunto ao **memo** se aquele estado já foi resolvido. O memo é a hashtable que eu implementei (buckets em lista encadeada, rehash com fator de carga 0,75) — **não** uso `dict` nem `lru_cache`.

In [4]:

```

class NoBucket:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.next = None

class HashTableChained:
    LIMAR = 0.75

    def __init__(self, capacidade=8):
        self.capacidade = capacidade
        self.buckets = [None] * capacidade
        self.quantidade = 0

    def _indice(self, key):
        return hash(key) % self.capacidade  # chaves aqui são inteiros ou tuplas de inteiros

    def put(self, key, value):
        i = self._indice(key)
        atual = self.buckets[i]
        while atual is not None:
            if atual.key == key:
                atual.value = value
                return
            atual = atual.next
        novo = NoBucket(key, value)
        novo.next = self.buckets[i]
        self.buckets[i] = novo
        self.quantidade += 1
        if self.quantidade / self.capacidade > self.LIMAR:
            self._rehash()

    def get(self, key):
        atual = self.buckets[self._indice(key)]
        while atual is not None:
            if atual.key == key:
                return atual.value
            atual = atual.next
        raise KeyError(key)

    def __contains__(self, key):
        try:
            self.get(key)
            return True

```

```

        except KeyError:
            return False

    def __len__(self):
        return self.quantidade

    def _rehash(self):
        antigos = self.buckets
        self.capacidade *= 2
        self.buckets = [None] * self.capacidade
        for cabeca in antigos:
            atual = cabeca
            while atual is not None:
                proximo = atual.next
                i = self._indice(atual.key)
                atual.next = self.buckets[i]
                self.buckets[i] = atual
                atual = proximo

def min_coins_memo(amount, coins, cont, memo):
    cont.chamadas += 1
    if amount in memo:
        return memo.get(amount)      # já resolvido: não recalcula
    if amount == 0:
        resultado = 0
    else:
        resultado = INFINITO
        for c in coins:
            if c <= amount:
                resultado = min(resultado, min_coins_memo(amount - c, coins, cont, memo) + 1)
    memo.put(amount, resultado)
    return resultado

def min_coins_memoizado(amount, coins):
    validar(amount, coins)
    cont, memo = Contador(), HashTableChained()
    r = min_coins_memo(amount, coins, cont, memo)
    return (-1 if r == INFINITO else r), cont, len(memo)

assert min_coins_memoizado(6, [1, 3, 4])[0] == 2
assert min_coins_memoizado(7, [2, 4])[0] == -1
print("versão memoizada conferida")

```

versão memoizada conferida

A redução, medida

Mesmas moedas `[1, 3, 4]`, valores crescentes. As duas versões têm que dar a **mesma resposta**:

In [5]:

```

moedas = [1, 3, 4]
linhas = []
for amount in [5, 10, 15, 20, 22, 24, 26]:
    r_puro, c_puro = min_coins(amount, moedas)
    r_memo, c_memo, distintos_memo = min_coins_memoizado(amount, moedas)
    assert r_puro == r_memo
    linhas.append({
        "amount": amount, "moedas_minimas": r_memo,

```

```

    "chamadas_puro": c_puro.chamadas,
    "subproblemas_distintos": len(c_puro.estados),
    "chamadas_memo": c_memo.chamadas,
    "estados_no_memo": distintos_memo,
    "reducao (puro / memo)": c_puro.chamadas / c_memo.chamadas,
})

dp = pd.DataFrame(linhas)
# quanto as chamadas se multiplicam a cada unidade a mais de amount
dp["crescimento_por_unidade"] = (dp["chamadas_puro"] / dp["chamadas_puro"].shift(1)) ** (1 / dp["amount"].diff())
dp["memo / (amount*k)"] = dp["chamadas_memo"] / (dp["amount"] * len(moedas))
dp
```

Out[5]:

	amount	moedas_minimas	chamadas_puro	subproblemas_distintos	chamadas_memo	estados_no_memo	reducao (puro / memo)	crescimento_por_unidade	memo / (amount*k)
0	5	2	15	6	11	6	1.363636	NaN	0.733333
1	10	3	168	11	26	11	6.461538	1.621226	0.866667
2	15	4	1869	16	41	16	45.585366	1.619049	0.911111
3	20	5	20736	21	56	21	370.285714	1.618166	0.933333
4	22	6	54288	23	62	23	875.612903	1.618041	0.939394
5	24	6	142129	25	68	25	2090.132353	1.618041	0.944444
6	26	7	372099	27	74	27	5028.364865	1.618035	0.948718

O que a tabela mostra:

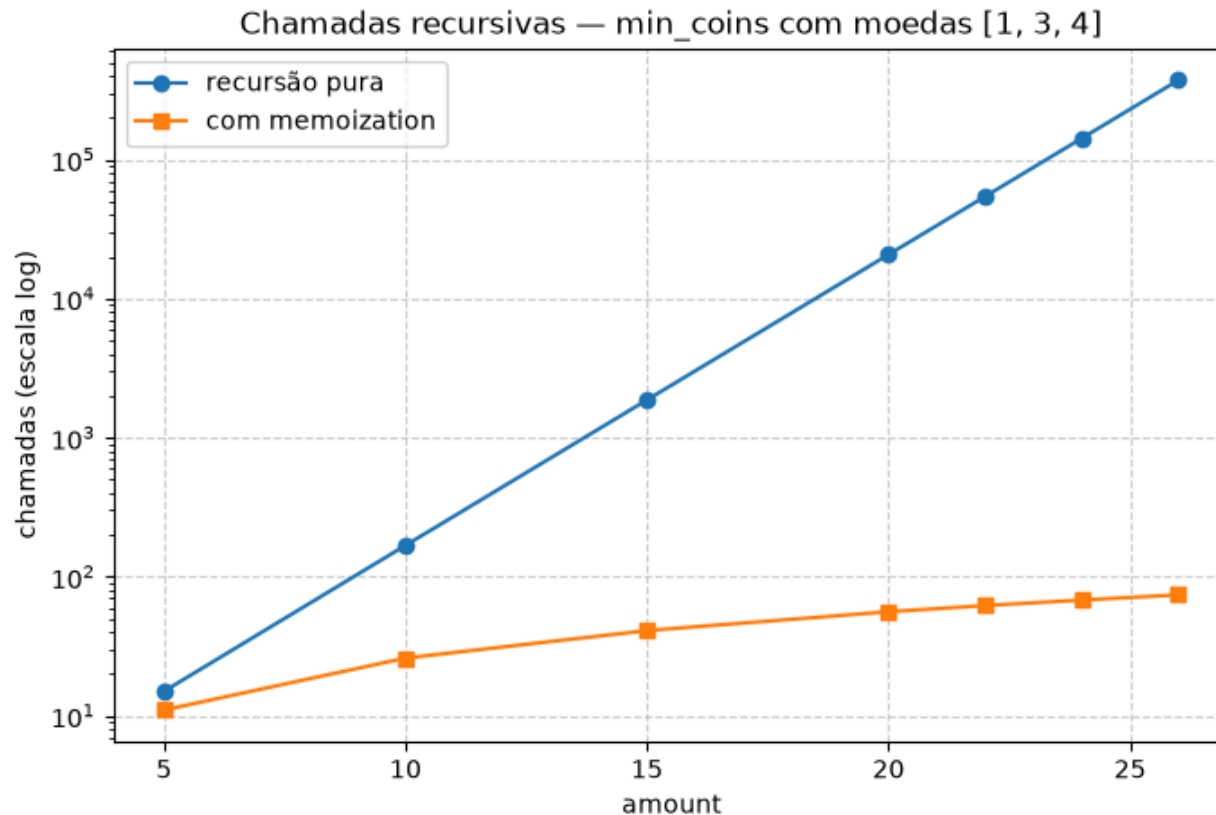
- **Pura:** as chamadas explodem. Os subproblemas **distintos** são só `amount + 1` (os valores de 0 até `amount`), mas cada um é recalculado milhares de vezes.
- **O crescimento é exponencial, e dá para medir o fator:** `crescimento_por_unidade` fica fixo em $\approx 1,618$ — cada unidade a mais em `amount` multiplica as chamadas por esse mesmo número. Com as moedas `[1, 3, 4]`, as chamadas seguem $T(a) = T(a-1) + T(a-3) + T(a-4)$, e o polinômio dessa recorrência é $x^4 - x^3 - x - 1 = (x^2 + 1)(x^2 - x - 1)$: a maior raiz é a **razão áurea** $\varphi = 1,618$. Então a versão pura é $\Theta(\varphi^{\text{amount}})$.
- **Memoizada:** `estados_no_memo` é igual a `subproblemas_distintos` — cada estado é **calculado uma vez só**. As chamadas passam a ser no máximo `k` por estado (uma por moeda), e `memo / (amount*k)` fica abaixo de 1, se aproximando dele conforme `amount` cresce (nos valores pequenos algumas moedas não cabem): custo **$O(\text{amount} \cdot k)$** .
- A **redução cresce sem parar** com `amount`: é a diferença entre uma função exponencial e uma linear.

In [6]:

```

plt.figure(figsize=(8, 5))
plt.plot(dp["amount"], dp["chamadas_puro"], marker="o", label="recursão pura")
plt.plot(dp["amount"], dp["chamadas_memo"], marker="s", label="com memoization")
plt.yscale("log")
plt.title("Chamadas recursivas - min_coins com moedas [1, 3, 4]")
```

```
plt.xlabel("amount")
plt.ylabel("chamadas (escala log)")
plt.grid(True, linestyle="--", alpha=0.6)
plt.legend()
plt.show()
```



Com o eixo vertical em escala log, a recursão pura vira uma **reta subindo** — assinatura de crescimento exponencial —, enquanto a memoizada fica praticamente no chão.

4. A chave do memo: o estado mínimo suficiente

A chave é **só o amount**. Isso basta porque a resposta de um subproblema depende apenas de quanto falta formar; o conjunto de moedas é o **mesmo** em toda a execução, então não precisa entrar na chave.

A chave precisa ser **suficiente** (senão o memo devolve resposta errada) e **mínima** (senão o memo não reaproveita nada). Para mostrar o efeito de uma chave "grande demais", comparo com uma chave que inclui também a **profundidade** da chamada (quantas moedas já foram usadas). Ela é correta, mas trata como diferentes estados

que são o mesmo problema:

In [7]:

```
def min_coins_chave_grande(amount, coins, profundidade, cont, memo):
    cont.chamadas += 1
    chave = (amount, profundidade)      # informação a mais na chave
    if chave in memo:
        return memo.get(chave)
    if amount == 0:
        resultado = 0
    else:
        resultado = INFINITO
        for c in coins:
            if c <= amount:
                resultado = min(resultado,
                                min_coins_chave_grande(amount - c, coins, profundidade + 1, cont, memo) + 1)
    memo.put(chave, resultado)
    return resultado

linhas = []
for amount in [10, 20, 30, 40]:
    c_minima, m_minima = Contador(), HashTableChained()
    min_coins_memo(amount, moedas, c_minima, m_minima)
    c_grande, m_grande = Contador(), HashTableChained()
    min_coins_chave_grande(amount, moedas, 0, c_grande, m_grande)
    linhas.append({
        "amount": amount,
        "chave = amount: chamadas": c_minima.chamadas, "estados": len(m_minima),
        "chave = (amount, profundidade): chamadas": c_grande.chamadas, "estados": len(m_grande),
    })
pd.DataFrame(linhas)
```

Out[7]:

	amount	chave = amount: chamadas	estados	chave = (amount, profundidade): chamadas	estados
0	10	26	11	68	39
1	20	56	21	346	152
2	30	86	31	848	339
3	40	116	41	1576	602

Com a chave grande, o memo guarda **much more states** e faz **much more calls**, porque "faltam 5" alcançado com 2 moedas e "faltam 5" alcançado com 3 moedas viram entradas diferentes — apesar de terem a mesma resposta. A chave `amount` é a menor que ainda determina a resposta.

5. A árvore de subproblemas

Numa execução memoizada, construo a árvore de chamadas: **cada nó é um estado** (o `amount`) e **os filhos são as transições** (tirar cada moeda). Quando um estado já estava no memo, ele aparece como folha marcada `(memo)` — é ali que a recursão foi **podada**.

In [8]:

```
class NoSubproblema:
    def __init__(self, estado):
        self.estado = estado
        self.filhos = []
        self.do_memo = False

def arvore_de_subproblemas(amount, coins):
    memo = HashTableChained()

    def resolver(a):
        no = NoSubproblema(a)
        if a in memo:
            no.do_memo = True
            return memo.get(a), no
        melhor = 0 if a == 0 else INFINITO
        for c in coins:
            if c <= a:
                r, filho = resolver(a - c)
                no.filhos.append(filho)
                melhor = min(melhor, r + 1)
        memo.put(a, melhor)
        return melhor, no

    return resolver(amount)

def imprimir(no, nivel=0):
    marca = " (memo)" if no.do_memo else ""
    print(" " * nivel + f"faltam {no.estado}{marca}")
    for filho in no.filhos:
        imprimir(filho, nivel + 1)

resposta, raiz = arvore_de_subproblemas(6, [1, 3, 4])
print(f"min_coins(6, [1, 3, 4]) = {resposta}\n")
imprimir(raiz)
```

```
min_coins(6, [1, 3, 4]) = 2
```

```
faltam 6
  faltam 5
    faltam 4
      faltam 3
        faltam 2
          faltam 1
            faltam 0
              faltam 0 (memo)
                faltam 1 (memo)
                  faltam 0 (memo)
                    faltam 2 (memo)
                      faltam 1 (memo)
                        faltam 3 (memo)
                          faltam 2 (memo)
```

Na árvore dá para ver a poda: o primeiro ramo (faltam 5 → 4 → 3 → ...) resolve os estados pequenos, e todos os outros ramos que chegam nesses estados param imediatamente, marcados (memo) .

Busca por um estado na árvore

A árvore **não é ordenada** pelo estado (não é uma BST), então achar um estado exige percorrer os nós: uma busca em profundidade com pilha explícita. Devolvo o nó onde o estado foi **realmente calculado** (não uma folha de memo) e quantos nós foram visitados.

In [9]:

```
def buscar_estado(raiz, estado):
    visitados = 0
    pilha = [raiz]
    while pilha:
        no = pilha.pop()
        visitados += 1
        if no.estado == estado and not no.do_memo:
            return no, visitados
        pilha.extend(reversed(no.filhos))    # mantém a ordem da esquerda para a direita
    return None, visitados

def contar_nos(raiz):
    return 1 + sum(contar_nos(f) for f in raiz.filhos)

_, raiz = arvore_de_subproblemas(30, [1, 3, 4])
total = contar_nos(raiz)
print(f"árvore de min_coins(30): {total} nós\n")
for estado in [30, 15, 2, 0, 99]:
    no, visitados = buscar_estado(raiz, estado)
    situacao = f"achado, com {len(no.filhos)} filho(s)" if no else "não existe"
    print(f" estado {estado:>2}: {situacao:<24} - {visitados:>3} de {total} nós visitados")
```

árvore de min_coins(30): 86 nós

estado 30: achado, com 3 filho(s)	–	1 de 86 nós visitados
estado 15: achado, com 3 filho(s)	–	16 de 86 nós visitados
estado 2: achado, com 1 filho(s)	–	29 de 86 nós visitados
estado 0: achado, com 0 filho(s)	–	31 de 86 nós visitados
estado 99: não existe	–	86 de 86 nós visitados

Custo da busca: no pior caso visita a árvore inteira, **O(N)**, com **N** nós. Com memoization **N** fica em $O(\text{amount} \cdot k)$, então a busca também. Na prática:

- a **raiz** é achada na primeira visita;
- quanto **menor** o estado, mais caro: a busca em profundidade desce primeiro pelo ramo da moeda 1 (`30 → 29 → 28 → ...`), e os estados pequenos estão lá no fundo desse ramo;
- um estado que **não existe** (como o 99) obriga a visitar **todos** os nós — é o pior caso.

Se eu precisasse buscar estados muitas vezes, não percorreria a árvore: guardaria cada nó na própria hashtable, indexado pelo estado, e a busca cairia para **O(1)** em média. A árvore serve para **visualizar e validar** a estrutura recursiva; para consultar, a hashtable é a estrutura certa.

6. O limite da versão recursiva, e a versão com tabela

A memoizada ainda é recursiva: para `amount = 5000` com moeda 1, a primeira descida faz 5.000 chamadas empilhadas e estoura o limite do Python. A saída é a versão **bottom-up**, como a do LCS e da linha de montagem da aula: preencho uma tabela do valor 0 até `amount`, sem recursão nenhuma.

In [10]:

```
def min_coins_tabela(amount, coins):
    validar(amount, coins)
    tabela = [0] + [INFINITO] * amount      # tabela[a] = menor número de moedas para formar a
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and tabela[a - c] + 1 < tabela[a]:
                tabela[a] = tabela[a - c] + 1
    return -1 if tabela[amount] == INFINITO else tabela[amount]

# as três versões concordam
for amount in range(0, 40):
    assert min_coins_tabela(amount, moedas) == min_coins_memoizado(amount, moedas)[0]
for amount in range(0, 20):
    assert min_coins_tabela(amount, moedas) == min_coins(amount, moedas)[0]

grande = 5000
print(f"limite de recursão: {sys.getrecursionlimit()}")
try:
    min_coins_memoizado(grande, moedas)
except RecursionError:
    print(f"memoizada com amount = {grande}: RecursionError")
print(f"tabela com amount = {grande}: {min_coins_tabela(grande, moedas)} moedas (0(amount · k), sem recursão)")
```

```
limite de recursão: 1000
memoizada com amount = 5000: RecursionError
tabela com amount = 5000: 1250 moedas (0(amount · k), sem recursão)
```

7. Casos de borda

In [11]:

```
assert min_coins(0, [5])[0] == 0          # nada a formar
assert min_coins(3, [5])[0] == -1         # moeda maior que o valor
assert min_coins_memoizado(7, [2, 4])[0] == -1 # impossível
assert min_coins_tabela(7, [2, 4]) == -1
assert min_coins(10, [10])[0] == 1        # moeda única que cabe exato
assert min_coins_memoizado(11, [5, 1, 5])[0] == 3 # moeda repetida na lista não atrapalha

for amount, coins in [(-1, [1]), (5, []), (5, [0, 1]), (5, [-2])]:
    try:
        min_coins(amount, coins)
        raise AssertionError(f"deveria recusar amount={amount}, coins={coins}")
    except ValueError as erro:
        print(f"min_coins({amount}, {coins}) → {erro}")

print("\ntodos os casos de borda passaram")
```

```
min_coins(-1, [1]) → amount não pode ser negativo: -1
min_coins(5, []) → as moedas precisam ser positivas: []
min_coins(5, [0, 1]) → as moedas precisam ser positivas: [0, 1]
min_coins(5, [-2]) → as moedas precisam ser positivas: [-2]
```

todos os casos de borda passaram

Resumo

versão	tempo	memória extra	limite
recursiva pura	exponencial — recalcula os mesmos estados	$O(\text{amount})$ na pilha de chamadas	fica inviável já com <code>amount</code> perto de 30
memoizada (hashtable)	$O(\text{amount} \cdot k)$ — cada estado uma vez	$O(\text{amount})$ no memo + pilha	<code>RecursionError</code> com <code>amount</code> acima do limite de recursão
tabela (bottom-up)	$O(\text{amount} \cdot k)$	$O(\text{amount})$	nenhum além da memória

A memoization troca **memória por tempo**: guardando `amount + 1` resultados, o custo cai de exponencial para linear em `amount`.

Exercício 9 — QuickSort e QuickSelect com contagem de operações e variante em lista encadeada

Henrique Goldstein Maluhy Mendes Amaral · DR1_AT — Algoritmos e Estruturas de Dados · Infnet

Para rodar: **Ambiente de execução** → **Executar tudo**. O notebook é independente, não precisa de nenhum arquivo externo.

Roteiro: (1) `quicksort(arr, short)` instrumentado, com insertion sort nos trechos curtos · (2) 4 padrões × 3 valores de `short` · (3) o pior caso e a escolha do pivô · (4) `quickselect(arr, goal_index)` × ordenar tudo · (5) variante em `SinglyLinkedList` com partição estável · (6) casos de borda

In [1]:

```
import math
import random
import time

import matplotlib.pyplot as plt
import pandas as pd

random.seed(42)

class Contador:
    def __init__(self):
        self.comparacoes = 0
        self.copias = 0
        self.chamadas = 0
```

1. O QuickSort

Partição de três vias. No quicksort da aula, a lista é dividida em `menores`, `iguais` e `maiores` que o pivô, criando três listas novas. Aqui faço **a mesma divisão dentro do próprio vetor**, sem criar listas: depois de particionar, o trecho fica assim:

```
[ menores que o pivô | iguais ao pivô | maiores que o pivô ]
```

Os iguais já estão no lugar certo e **não entram na recursão** — isso evita o pior caso quando há muitos valores repetidos.

Pivô: mediana de três elementos sorteados. Sorteio três posições do trecho e uso como pivô o valor do meio entre os três. Com o pivô ingênuo (sempre o último), um vetor já ordenado vira o pior caso (seção 3).

Minha primeira versão usava a mediana de posições **fixas** — o primeiro, o do meio e o último —, que é a forma mais comum. A medição mostrou que no vetor ordenado `comparacoes / (n · log2 n)` **crescia** com `n` (de 4,7 para 12,9 com `n` crescendo 16 vezes), ou seja, não era $O(n \log n)$. Rastreando à mão, achei a causa: a partição de três vias pega um trecho ordenado como `[5, 6, 7, 8, 9]` e devolve `[6, 7, 8, 9, 5]` — ordenado, mas com o **menor no fim**. Nesse formato, a mediana de (primeiro, meio, último) escolhe justamente o segundo menor, e a divisão fica péssima em todos os níveis. **Sorteando as posições**, nenhum formato de entrada consegue enganar a escolha do pivô.

Trechos curtos vão para o insertion sort: se o trecho tem `short` elementos ou menos, ordeno com insertion sort, que é muito bom em listas pequenas e evita uma cascata de chamadas recursivas para ordenar 3 ou 4 elementos.

Recursão só na parte menor: depois de particionar, chamo a recursão para o lado **menor** e continuo num laço para o lado maior. Assim a profundidade da recursão nunca passa de $\log_2 n$.

Contagem: **comparação** = cada comparação de chaves; **cópia** = cada escrita no vetor, com **troca = 3 cópias**.

In [2]:

```
def trocar(lista, i, j, cont):
    if i != j:
        lista[i], lista[j] = lista[j], lista[i]
        cont.copias += 3

def insertion_sort_trecho(lista, inicio, fim, cont):
    for i in range(inicio + 1, fim + 1):
        chave = lista[i]
        cont.copias += 1
        j = i - 1
        while j >= inicio:
            cont.comparacoes += 1
            if lista[j] <= chave:
                break
            lista[j + 1] = lista[j]
            cont.copias += 1
            j -= 1
        lista[j + 1] = chave
        cont.copias += 1
```

```

def escolher_pivo(lista, inicio, fim, estrategia, cont):
    """Devolve o VALOR do pivô."""
    if estrategia == "ultimo":
        return lista[fim]
    # três posições sorteadas; o pivô é o valor do meio entre os três
    a, b, c = (lista[random.randint(inicio, fim)] for _ in range(3))
    cont.comparacoes += 3 # conto as 3 comparações da mediana
    if a <= b <= c or c <= b <= a:
        return b
    if b <= a <= c or c <= a <= b:
        return a
    return c

def particionar(lista, inicio, fim, estrategia, cont):
    """Três vias. Devolve (lt, gt): o trecho lista[lt..gt] é todo igual ao pivô."""
    pivo = escolher_pivo(lista, inicio, fim, estrategia, cont)
    # lt: fim da região dos menores · i: próximo a examinar · gt: começo da região dos maiores
    lt, i, gt = inicio, inicio, fim
    while i <= gt:
        cont.comparacoes += 1
        if lista[i] < pivo:
            trocar(lista, lt, i, cont) # vai para a região dos menores
            lt += 1
            i += 1
        else:
            cont.comparacoes += 1
            if lista[i] > pivo:
                trocar(lista, i, gt, cont) # vai para a região dos maiores
                gt -= 1
            else:
                i += 1 # igual ao pivô: fica no meio
    return lt, gt

def _quicksort(lista, inicio, fim, short, estrategia, cont):
    cont.chamadas += 1
    while inicio < fim:
        if fim - inicio + 1 <= short:
            insertion_sort_trecho(lista, inicio, fim, cont)
            return
        lt, gt = particionar(lista, inicio, fim, estrategia, cont)
        # recursão no lado menor, Laço no lado maior: a pilha fica em O(log n)
        if lt - inicio < fim - gt:
            _quicksort(lista, inicio, lt - 1, short, estrategia, cont)
            inicio = gt + 1
        else:
            _quicksort(lista, gt + 1, fim, short, estrategia, cont)
            fim = lt - 1

def quicksort(arr, short, pivo="mediana3"):
    """Ordena uma cópia de arr. Devolve (lista_ordenada, comparacoes, copias, chamadas)."""
    if short < 0:
        raise ValueError(f"short não pode ser negativo: {short}")
    lista = arr.copy()
    cont = Contador()
    _quicksort(lista, 0, len(lista) - 1, short, pivo, cont)
    return lista, cont.comparacoes, cont.copias, cont.chamadas

for _ in range(300):
    arr = [random.randint(0, 30) for _ in range(random.randint(0, 150))]
    for short in (0, 8, 32):
        assert quicksort(arr, short)[0] == sorted(arr)
print("quicksort conferido contra sorted() em 300 listas sorteadas, com repetidos, para short = 0, 8 e 32")

```

quicksort conferido contra sorted() em 300 listas sorteadas, com repetidos, para short = 0, 8 e 32

2. Quatro padrões × três valores de short

In [3]:

```
def gerar(n, padrao):
    if padrao == "ordenado":
        return list(range(n))
    if padrao == "reverso":
        return list(range(n - 1, -1, -1))
    if padrao == "aleatorio":
        lista = list(range(n))
        random.shuffle(lista)
        return lista
    if padrao == "quase ordenado":
        lista = list(range(n))
        for _ in range(n // 20):
            i = random.randrange(n)
            j = min(n - 1, i + random.randint(1, 10))
            lista[i], lista[j] = lista[j], lista[i]
        return lista
    raise ValueError(padrao)

PADROES = ["ordenado", "reverso", "quase ordenado", "aleatorio"]
linhas = []
for n in [1_000, 4_000, 16_000]:
    for padrao in PADROES:
        entrada = gerar(n, padrao)
        for short in (0, 8, 32):
            inicio = time.perf_counter()
            _, comparacoes, copias, chamadas = quicksort(entrada, short)
            tempo = time.perf_counter() - inicio
            linhas.append({
                "n": n, "padrao": padrao, "short": short,
                "comparacoes": comparacoes, "copias": copias, "chamadas": chamadas,
                "comparacoes / (n·log2 n)": comparacoes / (n * math.log2(n)),
                "tempo_ms": tempo * 1000,
            })

qs = pd.DataFrame(linhas)
qs[qs["n"] == 16_000]
```

Out[3]:

	n	padrao	short	comparacoes	copias	chamadas	comparacoes / (n·log2 n)	tempo_ms
24	16000	ordenado	0	384160	348042	10093	1.719202	44.3384
25	16000	ordenado	8	363183	416458	2834	1.625325	38.2529
26	16000	ordenado	32	350245	373192	822	1.567425	29.7105
27	16000	reverso	0	392561	384144	10068	1.756798	46.1628
28	16000	reverso	8	349472	391650	2814	1.563965	33.8916
29	16000	reverso	32	350752	396250	812	1.569693	32.4996
30	16000	quase ordenado	0	390545	381870	10087	1.747776	46.9036
31	16000	quase ordenado	8	340139	348610	2831	1.522198	32.2059
32	16000	quase ordenado	32	368427	392508	823	1.648793	32.0882
33	16000	aleatorio	0	387594	493293	10157	1.734570	47.1108
34	16000	aleatorio	8	358061	506301	2786	1.602403	37.5441
35	16000	aleatorio	32	359103	490453	820	1.607066	36.2486

O que muda com o `short` :

- **As chamadas recursivas despencam:** cerca de 10 mil com `short = 0` , 2,8 mil com `short = 8` e 800 com `short = 32` . Sem o corte, o quicksort vai até trechos de 1 elemento; com ele, cada trecho pequeno é resolvido de uma vez pelo insertion sort.
- **As comparações caem um pouco** (em torno de 5% a 10%). Num trecho de poucos elementos, o insertion sort compara menos do que a partição de três vias, que ainda gasta 3 comparações só para escolher o pivô.
- **As cópias variam pouco e sem direção fixa** — às vezes sobem, às vezes descem, dependendo do padrão.
- **O tempo cai** junto com as chamadas, porque chamada de função é cara em Python.

Em Big O nada muda: continua $O(n \log n)$. O `short` é um ganho de **constante** — menos chamadas e um pouco menos de comparações —, não uma mudança de classe.

O que o padrão muda: com a mediana de três, os **quatro padrões ficam em $O(n \log n)$** — a coluna `comparacoes / (n·log2 n)` fica numa faixa estreita quando `n` cresce 16 vezes:

In [4]:

```
resumo = qs[qs["short"] == 0].pivot(index="n", columns="padrao", values="comparacoes / (n·log2 n)")
print("comparações / (n·log2 n), com short = 0:\n")
print(resumo.round(3))
```

comparações / (n·log2 n), com short = 0:

padrao	aleatorio	ordenado	quase ordenado	reverso
n				
1000	1.711	1.750	1.645	1.687
4000	1.724	1.755	1.709	1.779
16000	1.735	1.719	1.748	1.757

Os quatro padrões ficam **na mesma faixa**: como as posições do pivô são sorteadas, a ordem em que os dados chegam não muda o custo. É exatamente o que a mediana de posições fixas não conseguia garantir.

3. O pior caso: quando a escolha do pivô falha

Troco a mediana de três pelo pivô ingênuo (**sempre o último** elemento) e rodo num vetor **já ordenado**. O último é sempre o maior, então cada partição separa só um elemento:

In [5]:

```
linhas = []
for n in [500, 1000, 2000]:
    entrada = gerar(n, "ordenado")
    _, c_ingenuo, _ = quicksort(entrada, 0, pivô="ultimo")
    _, c_mediana, _ = quicksort(entrada, 0, pivô="mediana3")
    linhas.append({
        "n": n,
        "pivo_ultimo": c_ingenuo, "pivo_ultimo / n²": c_ingenuo / n**2,
        "mediana_de_tres": c_mediana, "mediana / (n·log2 n)": c_mediana / (n * math.log2(n)),
    })
pd.DataFrame(linhas)
```

Out[5]:

	n	pivo_ultimo	pivo_ultimo / n²	mediana_de_tres	mediana / (n·log2 n)
0	500	125748	0.502992	7569	1.688419
1	1000	501498	0.501498	17709	1.776980
2	2000	2002998	0.500749	38851	1.771465

Com o pivô no último, `comparacoes / n²` fica **estável** quando `n` dobra: é **O(n²)**, como se fosse um selection sort. Com a mediana de três, a mesma entrada fica em O(n log n). A escolha do pivô é o que separa o caso médio do pior caso.

E a recursão na parte menor garante que, mesmo nesse pior caso, **a pilha não estoura**: o lado grande é tratado no laço, não em recursão. O tempo vira O(n²), mas a memória continua O(log n).

4. QuickSelect: o k-ésimo sem ordenar tudo

`quickselect(arr, goal_index)` devolve o elemento que estaria na posição `goal_index` se o vetor estivesse ordenado. Ele usa a mesma partição, mas depois de cada partição **segue só para o lado onde a posição procurada está** — o outro lado é descartado.

In [6]:

```
def quickselect(arr, goal_index):
    """Devolve (elemento, comparacoes, copias)."""
    if not 0 <= goal_index < len(arr):
        raise IndexError(f"goal_index {goal_index} fora do vetor de tamanho {len(arr)}")
    lista = arr.copy()
    cont = Contador()
    inicio, fim = 0, len(lista) - 1
    while inicio < fim:
        lt, gt = particionar(lista, inicio, fim, "mediana3", cont)
        if goal_index < lt:
            fim = lt - 1          # está entre os menores
        elif goal_index > gt:
            inicio = gt + 1       # está entre os maiores
        else:
            break                # caiu no bloco dos iguais ao pivô: achou
    return lista[goal_index], cont.comparacoes, cont.copias

# confere contra a posição no vetor ordenado, com repetidos
for _ in range(300):
    arr = [random.randint(0, 40) for _ in range(random.randint(1, 120))]
    k = random.randrange(len(arr))
    assert quickselect(arr, k)[0] == sorted(arr)[k]
print("quickselect conferido em 300 casos sorteados")

linhas = []
for n in [1_000, 10_000, 100_000]:
    entrada = gerar(n, "aleatorio")
    _, c_sort, cp_sort, _ = quicksort(entrada, 0)
    for nome, k in [("minimo", 0), ("mediana", n // 2), ("percentil 90", int(n * 0.9))]:
        elemento, c_sel, cp_sel = quickselect(entrada, k)
        assert elemento == k          # os valores são 0..n-1
        linhas.append({
            "n": n, "posicao": nome,
            "quickselect_comparacoes": c_sel, "quickselect / n": c_sel / n,
            "ordenar_tudo_comparacoes": c_sort,
            "ganho (ordenar / selecionar)": c_sort / c_sel,
            "quickselect_copias": cp_sel, "ordenar_tudo_copias": cp_sort,
        })
pd.DataFrame(linhas)
```

quickselect conferido em 300 casos sorteados

Out[6]:

	n	posicao	quickselect_comparacoes	quickselect / n	ordenar_tudo_comparacoes	ganho (ordenar / selecionar)	quickselect_copias	ordenar_tudo_copias
0	1000	mínimo	3667	3.66700	17497	4.771475	5712	21882
1	1000	mediana	3223	3.22300	17497	5.428793	4446	21882
2	1000	percentil 90	4121	4.12100	17497	4.245814	5817	21882
3	10000	mínimo	30937	3.09370	232409	7.512332	46629	294417
4	10000	mediana	44138	4.41380	232409	5.265508	66267	294417
5	10000	percentil 90	24462	2.44620	232409	9.500818	36969	294417
6	100000	mínimo	317500	3.17500	2952180	9.298205	508827	3806982
7	100000	mediana	341375	3.41375	2952180	8.647909	362157	3806982
8	100000	percentil 90	220342	2.20342	2952180	13.398172	383334	3806982

- **QuickSelect:** `quickselect / n` fica numa faixa estável — ele é **O(n) em média**. Cada partição descarta um lado, então o trabalho é $n + n/2 + n/4 + \dots \approx 2n$ partições de elemento (cada uma com uma ou duas comparações).
- **Ordenar tudo:** $O(n \log n)$.
- O **ganho cresce com** `n` (é o fator $\log n$), e vale tanto para comparações quanto para cópias.

O pior caso do QuickSelect também é $O(n^2)$, com a mesma causa do quicksort (pivô ruim); a mediana de três torna isso muito improvável.

5. Variante em `SinglyLinkedList`

Numa lista encadeada não existe `lista[i]`: só dá para andar de nó em nó a partir do começo. A partição vira outra coisa: em vez de trocar valores de lugar, eu **desligo cada nó e religo** no fim de uma de três listas novas — `menores`, `iguais` e `maiores`. É literalmente o quicksort da aula, só que movendo nós em vez de criar listas com os valores.

Estabilidade: cada nó entra no **fim** da sua lista, na ordem em que foi encontrado. Então elementos com a mesma chave mantêm a ordem original entre si — a partição é **estável**. (A partição em vetor não é: as trocas embaralham os iguais.)

In [7]:

```

class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class SinglyLinkedList:
    def __init__(self, valores=()):
        self.head = None
        self.tail = None
        for v in valores:
            self.append_node(Node(v))

    def append_node(self, no):
        """Anexa um nó existente no fim. O(1) graças ao ponteiro tail."""
        no.next = None
        if self.head is None:
            self.head = no
        else:
            self.tail.next = no
            self.tail = no

    def to_list(self):
        valores, atual = [], self.head
        while atual is not None:
            valores.append(atual.value)
            atual = atual.next
        return valores

def particionar_lista(lista, pivo, chave, cont):
    """Separa os nós em três listas novas, mantendo a ordem relativa (estável)."""
    menores, iguais, maiores = SinglyLinkedList(), SinglyLinkedList(), SinglyLinkedList()
    atual = lista.head
    while atual is not None:
        proximo = atual.next          # guardo antes de religar o nó
        cont.comparacoes += 1
        if chave(atual.value) < pivo:
            menores.append_node(atual)
        else:
            cont.comparacoes += 1
            if chave(atual.value) > pivo:
                maiores.append_node(atual)
            else:
                iguais.append_node(atual)
        atual = proximo              # nenhum valor copiado: o nó só mudou de lista
    return menores, iguais, maiores

def concatenar(*listas):
    resultado = SinglyLinkedList()
    for l in listas:
        if l.head is None:
            continue
        if resultado.head is None:
            resultado.head = l.head
        else:
            resultado.tail.next = l.head      # junta em O(1), sem percorrer nada
            resultado.tail = l.tail
    return resultado

def quicksort_lista(lista, cont, chave=lambda v: v):
    cont.chamadas += 1
    if lista.head is None or lista.head.next is None:
        return lista
    pivo = chave(lista.head.value)          # o primeiro nó: é o único de acesso O(1)
    menores, iguais, maiores = particionar_lista(lista, pivo, chave, cont)

```

```
return concatenar(quicksort_lista(menores, cont, chave), iguais, quicksort_lista(maiores, cont, chave))

# estabilidade: pares (chave, ordem de chegada); os de mesma chave precisam manter a ordem
pares = [(3, "a"), (1, "b"), (3, "c"), (2, "d"), (1, "e"), (3, "f")]
cont = Contador()
m, i, M = particionar_lista(SinglyLinkedList(pares), 2, lambda p: p[0], cont)
print("pivô 2 → menores:", m.to_list())
print("           iguais: ", i.to_list())
print("           maiores:", M.to_list())
ordenado = quicksort_lista(SinglyLinkedList(pares), Contador(), chave=lambda p: p[0]).to_list()
print("\nordenado pela chave:", ordenado)
assert ordenado == sorted(pares, key=lambda p: p[0]) # o sorted do Python é estável: mesmo resultado

for _ in range(200):
    arr = [random.randint(0, 20) for _ in range(random.randint(0, 100))]
    assert quicksort_lista(SinglyLinkedList(arr), Contador()).to_list() == sorted(arr)
print("quicksort em lista conferido em 200 listas sorteadas")
```

pivô 2 → menores: [(1, 'b'), (1, 'e')]
 iguais: [(2, 'd')]
 maiores: [(3, 'a'), (3, 'c'), (3, 'f')]

ordenado pela chave: [(1, 'b'), (1, 'e'), (2, 'd'), (3, 'a'), (3, 'c'), (3, 'f')]
quicksort em lista conferido em 200 listas sorteadas

Por que o custo muda sem acesso por índice

	vetor (in-place)	lista encadeada
partição	O(n), trocando valores	O(n), religando nós
movimentação de dados	trocas = 3 cópias cada	nenhuma cópia de valor: só ponteiros
memória extra	O(log n) (só a recursão)	O(1) além da recursão: reaproveita os nós
escolher bom pivô (meio, mediana, sorteado)	O(1): lista[i]	O(n): precisa andar até a posição
juntar as partes	já estão no lugar	O(1) com ponteiro de fim
estável	não	sim

O ponto que pesa: sem acesso direto, **escolher um pivô bom custa O(n)**, então o natural é usar o **primeiro nó** — e aí uma lista já ordenada cai no pior caso O(n²). Medindo:

In [8]:

```
linhas = []
for n in [150, 300, 600]: # mais do que isso, a recursão na lista ordenada passa do limite do Python
    for padrao in ["aleatorio", "ordenado"]:
        cont = Contador()
        resultado = quicksort_lista(SinglyLinkedList(gerar(n, padrao)), cont)
        assert resultado.to_list() == list(range(n))
        linhas.append({
            "n": n, "padrao": padrao, "comparacoes": cont.comparacoes,
```

```

        "comparacoes / (n*log2 n)": cont.comparacoes / (n * math.log2(n)),
        "comparacoes / n^2": cont.comparacoes / n**2,
    })
pd.DataFrame(linhas)

```

Out[8]:

	n	padrao	comparacoes	comparacoes / (n*log2 n)	comparacoes / n²
0	150	aleatorio	1795	1.655411	0.079778
1	150	ordenado	22648	20.886769	1.006578
2	300	aleatorio	3915	1.585890	0.043500
3	300	ordenado	90298	36.577952	1.003311
4	600	aleatorio	9611	1.735686	0.026697
5	600	ordenado	360598	65.121733	1.001661

- **Aleatória:** `comparacoes / (n*log2 n)` estável → **$O(n \log n)$** , igual ao vetor.
- **Já ordenada:** `comparacoes / n²` estável → **$O(n^2)$** . O primeiro nó é sempre o menor, e cada partição tira um elemento só. No vetor, a mediana de três evitou isso por custar $O(1)$; na lista, a mesma defesa custaria $O(n)$ por partição.
- A recursão aqui não tem o truque do "lado menor", porque as três listas saem prontas da partição. Por isso a lista ordenada também usa **profundidade n** — é o motivo de eu ter parado em $n = 600$.

6. Casos de borda

In [9]:

```

for short in (0, 8, 32):
    assert quicksort([], short)[0] == []
    assert quicksort([5], short)[0] == [5]
    assert quicksort([2, 2, 2, 2], short)[0] == [2, 2, 2, 2]
    assert quicksort([3, -1, 0, -1], short)[0] == [-1, -1, 0, 3]

# vetor todo igual: a partição de três vias resolve numa passada só
_, comparacoes, _, chamadas = quicksort([7] * 10_000, 0)
print(f"10.000 elementos iguais: {comparacoes} comparações, {chamadas} chamada(s) — linear")

assert quickselect([9], 0)[0] == 9
assert quickselect([4, 4, 4], 1)[0] == 4
for invalido in (-1, 3):
    try:
        quickselect([1, 2, 3], invalido)
    except IndexError as erro:
        print(f"quickselect(goal_index={invalido}) → {erro}")

try:
    quicksort([1, 2], -1)
except ValueError as erro:
    print(f"quicksort(short=-1) → {erro}")

```

```
assert quicksort_lista(SinglyLinkedList(), Contador()).to_list() == []
assert quicksort_lista(SinglyLinkedList([1]), Contador()).to_list() == [1]
print("\ntodos os casos de borda passaram")
```

10.000 elementos iguais: 20003 comparações, 2 chamada(s) — linear
quickselect(goal_index=-1) → goal_index -1 fora do vetor de tamanho 3
quickselect(goal_index=3) → goal_index 3 fora do vetor de tamanho 3
quicksort(short=-1) → short não pode ser negativo: -1

todos os casos de borda passaram

Exercício 10 — Lista encadeada com operações completas e análise de eficiência

Henrique Goldstein Maluhy Mendes Amaral · DR1_AT — Algoritmos e Estruturas de Dados · Infnet

Para rodar: **Ambiente de execução** → **Executar tudo**. O notebook é independente, não precisa de nenhum arquivo externo.

Roteiro: (1) `SinglyLinkedList` com todas as operações pedidas · (2) operações por posição com validação · (3) teste longo contra um gabarito · (4) custo de cada operação e experimento da busca linear · (5) a decisão do ponteiro `tail`, medida · (6) casos de borda

In [1]:

```
import random
import time

import matplotlib.pyplot as plt
import pandas as pd

random.seed(42)
```

1. A implementação

Decisões:

- **Guardo três coisas:** `head` (primeiro nó), `tail` (último nó) e `size` (quantidade). O `size` é atualizado a cada operação, então `len()` é $O(1)$ — não preciso contar os nós toda vez.
- `search` **devolve a posição ou -1**, como as buscas do exercício 1. Ele também guarda quantas comparações fez na última busca, para as medições.
- `delete(value)` **remove a primeira ocorrência** e, se o valor não existir, levanta `ValueError` com uma mensagem clara — o mesmo comportamento do `remove` das listas do Python.

- **O cuidado principal é o `tail`** : ao remover o último nó, o `tail` precisa voltar para o nó anterior. Se esquecer disso, o próximo `insert_last` liga o nó novo num nó que já saiu da lista, e ele "desaparece".

In [2]:

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class SinglyLinkedList:
    def __init__(self, valores=()):
        self.head = None
        self.tail = None
        self.size = 0
        self.ultimas_comparacoes = 0
        for v in valores:
            self.insert_last(v)

    def __len__(self):
        return self.size

    def __str__(self):
        partes = []
        atual = self.head
        while atual is not None:
            partes.append(repr(atual.value))
            atual = atual.next
        return "[" + " -> ".join(partes) + "]"

    def to_list(self):
        valores, atual = [], self.head
        while atual is not None:
            valores.append(atual.value)
            atual = atual.next
        return valores

    # --- inserção -----
    def insert_first(self, value):
        novo = Node(value)
        novo.next = self.head
        self.head = novo
        if self.tail is None:      # Lista estava vazia: o novo também é o último
            self.tail = novo
        self.size += 1

    def insert_last(self, value):
        novo = Node(value)
        if self.head is None:
            self.head = novo
        else:
            self.tail.next = novo      # acesso direto ao fim: não percorro nada
            self.tail = novo
        self.size += 1

    # --- busca e remoção por valor -----
    def search(self, value):
        self.ultimas_comparacoes = 0
        posicao, atual = 0, self.head
        while atual is not None:
            self.ultimas_comparacoes += 1
            if atual.value == value:
                return posicao
            atual = atual.next
```

```

        posicao += 1
    return -1

def delete(self, value):
    anterior, atual = None, self.head
    while atual is not None and atual.value != value:
        anterior, atual = atual, atual.next
    if atual is None:
        raise ValueError(f"delete({value!r}): o valor não está na lista")
    self._desligar(anterior, atual)

# --- operações por posição -----
def _validar_indice(self, index, maximo, operacao):
    if isinstance(index, bool) or not isinstance(index, int):
        raise TypeError(f"{operacao}: o índice precisa ser inteiro, recebi {index!r}")
    if index < 0 or index > maximo:
        raise IndexError(
            f"{operacao}({index}): índice fora da faixa – a lista tem {self.size} "
            f"elemento(s), então o índice vai de 0 a {maximo}")

def insert_at(self, index, value):
    # inserir na posição `size` é o mesmo que inserir no fim, por isso o máximo é size
    self._validar_indice(index, self.size, "insert_at")
    if index == 0:
        self.insert_first(value)
    elif index == self.size:
        self.insert_last(value)
    else:
        anterior = self._no_na_posicao(index - 1)
        novo = Node(value)
        novo.next = anterior.next
        anterior.next = novo
        self.size += 1

def delete_at(self, index):
    if self.size == 0:
        raise IndexError(f"delete_at({index}): a lista está vazia")
    # remover exige que exista um elemento ali, por isso o máximo é size - 1
    self._validar_indice(index, self.size - 1, "delete_at")
    anterior = None if index == 0 else self._no_na_posicao(index - 1)
    atual = self.head if anterior is None else anterior.next
    self._desligar(anterior, atual)
    return atual.value

# --- auxiliares -----
def _no_na_posicao(self, index):
    atual = self.head
    for _ in range(index):
        atual = atual.next
    return atual

def _desligar(self, anterior, atual):
    if anterior is None:
        self.head = atual.next
    else:
        anterior.next = atual.next
    if atual is self.tail:
        self.tail = anterior
    self.size -= 1

def verificar(self):
    """Confere as invariantes: size bate com os nós, e o tail é mesmo o último."""
    contados, ultimo, atual = 0, None, self.head
    while atual is not None:
        contados += 1
        ultimo, atual = atual, atual.next
    assert contados <= self.size, "tem mais nós do que o size diz (ciclo?)"
    assert contados == self.size, f"size = {self.size}, mas há {contados} nós"

```

```
assert self.tail is ultimo, "o tail não aponta para o último nó"
assert self.tail is None or self.tail.next is None
```

2. Funcionando

In [3]:

```
lista = SinglyLinkedList()
lista.insert_last(20)
lista.insert_last(30)
lista.insert_first(10)
print("inserções:      ", lista, " tamanho", len(lista))

lista.insert_at(1, 15)
lista.insert_at(len(lista), 40)      # na posição size: vai para o fim
print("insert_at:      ", lista)

print("search(30):      ", lista.search(30), f"({lista.ultimas_comparacoes} comparações)")
print("search(99):      ", lista.search(99), f"({lista.ultimas_comparacoes} comparações)")

lista.delete(15)
print("delete(15):      ", lista)
print("delete_at(0) →   ", lista.delete_at(0), " sobra", lista)

# o caso do tail: remove o último e insiro de novo no fim
lista.delete(40)
lista.insert_last(50)
print("tira o 40, põe 50:", lista)
assert lista.to_list() == [20, 30, 50] and lista.tail.value == 50
lista.verificar()
```

```
inserções:      [10 -> 20 -> 30] tamanho 3
insert_at:      [10 -> 15 -> 20 -> 30 -> 40]
search(30):      3 (4 comparações)
search(99):      -1 (5 comparações)
delete(15):      [10 -> 20 -> 30 -> 40]
delete_at(0) →   10 sobra [20 -> 30 -> 40]
tira o 40, põe 50: [20 -> 30 -> 50]
```

Mensagens de erro

In [4]:

```
lista = SinglyLinkedList([1, 2, 3])
tentativas = [
    ("insert_at(-1, 9)", lambda: lista.insert_at(-1, 9)),
    ("insert_at(4, 9)", lambda: lista.insert_at(4, 9)),
    ("delete_at(3)", lambda: lista.delete_at(3)),
    ("delete_at('0')", lambda: lista.delete_at("0")),
    ("delete(99)", lambda: lista.delete(99)),
    ("delete_at(0) em lista vazia", lambda: SinglyLinkedList().delete_at(0)),
]
for descricao, chamada in tentativas:
    try:
        chamada()
        print(f"{descricao}: ERRO - deveria ter falhado")
```

```

except (IndexError, ValueError, TypeError) as erro:
    print(f"{descricao:<30} → {type(erro).__name__}: {erro}")

assert lista.to_list() == [1, 2, 3]      # nenhuma tentativa inválida mexeu na lista
lista.verificar()

```

```

insert_at(-1, 9)      → IndexError: insert_at(-1): índice fora da faixa – a lista tem 3 elemento(s), então o índice vai de 0 a 3
insert_at(4, 9)       → IndexError: insert_at(4): índice fora da faixa – a lista tem 3 elemento(s), então o índice vai de 0 a 3
delete_at(3)          → IndexError: delete_at(3): índice fora da faixa – a lista tem 3 elemento(s), então o índice vai de 0 a 2
delete_at('0')        → TypeError: delete_at: o índice precisa ser inteiro, recebi '0'
delete(99)            → ValueError: delete(99): o valor não está na lista
delete_at(0) em lista vazia → IndexError: delete_at(0): a lista está vazia

```

3. Teste longo contra um gabarito

1.000 operações sorteadas, e uma lista do Python fazendo as mesmas operações como gabarito. A cada passo confiro o conteúdo e as invariantes.

In [5]:

```

lista, gabarito = SinglyLinkedList(), []
for passo in range(1000):
    operacao = random.choice(["insert_first", "insert_last", "insert_at", "delete", "delete_at", "search"])
    if operacao == "insert_first":
        lista.insert_first(passo)
        gabarito.insert(0, passo)
    elif operacao == "insert_last":
        lista.insert_last(passo)
        gabarito.append(passo)
    elif operacao == "insert_at":
        i = random.randint(0, len(gabarito))
        lista.insert_at(i, passo)
        gabarito.insert(i, passo)
    elif operacao == "delete" and gabarito:
        valor = random.choice(gabarito)
        lista.delete(valor)
        gabarito.remove(valor)
    elif operacao == "delete_at" and gabarito:
        i = random.randrange(len(gabarito))
        assert lista.delete_at(i) == gabarito.pop(i)
    elif operacao == "search":
        valor = random.randint(0, passo + 1)
        esperado = gabarito.index(valor) if valor in gabarito else -1
        assert lista.search(valor) == esperado
    assert lista.to_list() == gabarito
    lista.verificar()

print(f"1.000 operações: a lista bateu com o gabarito em todos os passos (terminou com {len(lista)} elementos)")

```

1.000 operações: a lista bateu com o gabarito em todos os passos (terminou com 161 elementos)

4. Custo de cada operação

operação	custo	por quê
<code>insert_first</code>	$O(1)$	só religa o <code>head</code>
<code>insert_last</code>	$O(1)$	o <code>tail</code> dá acesso direto ao fim
<code>search(value)</code>	$O(n)$	sem acesso por índice, só dá para andar nó a nó
<code>delete(value)</code>	$O(n)$	precisa achar o valor e o nó anterior a ele
<code>insert_at(i, v)</code>	$O(i)$	anda até a posição <code>i - 1</code> ($O(1)$ nas pontas)
<code>delete_at(i)</code>	$O(i)$	anda até o nó anterior
<code>__len__</code>	$O(1)$	o <code>size</code> é mantido a cada operação
<code>__str__</code>	$O(n)$	passa por todos os nós

`delete` continua $O(n)$ **mesmo com o** `tail`: para tirar o último nó é preciso mudar o `next` do **penúltimo**, e numa lista simples não há como voltar do último para o penúltimo. É exatamente o que a lista duplamente encadeada (exercício 11) resolve.

Experimento: o custo da busca linear

Melhor caso (valor na primeira posição), caso médio (200 valores presentes sorteados) e pior caso (valor ausente):

In [6]:

```
linhas = []
for n in [1_000, 2_000, 4_000, 8_000, 16_000]:
    valores = list(range(n))
    random.shuffle(valores)
    lista = SinglyLinkedList(valores)

    lista.search(valores[0])
    melhor = lista.ultimas_comparacoes

    soma = 0
    for alvo in random.sample(valores, 200):
        lista.search(alvo)
        soma += lista.ultimas_comparacoes
    medio = soma / 200

    lista.search(-1)
    pior = lista.ultimas_comparacoes

    linhas.append({"n": n, "melhor": melhor, "medio": round(medio), "pior": pior,
                  "medio / n": medio / n, "pior / n": pior / n})

busca = pd.DataFrame(linhas)
busca
```

Out[6]:

	n	melhor	medio	pior	medio / n	pior / n
0	1000	1	507	1000	0.507250	1.0
1	2000	1	1000	2000	0.500243	1.0
2	4000	1	1999	4000	0.499856	1.0
3	8000	1	4312	8000	0.538999	1.0
4	16000	1	7770	16000	0.485653	1.0

- **Melhor caso:** 1 comparação em qualquer tamanho — $O(1)$.
- **Pior caso:** `pior / n` é exatamente **1,0** — compara com todos, $O(n)$.
- **Caso médio:** `medio / n` fica em torno de **0,5** — em média o valor está no meio, $O(n)$.

Quando `n` dobra, o pior caso e o caso médio dobram junto: é o crescimento linear.

5. A decisão do ponteiro `tail`, medida

Para medir o impacto do `tail`, escrevi uma segunda versão do `insert_last` que **ignora o `tail`** e anda do `head` até o último nó — que é o que a lista faria se não tivesse esse ponteiro. Conto os **saltos** (`atual = atual.next`) e o tempo para montar uma lista de `n` elementos só com inserções no fim.

In [7]:

```
def insert_last_sem_tail(lista, value):
    """Como seria o insert_last sem o ponteiro tail. Devolve quantos saltos deu."""
    novo = Node(value)
    saltos = 0
    if lista.head is None:
        lista.head = novo
    else:
        atual = lista.head
        while atual.next is not None:      # procura o último andando nó a nó
            atual = atual.next
            saltos += 1
        atual.next = novo
    lista.tail = novo                      # mantenho o tail certo só para o verificar() continuar valendo
    lista.size += 1
    return saltos

linhas = []
for n in [500, 1_000, 2_000, 4_000]:
    com_tail = SinglyLinkedList()
    inicio = time.perf_counter()
    for v in range(n):
        com_tail.insert_last(v)
```

```

tempo_com = time.perf_counter() - inicio

sem_tail = SinglyLinkedList()
saltos = 0
inicio = time.perf_counter()
for v in range(n):
    saltos += insert_last_sem_tail(sem_tail, v)
tempo_sem = time.perf_counter() - inicio

assert com_tail.to_list() == sem_tail.to_list() == list(range(n))
linhas.append({
    "n": n,
    "saltos_com_tail": 0, "saltos_sem_tail": saltos,
    "teoria (n-1)(n-2)/2": (n - 1) * (n - 2) // 2,
    "saltos_sem_tail / n²": saltos / n**2,
    "ms_com_tail": tempo_com * 1000, "ms_sem_tail": tempo_sem * 1000,
    "sem / com (tempo)": tempo_sem / tempo_com,
})

tail = pd.DataFrame(linhas)
tail

```

Out[7]:

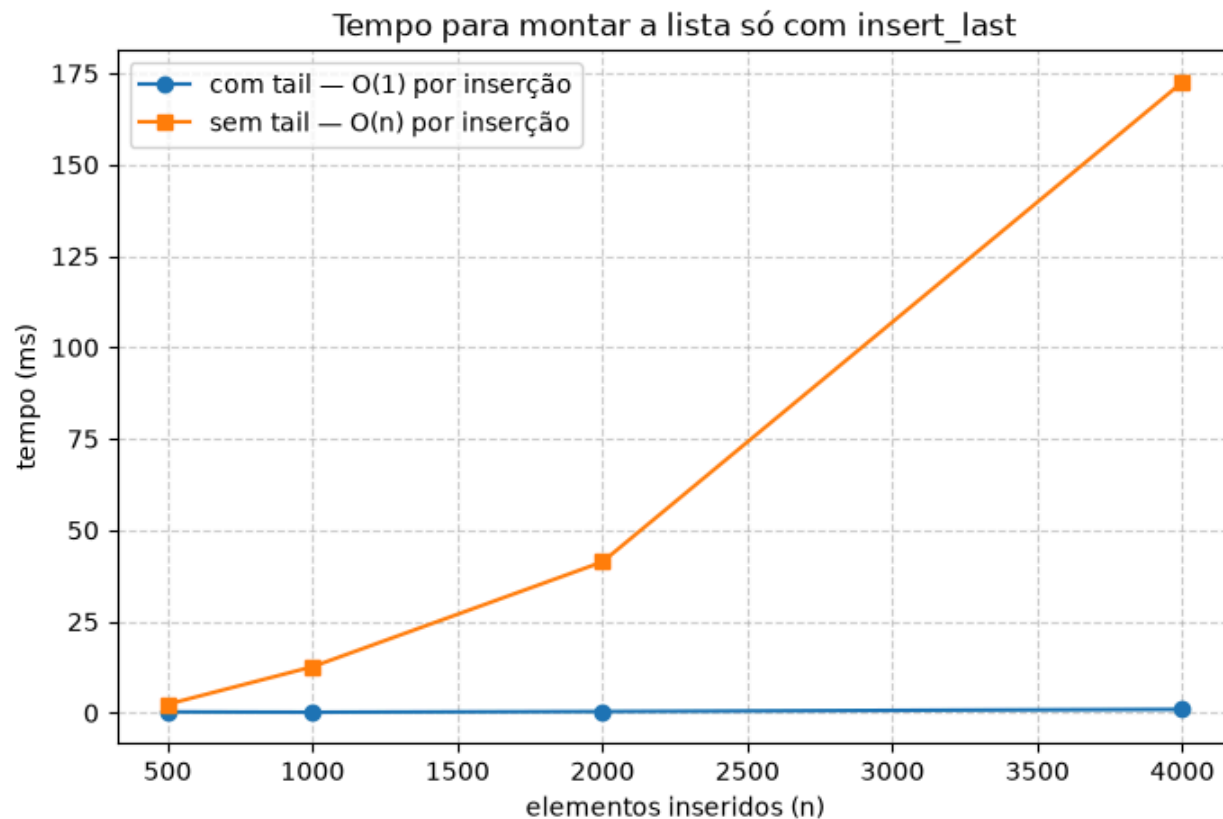
	n	saltos_com_tail	saltos_sem_tail	teoria (n-1)(n-2)/2	saltos_sem_tail / n²	ms_com_tail	ms_sem_tail	sem / com (tempo)
0	500	0	124251	124251	0.497004	0.2666	2.3510	8.818455
1	1000	0	498501	498501	0.498501	0.1843	12.6287	68.522518
2	2000	0	1997001	1997001	0.499250	0.3738	41.3191	110.537988
3	4000	0	7994001	7994001	0.499625	0.9645	172.5615	178.912908

In [8]:

```

plt.figure(figsize=(8, 5))
plt.plot(tail["n"], tail["ms_com_tail"], marker="o", label="com tail – O(1) por inserção")
plt.plot(tail["n"], tail["ms_sem_tail"], marker="s", label="sem tail – O(n) por inserção")
plt.title("Tempo para montar a lista só com insert_last")
plt.xlabel("elementos inseridos (n)")
plt.ylabel("tempo (ms)")
plt.grid(True, linestyle="--", alpha=0.6)
plt.legend()
plt.show()

```



A evidência:

- **Com** `tail` : zero saltos para qualquer `n`. Cada inserção no fim é $O(1)$, e montar a lista inteira é $O(n)$.
- **Sem** `tail` : a `k`-ésima inserção anda `k - 2` nós. O total bate **exatamente** com a conta $(n-1)(n-2)/2$ e `saltos / n²` fica estável perto de **0,5**. Montar a lista vira $O(n^2)$.
- **No tempo**, sem `tail` dobrar `n` quadruplica o tempo (a marca de $O(n^2)$). Com `tail` a lista inteira sai em cerca de 1 ms mesmo com 4.000 elementos — rápido demais para o relógio mostrar a proporção com precisão, e por isso a evidência que eu uso são os saltos, que não dependem da máquina.

Por que manter o `tail` : custa **um ponteiro por lista** (não por nó) e um cuidado a mais no código — atualizar o `tail` quando o último nó é removido, que é o `if atual is self.tail` do `_desligar`. Em troca, `insert_last` passa de $O(n)$ para $O(1)$. Como anexar no fim é justamente o uso mais comum de uma lista (montar uma lista na ordem em que os dados chegam, como as listas de ocorrências do exercício 12), a troca vale a pena.

6. Casos de borda

```

vazia = SinglyLinkedList()
assert len(vazia) == 0 and str(vazia) == "[]" and vazia.search(1) == -1
vazia.verificar()

# insert_at(0) em lista vazia vira o primeiro e o último ao mesmo tempo
vazia.insert_at(0, "único")
assert vazia.head is vazia.tail and str(vazia) == "['único']"

# esvaziar e reaproveitar
vazia.delete("único")
assert vazia.head is None and vazia.tail is None and len(vazia) == 0
vazia.insert_last("de novo")
assert vazia.to_list() == ["de novo"]
vazia.verificar()

# valores repetidos: search e delete agem na primeira ocorrência
repetidos = SinglyLinkedList([5, 7, 5, 7])
assert repetidos.search(7) == 1
repetidos.delete(5)
assert repetidos.to_list() == [7, 5, 7]

# delete_at no último ajusta o tail
l = SinglyLinkedList([1, 2, 3])
assert l.delete_at(2) == 3 and l.tail.value == 2
l.insert_last(4)
assert l.to_list() == [1, 2, 4]
l.verificar()

# índice booleano é recusado (em Python, True vale 1, mas aqui é quase sempre um engano)
try:
    l.insert_at(True, 0)
except TypeError as erro:
    print(f"insert_at(True, 0) → {erro}")

print("todos os casos de borda passaram")

```

insert_at(True, 0) → insert_at: o índice precisa ser inteiro, recebi True
 todos os casos de borda passaram

Exercício 11 — Lista duplamente encadeada e Deque

Henrique Goldstein Maluhy Mendes Amaral · DR1_AT — Algoritmos e Estruturas de Dados · Infnet

Para rodar: **Ambiente de execução** → **Executar tudo**. O notebook é independente, não precisa de nenhum arquivo externo.

Roteiro: (1) `DoublyLinkedList` com `prev` e `next` · (2) `Deque` construído sobre ela · (3) as invariantes e como verificar · (4) bateria de testes alternando as duas pontas · (5) evidência de $O(1)$ · (6) casos de borda

In [1]:

```

import random
import time

import pandas as pd

random.seed(42)

```

1. A lista duplamente encadeada

Cada nó aponta para o **próximo** (`next`) e para o **anterior** (`prev`). É esse `prev` que falta na lista simples: com ele, remover o último nó é $O(1)$, porque do último eu chego no penúltimo sem percorrer a lista.

Decisões:

- **Estrutura vazia** = `head` e `tail` iguais a `None` e `size` igual a 0. Remover de uma estrutura vazia levanta `EmptyDequeError` com mensagem clara. Não devolvo `None`, porque `None` pode ser um valor guardado de verdade.
- **Gerenciamento de nós**: quando um nó sai, zero o `prev` e o `next` dele. Assim ele não fica segurando referência para nós que continuam na lista, e se alguém guardou esse nó por engano, ele não dá acesso à estrutura.

In [2]:

```
class EmptyDequeError(Exception):
    """Remoção ou consulta numa estrutura vazia."""

class Node:
    def __init__(self, value):
        self.value = value
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def is_empty(self):
        return self.size == 0

    def __len__(self):
        return self.size

    def insert_first(self, value):
        novo = Node(value)
        if self.head is None:
            self.head = self.tail = novo          # único nó: é o primeiro e o último
        else:
            novo.next = self.head
            self.head.prev = novo
            self.head = novo
            self.size += 1

    def insert_last(self, value):
        novo = Node(value)
        if self.tail is None:
            self.head = self.tail = novo
        else:
            novo.prev = self.tail
            self.tail.next = novo
```

```

        self.tail = novo
        self.size += 1

    def delete_first(self):
        if self.head is None:
            raise EmptyDequeError("delete_first: a lista está vazia")
        no = self.head
        self.head = no.next
        if self.head is None:
            self.tail = None                # era o último nó: a lista esvaziou
        else:
            self.head.prev = None
        no.next = None                    # o nó removido não aponta mais para a lista
        self.size -= 1
        return no.value

    def delete_last(self):
        if self.tail is None:
            raise EmptyDequeError("delete_last: a lista está vazia")
        no = self.tail
        self.tail = no.prev                # o prev dá o penúltimo direto: O(1)
        if self.tail is None:
            self.head = None
        else:
            self.tail.next = None
        no.prev = None
        self.size -= 1
        return no.value

    def to_list(self):
        valores, atual = [], self.head
        while atual is not None:
            valores.append(atual.value)
            atual = atual.next
        return valores

    def to_list_reverse(self):
        valores, atual = [], self.tail
        while atual is not None:
            valores.append(atual.value)
            atual = atual.prev
        return valores

```

2. O Deque sobre a lista

O deque **usa** uma lista duplamente encadeada por dentro (composição), em vez de herdar dela. Assim a interface do deque tem **só** as operações das pontas, todas $O(1)$: quem usa o deque não consegue chamar, por acidente, nada que percorra a lista.

In [3]:

```

class Deque:
    def __init__(self):
        self._lista = DoublyLinkedList()

    def is_empty(self):
        return self._lista.is_empty()

    def __len__(self):
        return len(self._lista)

```

```

def insert_left(self, value):
    self._lista.insert_first(value)

def insert_right(self, value):
    self._lista.insert_last(value)

def remove_left(self):
    if self.is_empty():
        raise EmptyDequeError("remove_left: o deque está vazio")
    return self._lista.delete_first()

def remove_right(self):
    if self.is_empty():
        raise EmptyDequeError("remove_right: o deque está vazio")
    return self._lista.delete_last()

def peek_left(self):
    if self.is_empty():
        raise EmptyDequeError("peek_left: o deque está vazio")
    return self._lista.head.value

def peek_right(self):
    if self.is_empty():
        raise EmptyDequeError("peek_right: o deque está vazio")
    return self._lista.tail.value

def to_list(self):
    return self._lista.to_list()

d = Deque()
d.insert_right(2)
d.insert_right(3)
d.insert_left(1)
print("deque:", d.to_list(), "| esquerda:", d.peak_left(), "| direita:", d.peak_right())
print("remove_left:", d.remove_left(), "| remove_right:", d.remove_right(), "| sobra:", d.to_list())

```

```

deque: [1, 2, 3] | esquerda: 1 | direita: 3
remove_left: 1 | remove_right: 3 | sobra: [2]

```

3. As invariantes

Depois de qualquer operação, a lista precisa satisfazer:

1. `size` é igual ao número de nós percorridos do `head` até o fim;
2. `head.prev` é `None` e `tail.next` é `None` ;
3. para todo nó, `no.next.prev` é o próprio nó (os dois ponteiros concordam);
4. percorrer **de trás para frente** dá exatamente a sequência de ida **invertida**;
5. vazia $\Leftrightarrow$ `head is None` $\Leftrightarrow$ `tail is None` .

A 3ª e a 4ª são as que pegam os erros típicos de lista dupla: esquecer de atualizar um dos dois ponteiros deixa a ida certa e a volta errada.

In [4]:

```
def verificar(lista):
    if lista.size == 0:
        assert lista.head is None and lista.tail is None, "lista vazia com head/tail sobrando"
        return
    assert lista.head.prev is None, "head.prev deveria ser None"
    assert lista.tail.next is None, "tail.next deveria ser None"
    contados, atual = 0, lista.head
    while atual is not None:
        contados += 1
        assert contados <= lista.size, "mais nós do que o size (ciclo?)"
        if atual.next is not None:
            assert atual.next.prev is atual, f"ponteiros discordam no nó {atual.value!r}"
        atual = atual.next
    assert contados == lista.size, f"size = {lista.size}, mas há {contados} nós"
    assert lista.to_list_reverse() == lista.to_list()[::-1], "a volta não é a ida invertida"

# a verificação precisa FALHAR quando a estrutura está errada, senão ela não prova nada
quebrada = DoublyLinkedList()
for v in [1, 2, 3]:
    quebrada.insert_last(v)
quebrada.head.next.prev = None      # estrago um ponteiro de propósito
try:
    verificar(quebrada)
    print("ERRO: a verificação deixou passar uma lista corrompida")
except AssertionError as erro:
    print(f"lista corrompida de propósito → detectado: {erro}")
```

lista corrompida de propósito → detectado: ponteiros discordam no nó 1

4. Bateria de testes alternando as duas pontas

Cada sequência roda num deque novo, com uma lista do Python fazendo as mesmas operações como **gabarito**. Depois de **cada operação** confiro o conteúdo e as invariantes.

In [5]:

```
def executar(nome, operacoes):
    deque, gabarito = Deque(), []
    for op, valor in operacoes:
        if op == "insert_left":
            deque.insert_left(valor)
            gabarito.insert(0, valor)
        elif op == "insert_right":
            deque.insert_right(valor)
            gabarito.append(valor)
        elif op == "remove_left":
            assert deque.remove_left() == gabarito.pop(0)
        elif op == "remove_right":
            assert deque.remove_right() == gabarito.pop()
    verificar(deque._lista)
    assert deque.to_list() == gabarito
    if gabarito:
        assert deque.peek_left() == gabarito[0] and deque.peek_right() == gabarito[-1]
    return {"sequência": nome, "operações": len(operacoes), "tamanho final": len(deque),
            "invariantes": "ok em todos os passos"}
```

n = 500

```

sequencias = {
    "entra à esquerda, sai à direita (fila)":
        [ ("insert_left", i) for i in range(n) ] + [ ("remove_right", None) ] * n,
    "entra e sai à direita (pilha)":
        [ ("insert_right", i) for i in range(n) ] + [ ("remove_right", None) ] * n,
    "entra alternando, sai alternando":
        [ ("insert_left" if i % 2 else "insert_right", i) for i in range(n) ]
        + [ ("remove_left" if i % 2 else "remove_right", None) for i in range(n) ],
    "enche e esvazia várias vezes":
        [ ("insert_right", i) for i in range(50) ] + [ ("remove_left", None) ] * 50 * 10,
}

# e uma sequência longa sorteada, que nunca tenta remover de deque vazio
aleatoria, tamanho = [], 0
for i in range(20_000):
    if tamanho == 0 or random.random() < 0.5:
        aleatoria.append((random.choice(["insert_left", "insert_right"]), i))
        tamanho += 1
    else:
        aleatoria.append((random.choice(["remove_left", "remove_right"]), None))
        tamanho -= 1
sequencias["20.000 operações sorteadas nas duas pontas"] = aleatoria

pd.DataFrame([executar(nome, ops) for nome, ops in sequencias.items()])

```

Out[5]:

	sequência	operações	tamanho final	invariantes
0	entra à esquerda, sai à direita (fila)	1000	0	ok em todos os passos
1	entra e sai à direita (pilha)	1000	0	ok em todos os passos
2	entra alternando, sai alternando	1000	0	ok em todos os passos
3	enche e esvazia várias vezes	1000	0	ok em todos os passos
4	20.000 operações sorteadas nas duas pontas	20000	290	ok em todos os passos

As cinco sequências passaram com as invariantes conferidas **a cada operação** — 24 mil verificações no total. As três primeiras testam os usos clássicos (fila, pilha e as duas pontas intercaladas); a quarta testa a passagem repetida pelo estado vazio, que é onde `head` e `tail` precisam voltar a `None` juntos.

5. Evidência de O(1)

Se as operações são O(1), o tempo **por operação** não pode depender do tamanho do deque. Encho deques de tamanhos bem diferentes e meço 20.000 operações misturadas em cada um. Para comparar, faço o mesmo com uma **lista do Python usada como deque**, onde tirar e pôr na esquerda (`insert(0, x)` e `pop(0)`) desloca todos os elementos:

In [6]:

```

def medir(tamanho):
    deque, lista = Deque(), []
    for i in range(tamanho):

```

```

        deque.insert_right(i)
        lista.append(i)
ops = [random.choice(["esq", "dir"]) for _ in range(20_000)]

inicio = time.perf_counter()
for op in ops:
    if op == "esq":
        deque.insert_left(0)
        deque.remove_left()
    else:
        deque.insert_right(0)
        deque.remove_right()
tempo_deque = time.perf_counter() - inicio

inicio = time.perf_counter()
for op in ops:
    if op == "esq":
        lista.insert(0, 0)
        lista.pop()
    else:
        lista.append(0)
        lista.pop()
tempo_lista = time.perf_counter() - inicio
return tempo_deque, tempo_lista

linhas = []
for tamanho in [1_000, 10_000, 100_000, 400_000]:
    tempo_deque, tempo_lista = medir(tamanho)
    linhas.append({"tamanho do deque": tamanho,
                  "µs por operação — Deque": tempo_deque / 40_000 * 1e6,
                  "µs por operação — lista do Python": tempo_lista / 40_000 * 1e6})

o1 = pd.DataFrame(linhas)
o1["Deque: variação vs menor tamanho"] = o1["µs por operação — Deque"] / o1["µs por operação — Deque"].iloc[0]
o1

```

Out[6]:

	tamanho do deque	µs por operação — Deque	µs por operação — lista do Python	Deque: variação vs menor tamanho
0	1000	0.176087	0.085377	1.000000
1	10000	0.220717	5.261830	1.253454
2	100000	0.203943	52.917735	1.158188
3	400000	0.159373	129.021150	0.905076

- **Deque:** o tempo por operação fica praticamente o mesmo com o deque 400 vezes maior — é $O(1)$. Nenhuma operação percorre a lista: todas mexem só em `head` ou `tail` e em seus vizinhos imediatos.
- **Lista do Python:** o lado esquerdo é $O(n)$, porque `insert(0, x)` e `pop(0)` deslocam todo o vetor. O tempo por operação cresce com o tamanho.
- **Com 1.000 elementos a lista do Python ainda ganha:** deslocar mil referências num bloco contínuo de memória é muito rápido, mais rápido que criar um objeto `Node` e religar ponteiros. A vantagem do Deque não é ser rápido em qualquer tamanho — é **não depender do tamanho**. A partir de alguns milhares de elementos, a lista perde, e a diferença só aumenta.

Custo por operação

operação	custo	por quê
<code>insert_left</code> / <code>insert_first</code>	$O(1)$	liga o nó novo antes do <code>head</code>
<code>insert_right</code> / <code>insert_last</code>	$O(1)$	liga o nó novo depois do <code>tail</code>
<code>remove_left</code> / <code>delete_first</code>	$O(1)$	o <code>head</code> avança um nó
<code>remove_right</code> / <code>delete_last</code>	$O(1)$	o <code>tail.prev</code> dá o penúltimo direto (na lista simples era $O(n)$)
<code>peek_left</code> / <code>peek_right</code>	$O(1)$	leitura de <code>head</code> / <code>tail</code>
<code>is_empty</code> / <code>__len__</code>	$O(1)$	<code>size</code> mantido a cada operação
verificar invariantes	$O(n)$	percorre ida e volta — usado só nos testes

O preço do $O(1)$ nas duas pontas é **um ponteiro a mais por nó** (o `prev`) e o cuidado de atualizar os dois ponteiros em toda operação — que é exatamente o que a bateria de testes confere.

6. Casos de borda

In [7]:

```
vazio = Deque()
assert vazio.is_empty() and len(vazio) == 0

# as quatro consultas/remoções em deque vazio dão erro claro, e o deque continua utilizável
for operacao in (vazio.remove_left, vazio.remove_right, vazio.peek_left, vazio.peek_right):
    try:
        operacao()
        raise AssertionError("deveria ter levantado EmptyDequeError")
    except EmptyDequeError as erro:
        print(f"{erro}")
verificar(vazio._lista)

# um elemento só: ele é a esquerda e a direita ao mesmo tempo
vazio.insert_left("x")
assert vazio.peek_left() == vazio.peek_right() == "x"
assert vazio._lista.head is vazio._lista.tail
assert vazio.remove_right() == "x" and vazio.is_empty()
verificar(vazio._lista)

# None é um valor válido e não se confunde com "vazio"
vazio.insert_right(None)
assert len(vazio) == 1 and vazio.peek_left() is None and vazio.remove_left() is None

# o nó removido não fica apontando para a lista
lista = DoublyLinkedList()
for v in [1, 2, 3]:
    lista.insert_last(v)
primeiro = lista.head
lista.delete_first()
assert primeiro.next is None and primeiro.prev is None
```

```
print("\ntodos os casos de borda passaram")
```

```
remove_left: o deque está vazio  
remove_right: o deque está vazio  
peek_left: o deque está vazio  
peek_right: o deque está vazio
```

```
todos os casos de borda passaram
```

Exercício 12 — Motor de indexação e consulta com estruturas combinadas

Henrique Goldstein Maluhy Mendes Amaral · DR1_AT — Algoritmos e Estruturas de Dados · Infnet

Para rodar: **Ambiente de execução** → **Executar tudo**. O notebook é independente, não precisa de nenhum arquivo externo.

Roteiro: (1) documentos e tokenização · (2) as estruturas · (3) indexação invertida · (4) BST de termos: listagem e busca · (5) diagnóstico da BST com BFS · (6) consultas com AND, OR, NOT e parênteses · (7) ranqueamento com QuickSort instrumentado · (8) sugestão de termos com distância de edição memoizada · (9) duas otimizações com evidência · (10) análise de custos · (11) casos de borda · (12) conferência das oito peças obrigatórias

In [1]:

```
import random  
import time  
import unicodedata  
  
import matplotlib.pyplot as plt  
import pandas as pd  
  
random.seed(42)
```

1. Os documentos e a tokenização

Os documentos ficam definidos aqui mesmo no notebook. Cada um recebe um `doc_id` pela posição na lista.

Normalização: passo tudo para minúsculas, tiro os acentos (`binária` → `binaria` , para a busca não depender de o usuário digitar o acento), separo por tudo que não é letra ou número e descarto palavras muito comuns (artigos e preposições), que aparecem em todo documento e não ajudam a distinguir nenhum.

In [2]:

```
DOCUMENTOS = [  
    "A busca binária divide o vetor ordenado ao meio a cada passo.",  
    "A busca linear percorre a lista elemento por elemento até achar o valor.",  
    "Uma lista encadeada guarda cada valor em um nó que aponta para o próximo nó.",  
    "A pilha segue a regra: o último a entrar é o primeiro a sair.",  
    "A fila segue a regra: o primeiro a entrar é o primeiro a sair.",
```

```

"A tabela hash usa uma função hash para achar o bucket de cada chave.",
"Colisões na tabela hash podem ser tratadas com uma lista encadeada em cada bucket.",
"A árvore binária de busca guarda as chaves menores à esquerda e as maiores à direita.",
"O quicksort escolhe um pivô e divide o vetor em menores e maiores que ele.",
"A programação dinâmica guarda o resultado dos subproblemas para não recalculá-los.",
"A recursão resolve o problema chamando a própria função num caso menor.",
"Ordenar o vetor permite trocar a busca linear pela busca binária.",
]

PALAVRAS_VAZIAS = {"a", "o", "as", "os", "de", "da", "do", "das", "dos", "e", "em", "um", "uma",
                  "para", "com", "que", "no", "na", "num", "ao", "por", "pela", "pelo", "se", "ser", "ele", "cada"}

def normalizar(texto):
    sem_acento = unicodedata.normalize("NFD", texto.lower())
    return "".join(c for c in sem_acento if unicodedata.category(c) != "Mn")

def tokenizar(texto):
    """Devolve a lista de termos normalizados, na ordem em que aparecem."""
    termos, atual = [], ""
    for caractere in normalizar(texto) + " ":
        if caractere.isalnum():
            atual += caractere
        else:
            if atual and atual not in PALAVRAS_VAZIAS:
                termos.append(atual)
            atual = ""
    return termos

print(tokenizar(DOCUMENTOS[0]))
print(tokenizar(DOCUMENTOS[7]))

```

```

['busca', 'binaria', 'divide', 'vetor', 'ordenado', 'meio', 'passo']
['arvore', 'binaria', 'busca', 'guarda', 'chaves', 'menores', 'esquerda', 'maiores', 'direita']

```

2. As estruturas

Lista encadeada de ocorrências

Para cada termo, uma lista encadeada de **ocorrências**: cada nó guarda `(doc_id, posições)` — em que documento o termo aparece e em quais posições dele. Tem ponteiro `tail`, então anexar no fim é $O(1)$.

Detalhe importante: como os documentos são indexados **em ordem de** `doc_id`, cada nova ocorrência entra sempre no fim, e **toda lista fica ordenada por** `doc_id` **automaticamente**, sem custo nenhum. É isso que vai permitir as consultas eficientes da seção 6.

In [3]:

```

class NoOcorrencia:
    def __init__(self, doc_id, posicoes):
        self.value = (doc_id, posicoes)
        self.next = None

class ListaOcorrencias:

```

```

def __init__(self):
    self.head = None
    self.tail = None
    self.size = 0

def insert_last(self, doc_id, posicoes):
    novo = NoOcorrencia(doc_id, posicoes)
    if self.head is None:
        self.head = novo
    else:
        self.tail.next = novo
        self.tail = novo
        self.size += 1

def __iter__(self):
    atual = self.head
    while atual is not None:
        yield atual.value
        atual = atual.next

def doc_ids(self):
    return [doc_id for doc_id, _ in self]

```

Tabela hash com encadeamento

A mesma ideia do exercício 4: cada bucket é uma lista encadeada de pares `(chave, valor)`, com rehash quando o fator de carga passa de 0,75. A função hash é própria (e não o `hash()` do Python, que muda o hash das strings a cada execução), para os números saírem iguais em toda execução. Aceita strings e tuplas de inteiros — as tuplas são usadas no memo da seção 8.

In [4]:

```

class NoBucket:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.next = None

class HashTableChained:
    LIMIAR = 0.75

    def __init__(self, capacidade=8):
        self.capacidade = capacidade
        self.buckets = [None] * capacidade
        self.quantidade = 0
        self.comparacoes = 0

    def _funcao_hash(self, key):
        if isinstance(key, int):
            return key
        if isinstance(key, tuple):
            h = 0
            for parte in key:
                h = (h * 1_000_003 + self._funcao_hash(parte)) % (2**32)
            return h
        h = 0
        for caractere in key:
            h = (h * 31 + ord(caractere)) % (2**32)
        return h

```

```

def _indice(self, key):
    return self._funcao_hash(key) % self.capacidade

def put(self, key, value):
    i = self._indice(key)
    atual = self.buckets[i]
    while atual is not None:
        self.comparacoes += 1
        if atual.key == key:
            atual.value = value
            return
        atual = atual.next
    novo = NoBucket(key, value)
    novo.next = self.buckets[i]
    self.buckets[i] = novo
    self.quantidade += 1
    if self.quantidade / self.capacidade > self.LIMIAR:
        self._rehash()

def get(self, key, padrao=None):
    atual = self.buckets[self._indice(key)]
    while atual is not None:
        self.comparacoes += 1
        if atual.key == key:
            return atual.value
        atual = atual.next
    return padrao

def __len__(self):
    return self.quantidade

def _rehash(self):
    antigos = self.buckets
    self.capacidade *= 2
    self.buckets = [None] * self.capacidade
    for cabeca in antigos:
        atual = cabeca
        while atual is not None:
            proximo = atual.next
            i = self._indice(atual.key)
            atual.next = self.buckets[i]
            self.buckets[i] = atual
            atual = proximo

```

BST de termos

Uma árvore binária de busca com os termos do vocabulário, para **listar em ordem alfabética** e **buscar por termo**. Inserção, busca e in-order são **recursivas**: a altura esperada é pequena (perto de $2 \cdot \log_2 n$ para termos em ordem de aparição), então a recursão não corre risco aqui.

In [5]:

```

class NoTermo:
    def __init__(self, termo):
        self.termo = termo
        self.esquerda = None
        self.direita = None

class BSTTermos:

```

```

def __init__(self):
    self.raiz = None
    self.tamanho = 0

def insert(self, termo):
    self.raiz = self._inserir(self.raiz, termo)

def _inserir(self, no, termo):
    if no is None:
        self.tamanho += 1
        return NoTermo(termo)
    if termo < no.termo:
        no.esquerda = self._inserir(no.esquerda, termo)
    elif termo > no.termo:
        no.direita = self._inserir(no.direita, termo)
    return no  # termo repetido: não insere de novo

def search(self, termo):
    """Devolve (achou, comparações)."""
    return self._buscar(self.raiz, termo, 0)

def _buscar(self, no, termo, comparacoes):
    if no is None:
        return False, comparacoes
    if termo == no.termo:
        return True, comparacoes + 1
    proximo = no.esquerda if termo < no.termo else no.direita
    return self._buscar(proximo, termo, comparacoes + 1)

def in_order(self):
    termos = []
    self._em_ordem(self.raiz, termos)
    return termos

def _em_ordem(self, no, termos):
    if no is not None:
        self._em_ordem(no.esquerda, termos)
        termos.append(no.termo)
        self._em_ordem(no.direita, termos)

```

Pilha e fila

A **pilha** vai ser usada na conversão da consulta para pós-fixa e na avaliação. A **fila** vai ser usada na travessia em largura da BST. A fila é **por encadeamento** (como na aula de pilhas e filas por encadeamento): `enqueue` no fim e `dequeue` no começo, os dois O(1).

In [6]:

```

class Stack:
    def __init__(self):
        self.itens = []

    def push(self, item):
        self.itens.append(item)

    def pop(self):
        if not self.itens:
            raise IndexError("pop em pilha vazia")
        return self.itens.pop()

    def peek(self):

```

```

        return self.itens[-1] if self.itens else None

    def is_empty(self):
        return not self.itens

    def __len__(self):
        return len(self.itens)

class NoFila:
    def __init__(self, valor):
        self.valor = valor
        self.prox = None

class Queue:
    def __init__(self):
        self.inicio = None
        self.fim = None
        self.tamanho = 0

    def enqueue(self, valor):
        novo = NoFila(valor)
        if self.fim is None:
            self.inicio = novo
        else:
            self.fim.prox = novo
        self.fim = novo
        self.tamanho += 1

    def dequeue(self):
        if self.inicio is None:
            raise IndexError("dequeue em fila vazia")
        valor = self.inicio.valor
        self.inicio = self.inicio.prox
        if self.inicio is None:
            self.fim = None
        self.tamanho -= 1
        return valor

    def is_empty(self):
        return self.inicio is None

```

3. A indexação

Para cada documento, na ordem dos `doc_id` :

1. tokenizo e agrupo as posições de cada termo dentro do documento;
2. para cada termo, pego a lista de ocorrências dele na hashtable (ou crio uma nova, e aí o termo também entra na BST);
3. anexo `(doc_id, posições)` no fim da lista.

In [7]:

```

class IndiceInvertido:
    def __init__(self, documentos):
        self.documentos = documentos
        self.tabela = HashTableChained()

```

```

self.termos = BSTTermos()
self.total_tokens = 0
for doc_id, texto in enumerate(documentos):
    self._indexar(doc_id, texto)

def _indexar(self, doc_id, texto):
    posicoes_no_doc = HashTableChained() # termo → posições dentro deste documento
    ordem = [] # termos na ordem em que apareceram
    for posicao, termo in enumerate(tokenizar(texto)):
        self.total_tokens += 1
        posicoes = posicoes_no_doc.get(termo)
        if posicoes is None:
            posicoes = []
            posicoes_no_doc.put(termo, posicoes)
            ordem.append(termo)
        posicoes.append(posicao)
    for termo in ordem:
        lista = self.tabela.get(termo)
        if lista is None:
            lista = ListaOcorrencias()
            self.tabela.put(termo, lista)
            self.termos.insert(termo) # termo novo no vocabulário
        lista.insert_last(doc_id, posicoes_no_doc.get(termo))

def ocorrencias(self, termo):
    return self.tabela.get(termo) or ListaOcorrencias()

def doc_ids(self, termo):
    return self.ocorrencias(termo).doc_ids()

indice = IndiceInvertido(DOCUMENTOS)
print(f"{len(DOCUMENTOS)} documentos, {indice.total_tokens} termos indexados, "
      f"vocabulário de {indice.termos.tamanho} termos distintos\n")
for termo in ["busca", "hash", "lista", "encadeada", "vetor"]:
    print(f"{termo:<10} → {list(indice.ocorrencias(termo))}")

```

12 documentos, 91 termos indexados, vocabulário de 60 termos distintos

```

busca    → [(0, [0]), (1, [0]), (7, [2]), (11, [4, 6])]
hash     → [(5, [1, 4]), (6, [2])]
lista    → [(1, [3]), (2, [0]), (6, [5])]
encadeada → [(2, [1]), (6, [6])]
vetor    → [(0, [3]), (8, [4]), (11, [1])]

```

Cada linha é `(doc_id, [posições])`. Por exemplo, `busca` aparece duas vezes no documento 11 (posições 4 e 6).

Conferência: comparo o índice com uma contagem direta, feita do jeito mais simples possível (e mais lento), percorrendo todos os documentos para cada termo. As duas têm que bater, e todas as listas têm que estar ordenadas por `doc_id`:

In [8]:

```

vocabulario = indice.termos.in_order()
for termo in vocabulario:
    esperado = []
    for doc_id, texto in enumerate(DOCUMENTOS):
        posicoes = [p for p, t in enumerate(tokenizar(texto)) if t == termo]
        if posicoes:
            esperado.append((doc_id, posicoes))
    assert list(indice.ocorrencias(termo)) == esperado, termo
ids = indice.doc_ids(termo)

```

```
assert ids == sorted(ids) # ordenada por doc_id, sem esforço extra
print(f"os {len(vocabulario)} termos conferidos: ocorrências corretas e listas ordenadas por doc_id")
```

os 60 termos conferidos: ocorrências corretas e listas ordenadas por doc_id

4. BST de termos: listagem ordenada e busca

In [9]:

```
print("vocabulário em ordem alfabética (in-order):\n")
print(", ".join(vocabulario))
assert vocabulario == sorted(vocabulario)

print()
for termo in ["recursao", "quicksort", "grafo"]:
    achou, comparacoes = indice.termos.search(termo)
    print(f"busca por '{termo}': {'achou' if achou else 'não existe'} com {comparacoes} comparações")
```

vocabulário em ordem alfabética (in-order):

achar, aponta, arvore, ate, binaria, bucket, busca, caso, chamando, chave, chaves, colisoes, dinamica, direita, divide, elemento, encadeada, entrar, escolhe, esquerda, fila, funcao, guarda, hash, linear, lista, maiores, meio, menor, menores, nao, ordenado, ordenar, passo, percorre, permite, pilha, pivo, podem, primeiro, problema, programacao, propria, proximo, quicksort, recalcular, recursao, regra, resolve, resultado, sair, segue, subproblemas, tabela, tratadas, trocar, ultimo, usa, valor, vetor

busca por 'recursao': achou com 13 comparações
busca por 'quicksort': achou com 11 comparações
busca por 'grafo': não existe com 12 comparações

5. Diagnóstico da BST com travessia em largura

A travessia em largura (BFS) visita a árvore **nível por nível**, usando a **fila**: tiro um nó do começo, registro, e coloco os filhos dele no fim. Com isso monto o relatório: quantos termos em cada nível, a altura real, a altura mínima possível e o equilíbrio entre os dois lados da raiz.

In [10]:

```
def altura(no):
    if no is None:
        return 0
    return 1 + max(altura(no.esquerda), altura(no.direita))

def relatorio_bfs(bst):
    fila = Queue()
    fila.enqueue((bst.raiz, 0))
    por_nivel = []
    while not fila.is_empty():
        no, nivel = fila.dequeue()
        if no is None:
            continue
        if nivel == len(por_nivel):
            por_nivel.append([])
        por_nivel[nivel].append(no.termo)
        fila.enqueue((no.esquerda, nivel + 1))
        fila.enqueue((no.direita, nivel + 1))
```

```

n = bst.tamanho
minima = n.bit_length() # /log2(n + 1): altura da árvore mais cheia possível
return {
    "termos": n,
    "altura": len(por_nivel),
    "altura_minima": minima,
    "altura / minima": len(por_nivel) / minima,
    "altura_esquerda_da_raiz": altura(bst.raiz.esquerda),
    "altura_direita_da_raiz": altura(bst.raiz.direita),
    "por_nivel": por_nivel,
}

diag = relatorio_bfs(indice.termos)
print(f"raiz: '{indice.termos.raiz.termo}' · {diag['termos']} termos · altura {diag['altura']} "
      f"(mínima possível: {diag['altura_minima']}) · razão {diag['altura / minima']:.2f}")
print(f"subárvore esquerda com altura {diag['altura_esquerda_da_raiz']}, "
      f"direita com {diag['altura_direita_da_raiz']}\n")
for nivel, termos in enumerate(diag["por_nivel"]):
    print(f"nível {nivel:>2} ({len(termos):>2} termos): {' · '.join(termos)}")

```

raiz: 'busca' · 60 termos · altura 13 (mínima possível: 6) · razão 2.17
subárvore esquerda com altura 5, direita com 12

```

nível 0 ( 1 termos): busca
nível 1 ( 2 termos): binaria, divide
nível 2 ( 4 termos): ate, bucket, chave, vetor
nível 3 ( 4 termos): achar, chamando, colisoes, ordenado
nível 4 ( 6 termos): aponta, caso, chaves, direita, meio, passo
nível 5 ( 6 termos): arvore, dinamica, linear, menores, ordenar, percorre
nível 6 ( 5 termos): elemento, lista, menor, nao, valor
nível 7 ( 3 termos): encadeada, maiores, proximo
nível 8 ( 3 termos): guarda, pilha, segue
nível 9 ( 6 termos): entrar, hash, permite, primeiro, regra, ultimo
nível 10 ( 7 termos): fila, podem, programacao, quicksort, sair, tabela, usa
nível 11 ( 9 termos): esquerda, funcao, pivo, problema, propria, recalcular, resultado, subproblemas, tratadas
nível 12 ( 4 termos): escolhe, recursao, resolve, trocar

```

Leitura: a altura fica perto do dobro da mínima, e a raiz está desequilibrada: a subárvore da direita é bem mais alta que a da esquerda. O motivo é que o primeiro termo inserido — que vira a raiz — foi `busca`, que começa com `b`: quase todo o vocabulário vem depois dele em ordem alfabética e vai para a direita. Isso é o esperado para uma BST sem balanceamento em que os termos chegam numa ordem mais ou menos aleatória (a ordem em que aparecem nos textos). A busca custa $O(h)$, então aqui cada busca faz no máximo `altura` comparações. Se os termos chegassem em ordem alfabética, a árvore degeneraria numa lista (altura = n) — e aí uma árvore balanceada, como a AVL, seria necessária.

6. Consultas booleanas

Passo 1 — tokens da consulta: parênteses, operadores (`AND`, `OR`, `NOT`, em maiúsculas) e termos (normalizados como os documentos).

Passo 2 — infix para pós-fix (shunting-yard) com a pilha, a mesma ideia da aula de infix para pós-fix, com a precedência `NOT` > `AND` > `OR`. O `NOT` é unário e fica à direita (`( NOT NOT x = NOT (NOT x) )`).

Passo 3 — avaliação da pós-fix com a pilha: termo → empilha a lista de `doc_id` dele; operador → desempilha, combina e empilha o resultado.

Como as listas estão **ordenadas**, `AND` e `OR` são feitos com **dois ponteiros** andando juntos, em $O(m + n)$ — a otimização 1 (seção 9) mede o ganho disso.

In [11]:

```
class ConsultaInvalidaError(Exception):
    """A consulta não pode ser interpretada."""

PRECEDENCIA = {"NOT": 3, "AND": 2, "OR": 1}

def tokens_da_consulta(consulta):
    tokens, atual = [], ""
    for caractere in consulta + " ":
        if caractere in "() ":
            if atual:
                tokens.append(atual if atual in PRECEDENCIA else normalizar(atual))
                atual = ""
            if caractere != " ":
                tokens.append(caractere)
        else:
            atual += caractere
    if not tokens:
        raise ConsultaInvalidaError("consulta vazia")
    return tokens

def para_posfixa(tokens):
    saida, pilha = [], Stack()
    anterior = None  # para detectar dois termos seguidos sem operador
    for posicao, token in enumerate(tokens, start=1):
        if token == "(":
            pilha.push(token)
        elif token == ")":
            while not pilha.is_empty() and pilha.peek() != "(":
                saida.append(pilha.pop())
            if pilha.is_empty():
                raise ConsultaInvalidaError(f"')' na posição {posicao} não tem '(' correspondente")
            pilha.pop()  # descarta o '('
        elif token in PRECEDENCIA:
            # desempilha operadores de precedência maior (ou igual, se não for o NOT)
            while (not pilha.is_empty() and pilha.peek() in PRECEDENCIA and token != "NOT"
                  and PRECEDENCIA[pilha.peek()] >= PRECEDENCIA[token]):
                saida.append(pilha.pop())
            pilha.push(token)
        else:
            if anterior is not None and anterior not in PRECEDENCIA and anterior != "(":
                raise ConsultaInvalidaError(
                    f"falta um operador entre '{anterior}' e '{token}' (posição {posicao})")
            saida.append(token)
            anterior = token
    while not pilha.is_empty():
        topo = pilha.pop()
        if topo == "(":
            raise ConsultaInvalidaError("tem '(' sem o ')' correspondente")
        saida.append(topo)
    return saida

def intersecao(a, b, cont):
    """AND em listas ordenadas: dois ponteiros, O(m + n)."""
    i = j = 0
    resultado = []
    while i < len(a) and j < len(b):
        cont["comparacoes"] += 1
        if a[i] == b[j]:
```

```

        resultado.append(a[i])
        i += 1
        j += 1
    elif a[i] < b[j]:
        i += 1
    else:
        j += 1
    return resultado

def uniao(a, b, cont):
    """OR em listas ordenadas: intercalação sem repetir, O(m + n)."""
    i = j = 0
    resultado = []
    while i < len(a) and j < len(b):
        cont["comparacoes"] += 1
        if a[i] == b[j]:
            resultado.append(a[i])
            i += 1
            j += 1
        elif a[i] < b[j]:
            resultado.append(a[i])
            i += 1
        else:
            resultado.append(b[j])
            j += 1
    return resultado + a[i:] + b[j:]

def diferenca(todos, a, cont):
    """NOT: todos os documentos menos os de `a`, também com dois ponteiros."""
    resultado, j = [], 0
    for doc in todos:
        while j < len(a) and a[j] < doc:
            cont["comparacoes"] += 1
            j += 1
        if j < len(a) and a[j] == doc:
            cont["comparacoes"] += 1
            continue
        resultado.append(doc)
    return resultado

def avaliar(posfixa, indice, cont):
    pilha = Stack()
    todos = list(range(len(indice.documentos)))
    for token in posfixa:
        if token == "NOT":
            if pilha.is_empty():
                raise ConsultaInvalidaError("NOT sem termo depois dele")
            pilha.push(diferenca(todos, pilha.pop(), cont))
        elif token in ("AND", "OR"):
            if len(pilha) < 2:
                raise ConsultaInvalidaError(f"{token} precisa de um termo de cada lado")
            b, a = pilha.pop(), pilha.pop()
            pilha.push(intersecao(a, b, cont) if token == "AND" else uniao(a, b, cont))
        else:
            pilha.push(indice.doc_ids(token))
    if len(pilha) != 1:
        raise ConsultaInvalidaError("sobraram termos sem operador entre eles")
    return pilha.pop()

def consultar(consulta, indice):
    cont = {"comparacoes": 0}
    posfixa = para_posfixa(tokens_da_consulta(consulta))
    return avaliar(posfixa, indice, cont), posfixa, cont["comparacoes"]

```

```

for consulta in ["busca AND binaria",
                "busca AND NOT binaria",
                "(pilha OR fila) AND regra",
                "hash AND NOT (lista OR encadeada)",
                "vetor AND (quicksort OR busca) AND NOT linear",
                "NOT NOT arvore"]:
    docs, posfixa, comparacoes = consultar(consulta, indice)
    print(f"{consulta:<45} → docs {docs}")
    print(f"{'':<45} pós-fixa: {' '.join(posfixa)} ({comparacoes} comparações)\n")

```

busca AND binaria	→ docs [0, 7, 11] pós-fixa: busca binaria AND (4 comparações)
busca AND NOT binaria	→ docs [1] pós-fixa: busca binaria NOT AND (16 comparações)
(pilha OR fila) AND regra	→ docs [3, 4] pós-fixa: pilha fila OR regra AND (3 comparações)
hash AND NOT (lista OR encadeada)	→ docs [5] pós-fixa: hash lista encadeada OR NOT AND (14 comparações)
vetor AND (quicksort OR busca) AND NOT linear	→ docs [0, 8] pós-fixa: vetor quicksort busca OR AND linear NOT AND (22 comparações)
NOT NOT arvore	→ docs [7] pós-fixa: arvore NOT NOT (23 comparações)

Conferência das consultas: comparo cada resultado com a resposta calculada "na força bruta" — testando documento por documento se a expressão é verdadeira para ele, usando as operações de conjunto do Python só como gabarito:

In [12]:

```

def gabarito(consulta):
    termos_por_doc = [set(tokenizar(t)) for t in DOCUMENTOS]
    expressao = []
    for token in tokens_da_consulta(consulta):
        if token in ("AND", "OR", "NOT"):
            expressao.append(token.lower())
        elif token in "()":
            expressao.append(token)
        else:
            expressao.append(f"({token!r} in termos)")
    codigo = " ".join(expressao)
    return [d for d, termos in enumerate(termos_por_doc) if eval(codigo, {}, {"termos": termos})]

for consulta in ["busca AND binaria", "busca OR vetor", "NOT hash", "(pilha OR fila) AND NOT primeiro",
                "hash AND NOT (lista OR encadeada)", "NOT (busca OR lista) AND NOT vetor",
                "arvore OR recursao OR quicksort", "NOT NOT arvore", "inexistente OR fila"]:
    assert consultar(consulta, indice)[0] == gabarito(consulta), consulta
print("9 consultas conferidas contra a avaliação documento a documento")

```

9 consultas conferidas contra a avaliação documento a documento

7. Ranqueamento com QuickSort instrumentado

Para os documentos de uma consulta, calculo uma pontuação simples com os termos da consulta que **não vêm logo depois de um** `NOT` (é uma simplificação: num `NOT` `(a OR b)` os termos de dentro do parêntese ainda contariam, mas para ordenar resultados isso basta):

- **frequência:** +1 para cada vez que o termo aparece no documento;
- **posição:** + $1 / (1 + \text{primeira posição})$ — um termo que aparece logo no começo do texto vale mais que um que só aparece no fim.

Os resultados são ordenados pela pontuação (maior primeiro; empate, menor `doc_id` primeiro) com um **QuickSort recursivo** no estilo da aula — dividindo em menores, iguais e maiores que o pivô —, contando as comparações. O pivô é sorteado, para nenhuma ordem de entrada cair no pior caso.

In [13]:

```
def termos_positivos(consulta):
    tokens = tokens_da_consulta(consulta)
    positivos = []
    for i, token in enumerate(tokens):
        if token not in PRECEDENCIA and token not in "()" and (i == 0 or tokens[i - 1] != "NOT"):
            positivos.append(token)
    return positivos

def pontuar_todos(docs, termos, indice):
    """Pontos de cada documento do resultado, com UMA passada por lista de ocorrências."""
    pontos = HashTableChained()
    # doc_id → pontos
    for d in docs:
        pontos.put(d, 0.0)
    for termo in termos:
        for doc, posicoes in indice.ocorrencias(termo):
            atual = pontos.get(doc)
            if atual is not None:
                # só pontua quem está no resultado
                pontos.put(doc, atual + len(posicoes) + 1 / (1 + posicoes[0]))
    return pontos

def quicksort(itens, cont):
    """Recursivo, em três listas, como na aula. Cada item é (chave, dados)."""
    cont["chamadas"] += 1
    if len(itens) <= 1:
        return itens
    pivo = random.choice(itens)[0]
    menores, iguais, maiores = [], [], []
    for item in itens:
        cont["comparacoes"] += 1
        if item[0] < pivo:
            menores.append(item)
        else:
            cont["comparacoes"] += 1
            (maiores if item[0] > pivo else iguais).append(item)
    return quicksort(menores, cont) + iguais + quicksort(maiores, cont)

def buscar(consulta, indice):
    docs, _, _ = consultar(consulta, indice)
    termos = termos_positivos(consulta)
    pontos = pontuar_todos(docs, termos, indice)
    # chave (-pontos, doc_id): ordenar crescente põe a maior pontuação primeiro
    itens = [((-pontos.get(d), d), d) for d in docs]
```

```

cont = {"comparacoes": 0, "chamadas": 0}
ordenados = quicksort(itens, cont)
return [(d, -chave[0]) for chave, d in ordenados], cont

resultado, cont = buscar("busca OR vetor OR binaria", indice)
print(f"consulta: busca OR vetor OR binaria ({cont['comparacoes']} comparações no quicksort)\n")
for posicao, (doc_id, pontos) in enumerate(resultado, start=1):
    print(f"{posicao}ª doc {doc_id:>2} pontos {pontos:5.2f} {DOCUMENTOS[doc_id]}")

```

consulta: busca OR vetor OR binaria (14 comparações no quicksort)

```

1ª doc 11 pontos 4.83 Ordenar o vetor permite trocar a busca linear pela busca binária.
2ª doc 0 pontos 4.75 A busca binária divide o vetor ordenado ao meio a cada passo.
3ª doc 7 pontos 2.83 A árvore binária de busca guarda as chaves menores à esquerda e as maiores à direita.
4ª doc 1 pontos 2.00 A busca linear percorre a lista elemento por elemento até achar o valor.
5ª doc 8 pontos 1.20 O quicksort escolhe um pivô e divide o vetor em menores e maiores que ele.

```

O documento 11 fica em primeiro: tem `busca` duas vezes, `vetor` e `binaria`. O 0 vem logo depois, com os três termos, `busca` já na primeira posição.

8. Sugestão de termos: distância de edição com memoization

Se um termo da consulta não existe no índice, sugiro o termo do vocabulário **mais parecido**. "Parecido" = **distância de edição**: o menor número de inserções, remoções e trocas de letra que transformam uma palavra na outra.

A recursão: comparando `a[i:]` com `b[j:]`, se as letras `a[i]` e `b[j]` são iguais, sigo em frente de graça; senão, pago 1 e escolho a melhor entre remover, inserir ou trocar. O mesmo par `(i, j)` aparece em muitos ramos, então guardo cada resultado num **memo na minha** `HashTableChained`, com a chave `(i, j)` — o estado mínimo: as duas palavras não mudam durante o cálculo, só as posições.

In [14]:

```

def distancia_edicao(a, b, cont, memo=None):
    if memo is None:
        memo = HashTableChained()

    def d(i, j):
        cont["chamadas"] += 1
        guardado = memo.get((i, j))
        if guardado is not None:
            return guardado
        if i == len(a):
            resultado = len(b) - j          # sobrou b: insere o resto
        elif j == len(b):
            resultado = len(a) - i          # sobrou a: remove o resto
        elif a[i] == b[j]:
            resultado = d(i + 1, j + 1)     # letras iguais: sem custo
        else:
            resultado = 1 + min(d(i + 1, j), # remove a[i]
                               d(i, j + 1), # insere b[j]
                               d(i + 1, j + 1)) # troca a[i] por b[j]
        memo.put((i, j), resultado)
        return resultado

    return d(0, 0)

```

```
def distancia_sem_memo(a, b, cont, i=0, j=0):
    cont["chamadas"] += 1
    if i == len(a):
        return len(b) - j
    if j == len(b):
        return len(a) - i
    if a[i] == b[j]:
        return distancia_sem_memo(a, b, cont, i + 1, j + 1)
    return 1 + min(distancia_sem_memo(a, b, cont, i + 1, j),
                   distancia_sem_memo(a, b, cont, i, j + 1),
                   distancia_sem_memo(a, b, cont, i + 1, j + 1))

for a, b, esperado in [("gato", "rato", 1), ("casa", "casas", 1), ("", "abc", 3), ("kitten", "sitting", 3)]:
    assert distancia_edicao(a, b, {"chamadas": 0}) == esperado

linhas = []
for a, b in [("pilha", "fila"), ("algoritmo", "logaritmo"), ("recursao", "execucao"), ("programacao", "memorizacao")]:
    c_puro, c_memo = {"chamadas": 0}, {"chamadas": 0}
    dist = distancia_edicao(a, b, c_puro)
    assert dist == distancia_sem_memo(a, b, c_puro)
    linhas.append({"a": a, "b": b, "distancia": dist,
                  "chamadas_sem_memo": c_puro["chamadas"], "chamadas_com_memo": c_memo["chamadas"],
                  "reducao": c_puro["chamadas"] / c_memo["chamadas"]})
pd.DataFrame(linhas)
```

Out[14]:

	a	b	distancia	chamadas_sem_memo	chamadas_com_memo	reducao
0	pilha	fila	2	271	55	4.927273
1	algoritmo	logaritmo	3	155704	173	900.023121
2	recursao	execucao	4	45344	161	281.639752
3	programacao	memorizacao	6	4423302	322	13736.962733

Sem memo, cada letra diferente abre três ramos, e o número de chamadas explode (exponencial no tamanho das palavras). Com memo, cada par (i, j) é calculado uma vez só: no máximo $(|a| + 1) \cdot (|b| + 1)$ estados — $O(|a| \cdot |b|)$.

In [15]:

```
def sugerir(termo, indice, podar=True):
    """Termo do vocabulário mais próximo; empate fica com o primeiro em ordem alfabética.
    Devolve (sugestão, distância, quantas distâncias foram calculadas)."""
    termo = normalizar(termo)
    vocabulario = indice.terminos.in_order() # já sai em ordem alfabética
    melhor, calculos = None, 0 # melhor = (distância, termo)

    def comparar(candidato):
        nonlocal melhor, calculos
        calculos += 1
        dist = distancia_edicao(termo, candidato, {"chamadas": 0})
        if melhor is None or (dist, candidato) < melhor:
            melhor = (dist, candidato)

    if not podar:
        for candidato in vocabulario:
            comparar(candidato)
```

```

else:
    # agrupo os termos por tamanho e visito do tamanho mais parecido para o menos parecido
    maior = max((len(c) for c in vocabulario), default=0)
    por_tamanho = [[] for _ in range(maior + 1)]
    for candidato in vocabulario:
        por_tamanho[len(candidato)].append(candidato)
    diferenca = 0
    # a distância é no mínimo a diferença de tamanho: passou da melhor, ninguém mais ganha
    while diferenca <= maior + len(termo) and (melhor is None or diferenca <= melhor[0]):
        for tamanho in {len(termo) - diferenca, len(termo) + diferenca}:
            if 0 <= tamanho <= maior:
                for candidato in por_tamanho[tamanho]:
                    comparar(candidato)
        diferenca += 1

    if melhor is None:
        return None, None, calculos
    return melhor[1], melhor[0], calculos

def buscar_com_sugestao(consulta, indice):
    ausentes = [t for t in termos_positivos(consulta) if indice.tabela.get(t) is None]
    for termo in ausentes:
        sugestao, dist, _ = sugerir(termo, indice)
        print(f"  '{termo}' não está no índice. Você quis dizer '{sugestao}'? (distância {dist})")
    return buscar(consulta, indice)[0]

for consulta in ["binaira AND busca", "lista AND encadeda", "quiksort", "recurssao OR pilah"]:
    print(f"consulta: {consulta}")
    buscar_com_sugestao(consulta, indice)
    print()

```

```

consulta: binaira AND busca
  'binaira' não está no índice. Você quis dizer 'binaria'? (distância 2)

consulta: lista AND encadeda
  'encadeda' não está no índice. Você quis dizer 'encadeada'? (distância 1)

consulta: quiksort
  'quiksort' não está no índice. Você quis dizer 'quicksort'? (distância 1)

consulta: recurssao OR pilah
  'recurssao' não está no índice. Você quis dizer 'recursao'? (distância 1)
  'pilah' não está no índice. Você quis dizer 'fila'? (distância 2)

```

Um limite que apareceu: para `pilah` a sugestão foi `fila`, e não `pilha`, que é claramente o que se quis digitar. A causa é a própria distância de Levenshtein: ela não tem a operação "trocar duas letras vizinhas de lugar", então `ah` → `ha` custa **2** (duas trocas de letra). Com isso `fila` e `pilha` empatam em distância 2, e o empate fica com a primeira em ordem alfabética. A variante de **Damerau-Levenshtein** conta a transposição como uma operação só (distância 1) e resolveria esse caso — é uma melhoria natural para uma próxima versão, já que inverter duas letras é um dos erros de digitação mais comuns.

9. Duas otimizações, com evidência

Otimização 1 — interseção em listas ordenadas: de $O(m \cdot n)$ para $O(m + n)$

A forma ingênua de fazer `a AND b` é, para cada documento de `a`, procurar se ele está em `b`: $O(m \cdot n)$. Como as listas de ocorrências já saem **ordenadas por** `doc_id` da indexação (sem custo extra, graças à inserção no fim com `tail`), dá para andar com dois ponteiros: $O(m + n)$.

O corpus deste notebook é pequeno demais para mostrar o crescimento, então meço com listas sorteadas de tamanhos crescentes:

In [16]:

```
def intersecao_ingenua(a, b, cont):
    resultado = []
    for x in a:
        for y in b:
            cont["comparacoes"] += 1
            if x == y:
                resultado.append(x)
                break
    return resultado

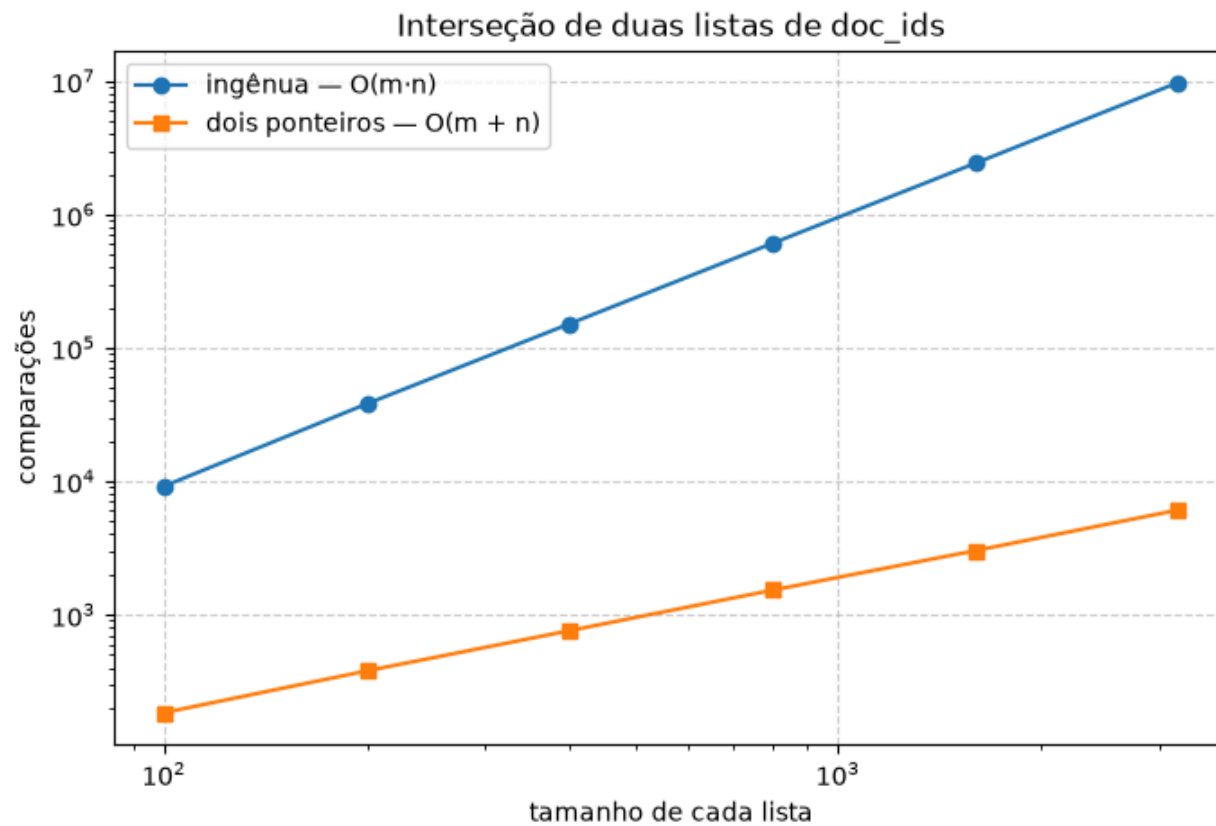
linhas = []
for tamanho in [100, 200, 400, 800, 1600, 3200]:
    a = sorted(random.sample(range(10 * tamanho), tamanho))
    b = sorted(random.sample(range(10 * tamanho), tamanho))
    c_ing, c_ord = {"comparacoes": 0}, {"comparacoes": 0}
    assert intersecao_ingenua(a, b, c_ing) == intersecao(a, b, c_ord) # mesmo resultado
    linhas.append({"tamanho de cada lista": tamanho,
                  "ingenua": c_ing["comparacoes"], "ordenada": c_ord["comparacoes"],
                  "ingenua / n²": c_ing["comparacoes"] / tamanho**2,
                  "ordenada / n": c_ord["comparacoes"] / tamanho,
                  "ganho": c_ing["comparacoes"] / c_ord["comparacoes"]})
otim1 = pd.DataFrame(linhas)
otim1
```

Out[16]:

	tamanho de cada lista	ingenua	ordenada	ingenua / n²	ordenada / n	ganho
0	100	9192	184	0.919200	1.840000	49.956522
1	200	38074	379	0.951850	1.895000	100.459103
2	400	151810	759	0.948812	1.897500	200.013175
3	800	608747	1527	0.951167	1.908750	398.655534
4	1600	2424306	3021	0.946995	1.888125	802.484608
5	3200	9771413	6093	0.954240	1.904062	1603.711308

In [17]:

```
plt.figure(figsize=(8, 5))
plt.plot(otim1["tamanho de cada lista"], otim1["ingenua"], marker="o", label="ingênua —  $O(m \cdot n)$ ")
plt.plot(otim1["tamanho de cada lista"], otim1["ordenada"], marker="s", label="dois ponteiros —  $O(m + n)$ ")
plt.xscale("log")
plt.yscale("log")
plt.title("Interseção de duas listas de doc_ids")
plt.xlabel("tamanho de cada lista")
plt.ylabel("comparações")
plt.grid(True, linestyle="--", alpha=0.6)
plt.legend()
plt.show()
```



ordenada / n e ingenua / n^2 ficam estáveis enquanto o tamanho cresce 32 vezes: uma é linear, a outra quadrática. Por isso o **ganho cresce com o tamanho** — é ganho de classe, não de constante. E o resultado é idêntico nas duas.

Otimização 2 — poda por comprimento na sugestão

A distância de edição entre duas palavras é **no mínimo a diferença de tamanho** entre elas (é preciso inserir ou remover pelo menos essas letras). A `sugerir` usa isso assim: agrupa o vocabulário por tamanho e visita primeiro os termos do **mesmo tamanho**, depois os que diferem em 1, depois em 2, e assim por diante. Quando a diferença de tamanho passa da melhor distância já encontrada, **para**: nenhum termo mais distante em tamanho pode ganhar.

Meço num vocabulário maior, sorteado, com consultas que são **erros de digitação** de palavras do vocabulário (uma letra trocada) — que é o uso real da sugestão:

In [18]:

```
def palavra_aleatoria():
    return "".join(random.choice("abcdefghijklmnopqrstuvwxyz") for _ in range(random.randint(3, 14)))

def com_erro_de_digitacao(palavra):
    i = random.randrange(len(palavra))
    return palavra[:i] + random.choice("abcdefghijklmnopqrstuvwxyz") + palavra[i + 1:]

class VocabularioSimples:
    """Só o que `sugerir` usa: um objeto com .termos.in_order()."""
    def __init__(self, palavras):
        self.termos = BSTTermos()
        for p in palavras:
            self.termos.insert(p)

linhas = []
for tamanho in [200, 400, 800, 1600]:
    palavras = [palavra_aleatoria() for _ in range(tamanho)]
    vocab = VocabularioSimples(palavras)
    consultas = [com_erro_de_digitacao(random.choice(palavras)) for _ in range(4)]
    sem = com = 0
    for termo in consultas:
        s1, d1, c_sem = sugerir(termo, vocab, podar=False)
        s2, d2, c_com = sugerir(termo, vocab, podar=True)
        assert (s1, d1) == (s2, d2) # a poda não muda a resposta
        sem, com = sem + c_sem, com + c_com
    linhas.append({"vocabulário": vocab.termos.tamanho, "distâncias sem poda": sem,
                  "distâncias com poda": com, "evitadas": f"{1 - com / sem:.0%}",
                  "ganho": sem / com})
otim2 = pd.DataFrame(linhas)
otim2
```

Out[18]:

	vocabulário	distâncias sem poda	distâncias com poda	evitadas	ganho
0	199	796	185	77%	4.302703
1	400	1600	296	82%	5.405405
2	800	3200	828	74%	3.864734
3	1600	6400	1457	77%	4.392588

A poda **não muda a resposta** (o `assert` confere, inclusive nos empates) e evita a maior parte dos cálculos. Com um erro de uma letra, a melhor distância cai logo para 1, e só os termos com tamanho igual ou vizinho continuam sendo comparados. A fração evitada fica estável quando o vocabulário cresce: é um ganho de **constante** (a sugestão continua $O(V \cdot L^2)$), mas cada cálculo evitado é um $O(L^2)$ inteiro que não acontece.

10. Análise de custos

`D` documentos, `T` termos no total, `V` termos distintos, `h` a altura da BST, `L` o tamanho de um termo, `m` e `n` os tamanhos das listas de uma consulta, `r` o número de resultados:

etapa	custo	de onde vem
indexação	$O(T + V \cdot h)$	cada termo: $O(1)$ na hashtable e $O(1)$ no fim da lista encadeada (<code>tail</code>); cada termo nov o ainda desce a BST
listagem ordenada	$O(V)$	in-order na BST
busca de termo na BST	$O(h)$	$h \approx 2 \cdot \log_2 V$ aqui; $O(V)$ se degenerar
busca de termo na hashtable	$O(1)$ em média	é a que a consulta usa
consulta	$O(q + \Sigma \text{ tamanhos das listas})$	shunting-yard $O(q)$ nos <code>q</code> tokens; cada <code>AND / OR / NOT</code> é linear nas listas
ranqueamento	$O(\Sigma \text{ tamanhos das listas} + r \log r)$	uma passada por lista de ocorrências para pontuar, e o QuickSort nos <code>r</code> resultados
sugestão	$O(V \cdot L^2)$	uma distância de edição $O(L^2)$ por termo do vocabulário (a poda reduz a constante)
relatório BFS	$O(V)$	cada nó entra e sai da fila uma vez

Relatório técnico de desempenho

Meço as quatro etapas num corpus sintético maior, dobrando o número de documentos:

In [19]:

```
def corpus_sintetico(qtd, vocabulario, palavras_por_doc=40):
    return " ".join(random.choice(vocabulario) for _ in range(palavras_por_doc)) for _ in range(qtd)]

base = [palavra_aleatoria() for _ in range(400)]
linhas = []
for qtd in [100, 200, 400, 800]:
    docs = corpus_sintetico(qtd, base)

    t_indexar = float("inf")
    for _ in range(3):
        inicio = time.perf_counter()
        idx = IndiceInvertido(docs)
        t_indexar = min(t_indexar, time.perf_counter() - inicio)

a, b, c = base[0], base[1], base[2]
```

```

inicio = time.perf_counter()
resultado, _, comparacoes_consulta = consultar(f"{{a}} OR {{b}} AND NOT {{c}}", idx)
t_consultar = time.perf_counter() - inicio

t_ranquear = t_sugerir = float("inf")
for _ in range(3):
    inicio = time.perf_counter()
    ranking, _ = buscar(f"{{a}} OR {{b}}", idx)
    t_ranquear = min(t_ranquear, time.perf_counter() - inicio)
    inicio = time.perf_counter()
    _, _, distancias = sugerir(a[:-1] + "x", idx)
    t_sugerir = min(t_sugerir, time.perf_counter() - inicio)

linhas.append({"documentos": qtd, "termos (T)": idx.total_tokens, "vocabulário (V)": idx.termos.tamanho,
              "indexar_ms": t_indexar * 1000, "indexar_us_por_termo": t_indexar / idx.total_tokens * 1e6,
              "consulta_comparacoes": comparacoes_consulta, "consulta_ms": t_consultar * 1000,
              "resultados (r)": len(ranking), "ranquear_ms": t_ranquear * 1000,
              "sugerir_distancias": distancias, "sugerir_ms": t_sugerir * 1000})

relatorio = pd.DataFrame(linhas)
relatorio

```

Out[19]:

	documentos	termos (T)	vocabulário (V)	indexar_ms	indexar_us_por_termo	consulta_comparacoes	consulta_ms	resultados (r)	ranquear_ms	sugerir_distancias	sugerir_ms
0	100	4000	400	28.9673	7.241825	124	0.0872	15	0.0786	106	45.2194
1	200	8000	400	55.1512	6.893900	268	0.1923	40	0.2200	106	45.6978
2	400	16000	400	110.5279	6.907994	519	0.1851	83	0.2757	106	44.4021
3	800	32000	400	274.0171	8.563034	1007	0.3573	125	0.4310	106	43.9892

Leitura do relatório:

- **Indexação:** o tempo **por termo** fica na mesma faixa enquanto T cresce 8 vezes — se a indexação fosse quadrática, ele teria crescido 8 vezes junto. É linear em T . O vocabulário aqui para de crescer em 400 (todas as palavras da base já apareceram), então o termo $V \cdot h$ da BST deixa de pesar.
- **Consulta:** as comparações crescem na mesma proporção que os documentos, porque as listas de ocorrências crescem junto: linear no tamanho das listas.
- **Ranqueamento:** cresce junto com o número de resultados r . Na primeira versão ele crescia **mais rápido** que isso: para cada documento do resultado, a pontuação percorria a lista de ocorrências do termo até achar o documento — $O(r \cdot \text{tamanho da lista})$, que vira $O(D^2)$ quando as duas coisas crescem com o número de documentos. Troquei por uma passada só em cada lista (`pontuar_todos`), e o custo passou a ser linear no tamanho das listas.
- **Sugestão:** depende só do vocabulário, que ficou fixo em 400 — tanto que `sugerir_distancias` é o mesmo número em todas as linhas (o tempo oscila um pouco, a contagem não).

Gargalo: a sugestão é a etapa mais cara por termo ($O(V \cdot L^2)$), mesmo com a poda por tamanho. Se o vocabulário crescesse muito, o próximo passo seria indexar os termos por pedaços de 2 ou 3 letras (n-gramas) e só calcular a distância para os termos que compartilham pedaços com o termo digitado.

11. Casos de borda

In [20]:

```
invalidas = ["", " ", "busca AND", "AND busca", "(busca OR fila", "busca OR fila)", "busca fila", "NOT"]
for consulta in invalidas:
    try:
        consultar(consulta, indice)
        raise AssertionError(f"deveria recusar {consulta!r}")
    except ConsultaInvalidaError as erro:
        print(f"{consulta!r:<18} → {erro}")

# termo inexistente dá lista vazia, não erro; e combina normalmente
assert consultar("inexistente", indice)[0] == []
assert consultar("inexistente OR fila", indice)[0] == consultar("fila", indice)[0]
assert consultar("NOT inexistente", indice)[0] == list(range(len(DOCUMENTOS)))

# acento e maiúscula na consulta não atrapalham
assert consultar("BINÁRIA", indice)[0] == consultar("binaria", indice)[0]

# índice de coleção vazia
vazio = IndiceInvertido([])
assert vazio.terminos.tamanho == 0 and consultar("qualquer", vazio)[0] == []

# ranqueamento com um resultado só e sem resultado
assert len(buscar("quicksort", indice)[0]) == 1
assert buscar("inexistente", indice)[0] == []

# sugestão para um termo que existe: distância 0, ele mesmo
assert sugerir("fila", indice)[:2] == ("fila", 0)

print("\ntodos os casos de borda passaram")
```

```
' '          → consulta vazia
' '          → consulta vazia
'busca AND'   → AND precisa de um termo de cada lado
'AND busca'   → AND precisa de um termo de cada lado
'(busca OR fila' → tem '(' sem o ')' correspondente
'busca OR fila)' → ')' na posição 4 não tem '(' correspondente
'busca fila'   → falta um operador entre 'busca' e 'fila' (posição 2)
'NOT'         → NOT sem termo depois dele
```

todos os casos de borda passaram

12. Conferência das oito peças obrigatórias

O enunciado exige a integração de hashtable, pilha, fila, recursão, programação dinâmica, ordenação eficiente, lista encadeada e árvore. Confiro que cada uma está **de fato** no caminho que o motor usa:

In [21]:

```
cont = {"comparacoes": 0, "chamadas": 0}
quicksort([(1, 0), 0], [(0, 1), 1], [(2, 2), 2], cont)

memo = HashTableChained()
distancia_edicao("fila", "filha", {"chamadas": 0}, memo)

pecas = [
    ("hashtable", "índice invertido: termo → lista", isinstance(indice.tabela, HashTableChained)),
```

```
("lista encadeada", "ocorrências (doc_id, posições)", isinstance(indice.tabela.get("busca"), ListaOcorrencias)),
("árvore", "BST de termos", isinstance(indice.termos, BSTTermos) and indice.termos.tamanho > 0),
("pilha", "shunting-yard e avaliação", para_posfixa(["a", "AND", "b"]) == ["a", "b", "AND"]),
("fila", "BFS do relatório de diagnóstico", len(relatorio_bfs(indice.termos)["por_nivel"]) > 1),
("recursão", "BST, QuickSort e distância de edição", cont["chamadas"] > 1),
("programação dinâmica", "distância de edição com memo", len(memo) > 0),
("ordenação eficiente", "QuickSort no ranqueamento", cont["comparacoes"] > 0),
]
for nome, onde, ok in pecas:
    assert ok, nome
pd.DataFrame(pecas, columns=["peça", "onde é usada", "conferida"])
```

Out[21]:

	peça	onde é usada	conferida
0	hashtable	índice invertido: termo → lista	True
1	lista encadeada	ocorrências (doc_id, posições)	True
2	árvore	BST de termos	True
3	pilha	shunting-yard e avaliação	True
4	fila	BFS do relatório de diagnóstico	True
5	recursão	BST, QuickSort e distância de edição	True
6	programação dinâmica	distância de edição com memo	True
7	ordenação eficiente	QuickSort no ranqueamento	True